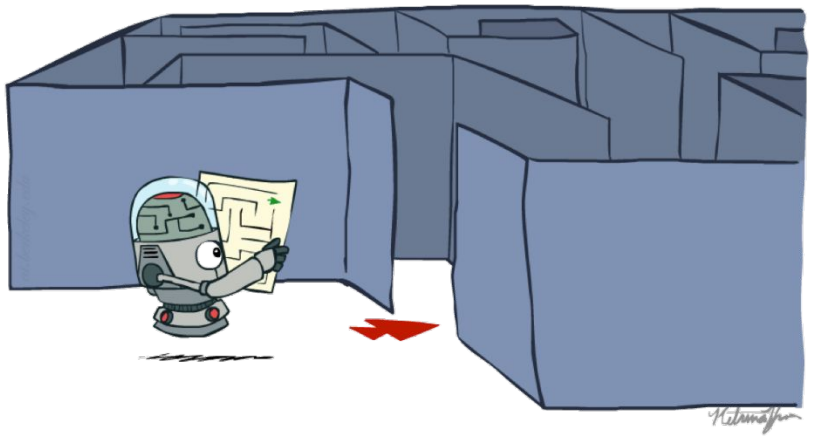
Problem Solving Using Search

Introduction to State Space Search



Slide credits: AI BERKELEY

Previous Lecture

- Intelligent Agent
- Performance measure
- Different types of Environments
- Agent types
  - Simple reflex agents
  - Reflex agents with state/model
  - Goal-based agents
  - Utility-based agents

Problem Solving

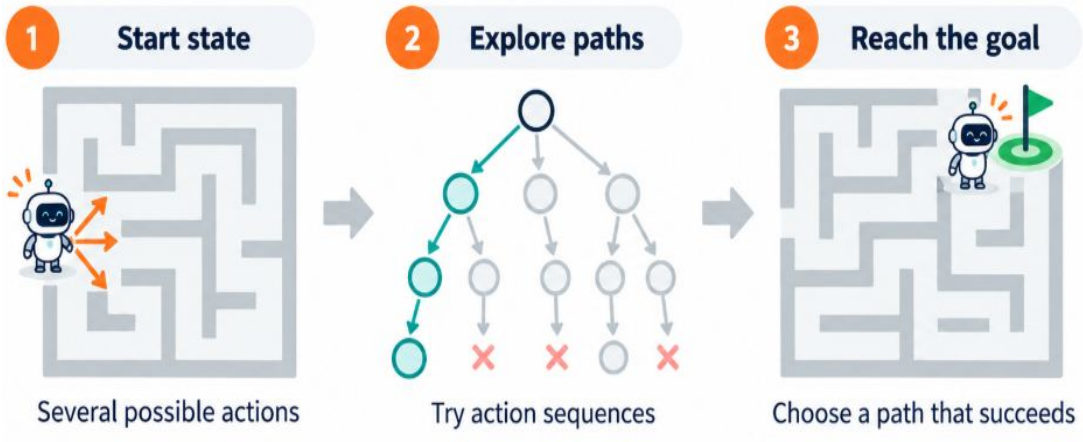
How does an agent actually decide what sequence of actions to take to reach a goal?”

Current State $\rightarrow$??? $\rightarrow$ Goal State

- The “???” part is called **SEARCH**.

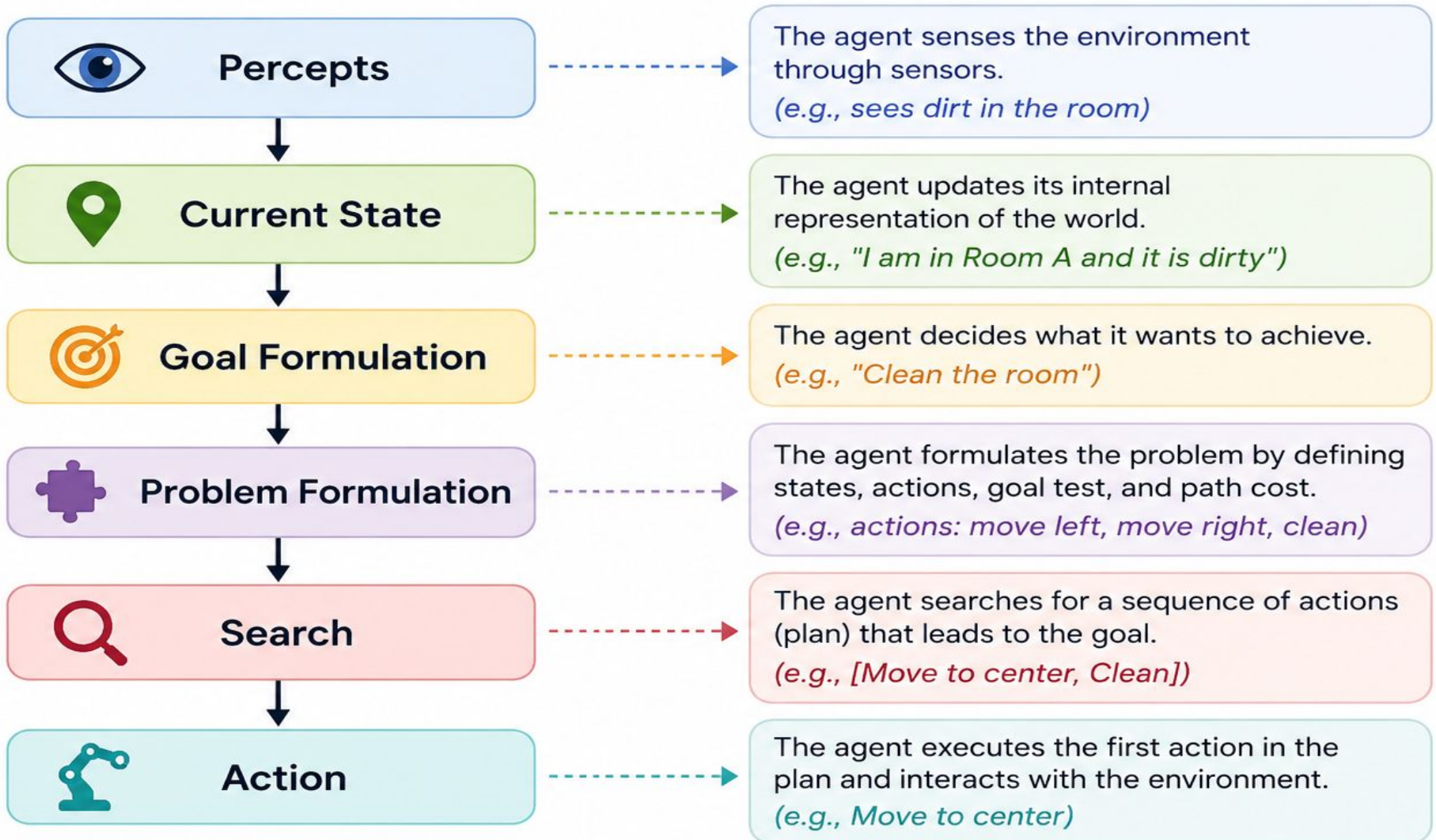
Introduction

- Suppose an agent can execute several actions immediately in a given state
- It doesn't know the utility of these actions
- Then, for each action, it can execute a sequence of actions until it reaches the goal
- The immediate action which has the best sequence (according to the performance measure) is then the solution
- Finding this sequence of actions is called search, and the agent which does this is called the problem-solver.
- Its possible that some sequence might fail, e.g., getting stuck in an infinite loop, or unable to find the goal at all.



Problem Solving Agent

The steps followed by a problem solving agent to achieve its goal



Problem-Solving Agent

- **Problem solving agent:**
  - Is goal-based
  - Does not know immediately what to do
  - Tries different sequences of actions
  - Picks the sequence that reaches the goal

- Many problems can be represented as a **set of states** and a **set of rules** of how one state is transformed to another.

Examples

- Getting from home to UCP
  - **START**: home location
  - **GOAL**: UCP FIT Department.
  - **OPERATORS**: move one block, turn
- Loading a moving truck
  - **START**: apartment full of boxes and furniture
  - **GOAL**: empty apartment, all boxes and furniture in the truck
  - **OPERATORS**: select item, carry item from apartment to truck, load item
- Getting settled
  - **START**: items randomly distributed over the place
  - **GOAL**: satisfactory arrangement of items
  - **OPERATORS**: select item, move item

Example Problems

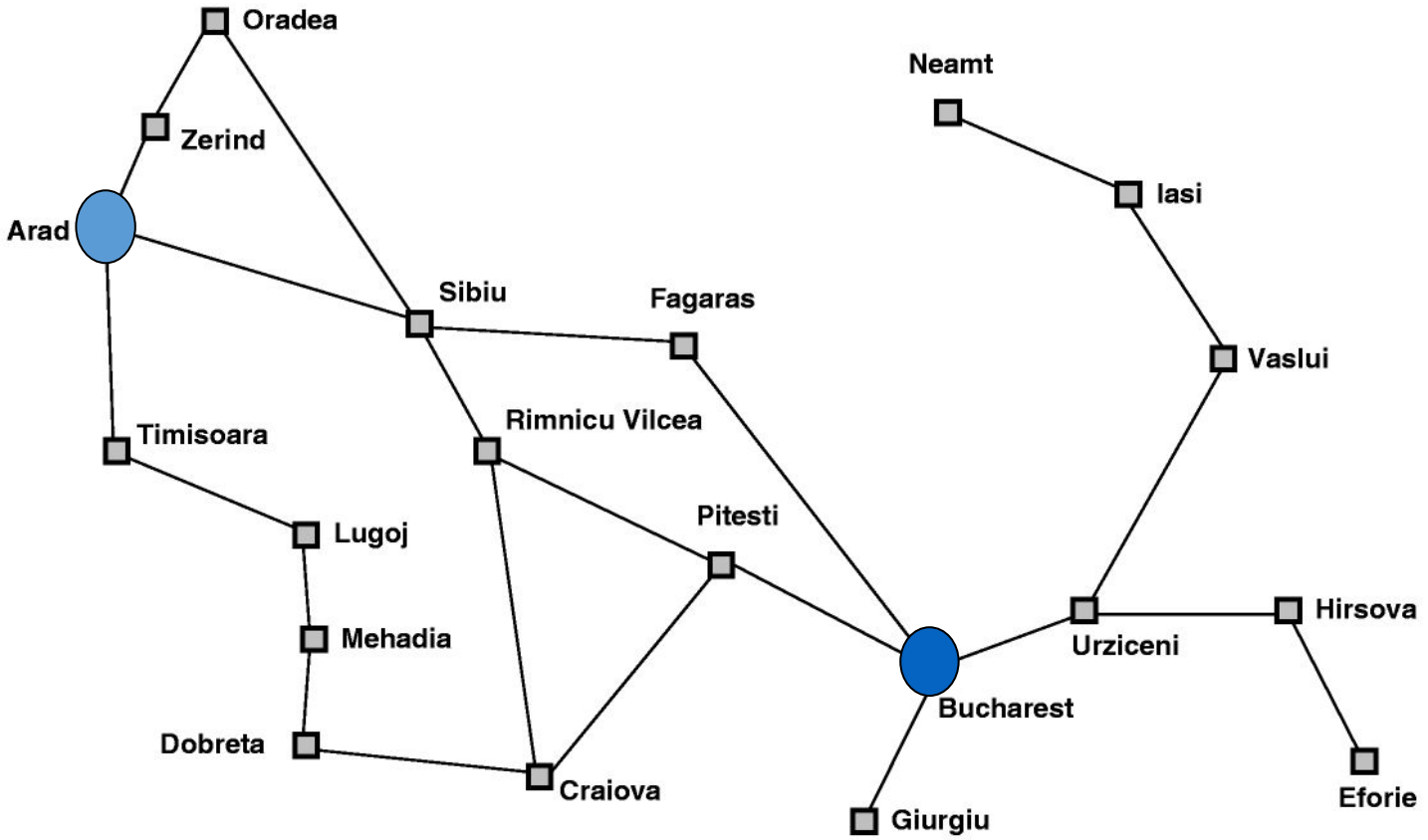
- **Toy Problems**
  - vacuum world
    - Two rooms A and B, Each room may be clean or dirty. Agent can turn Left, turn right, and clean by suction
  - 8-puzzle
    - 8 numbered tiles, one empty space, arrange numbers in order
  - 8-queens
    - Place 8 queens on a chessboard such that no queen attacks another
  - vacuum agent
    - Multiple rooms, more dirt locations, possibly partially observable
  - missionaries and cannibals
    - 3 missionaries and 3 cannibals must cross a river.
    - Rules:
      - Boat holds 2 people max.
      - Cannibals can’t outnumber missionaries on either side.
    - Goal: Move everyone safely to the other side

Example Problems

- **Real-world Problems**
  - route finding
    - Finding shortest path from Lahore to Islamabad.
      - GPS Systems, Google Maps
  - touring problems
    - traveling salesperson
      - Visit all cities exactly once and return to start.
      - Goal:
        Minimize total distance.
      - Used in Delivery optimization, airline planning
  - Very Large Scale Integration (VLSI) layout
    - Goal:
      Place components efficiently:
    - Minimize wire length
    - Reduce delay
    - Used in processor design, chip manufacturing
  - robot navigation
    - Robot must move from start to goal:
    - Avoid obstacles
    - Minimize time or distance
    - Used in Self driving cars, drones
  - assembly sequencing
    - Minimize time
    - Optimize efficiency
    - Used in car manufacturing

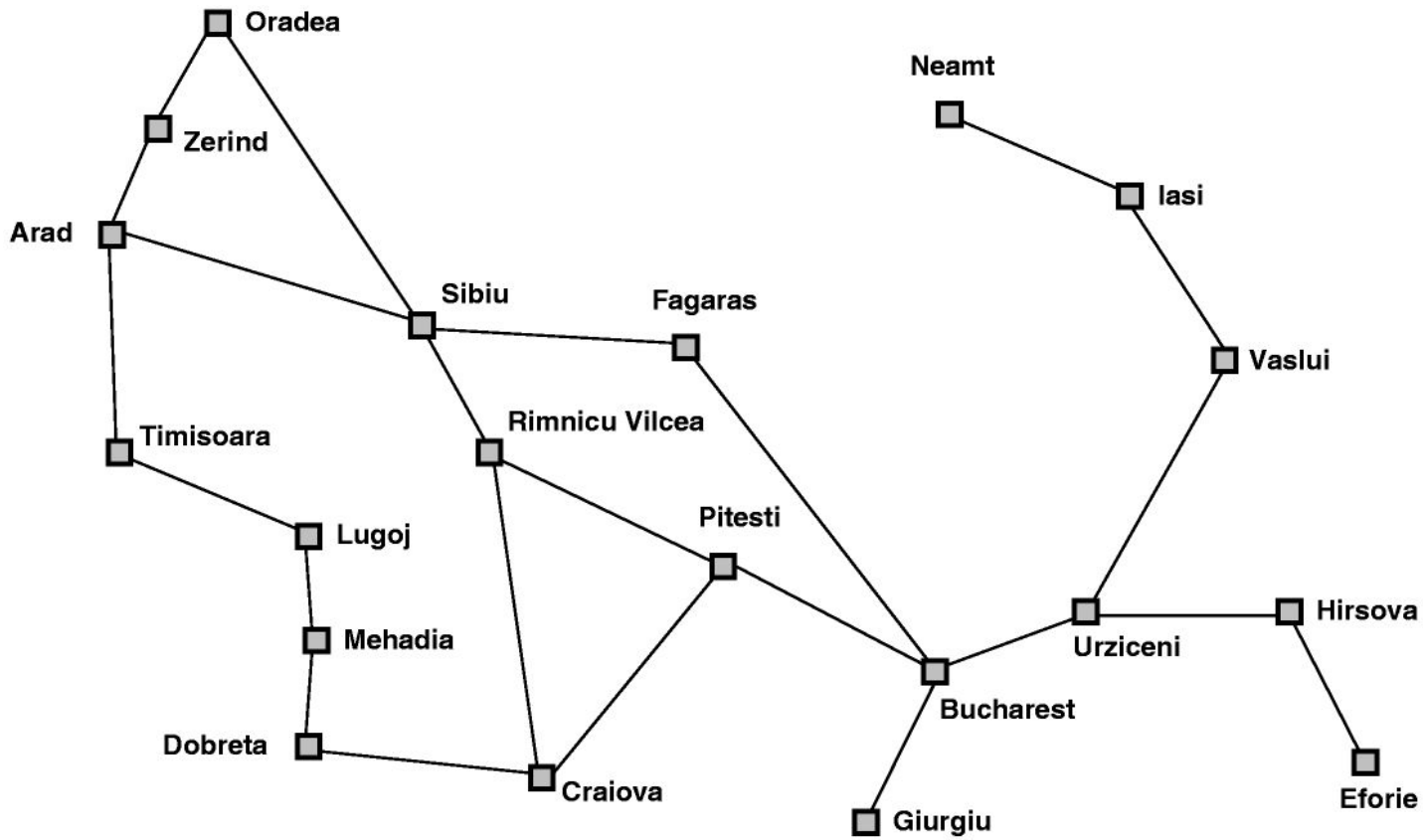
Classical AI Search Problem

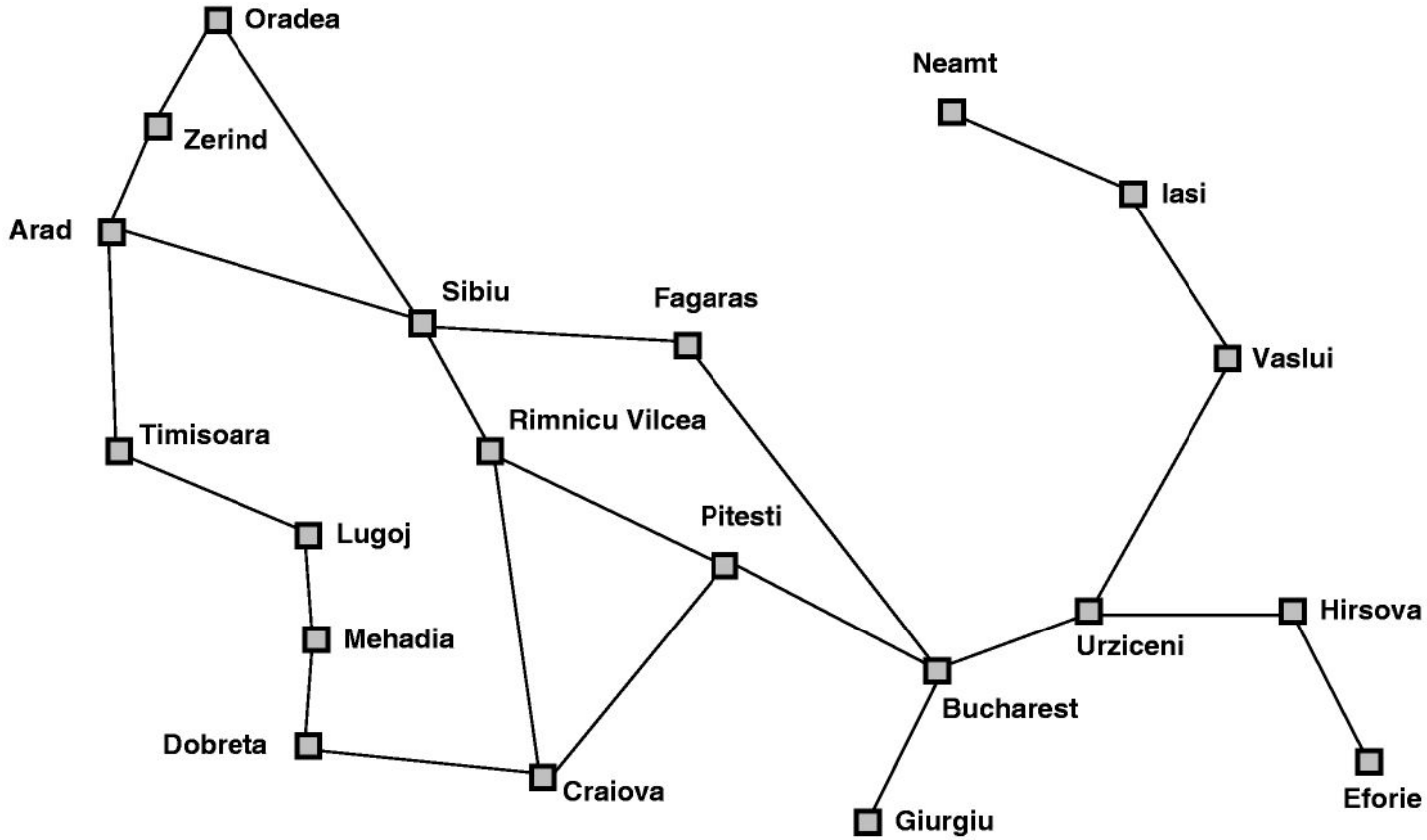
- Touring Agent Problem (Map Search / Navigation)



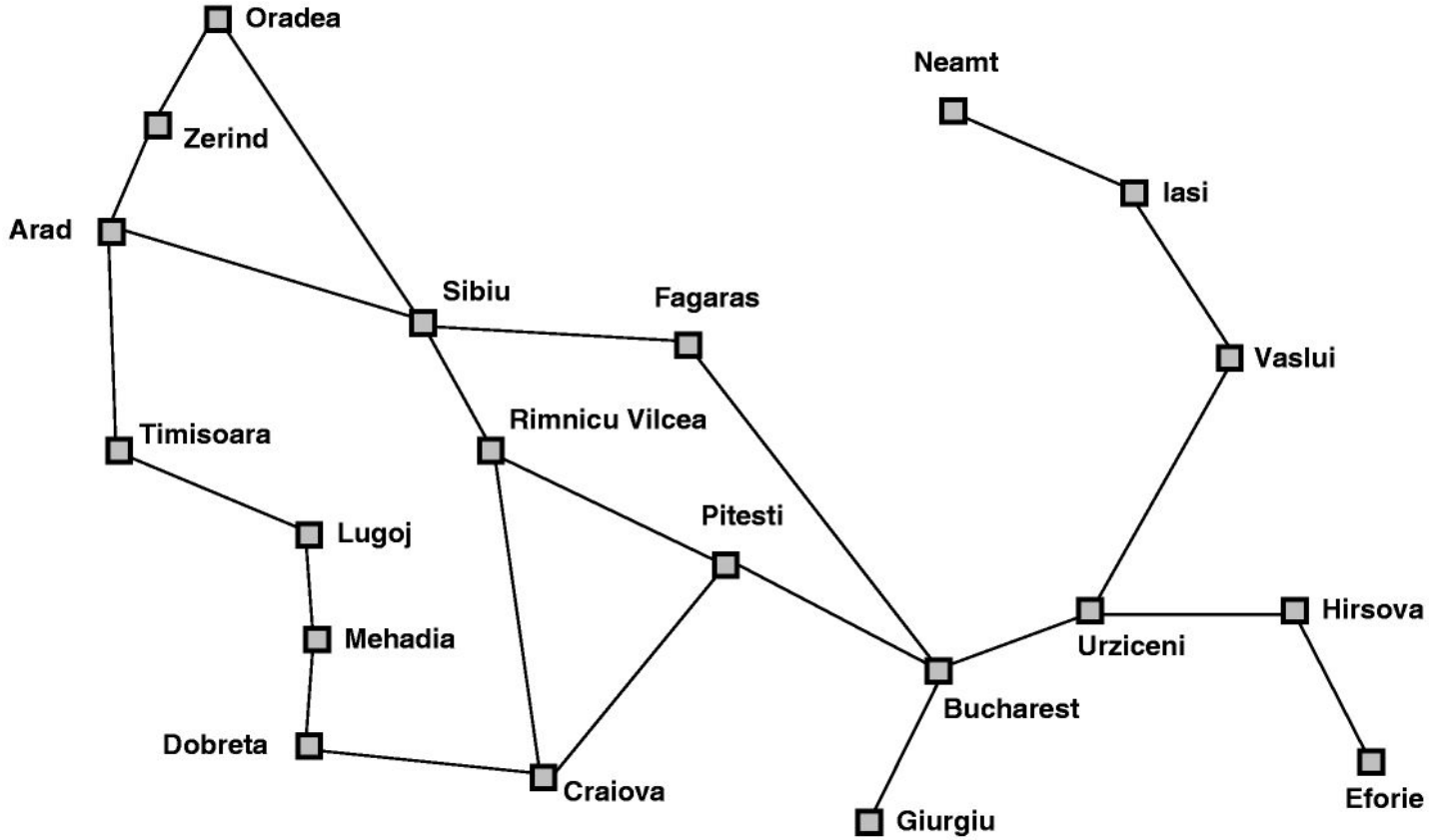
A **problem** can be defined formally by five components:

- The **initial state** that the agent starts in. For example, the initial state for our agent in Romania might be described as $In(Arad)$.



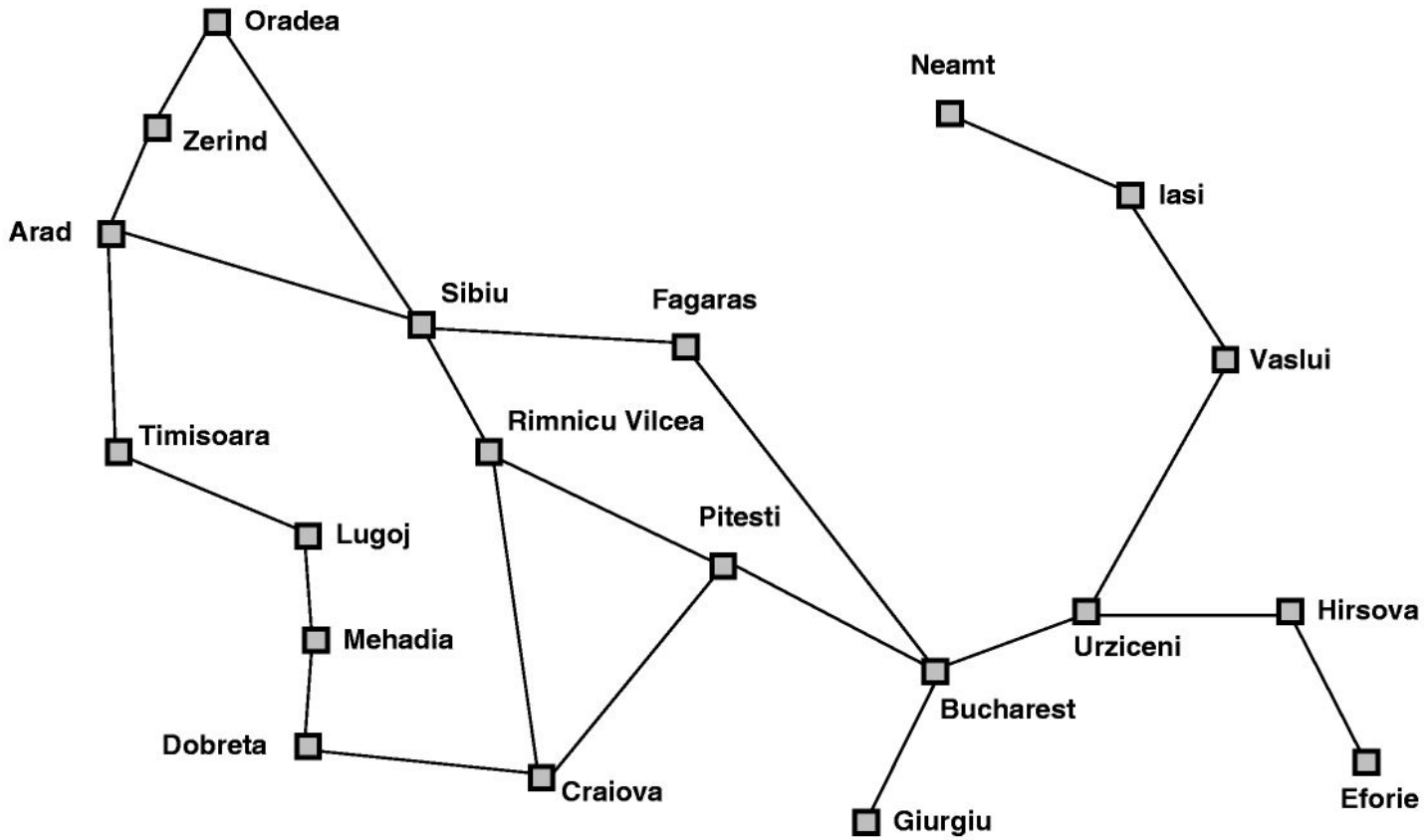


- A description of the possible **actions** available to the agent. Given a particular state s, ACTIONS(s) returns the set of actions that can be executed in s. We say that each of these actions is **applicable** in s. For example, from the state $In(Arad)$, the applicable actions are $\{Go(Sibiu), Go(Timisoara), Go(Zerind)\}$.

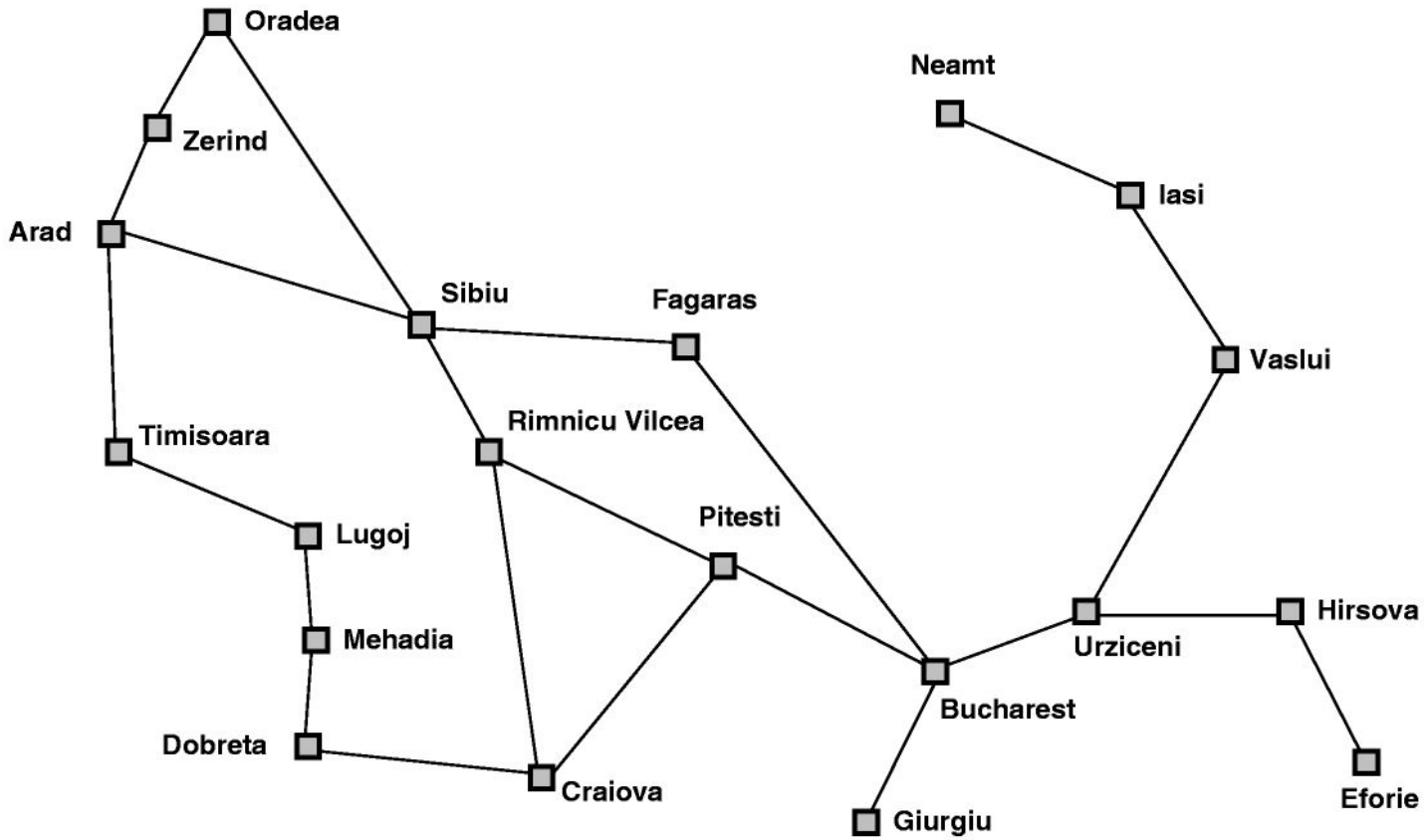


- A description of what each action does; the formal name for this is the **transition model**, specified by a function RESULT(s, a) that returns the state that results from doing action a in state s. We also use the term **successor** to refer to any state reachable from a given state by a single action.[2] For example, we have

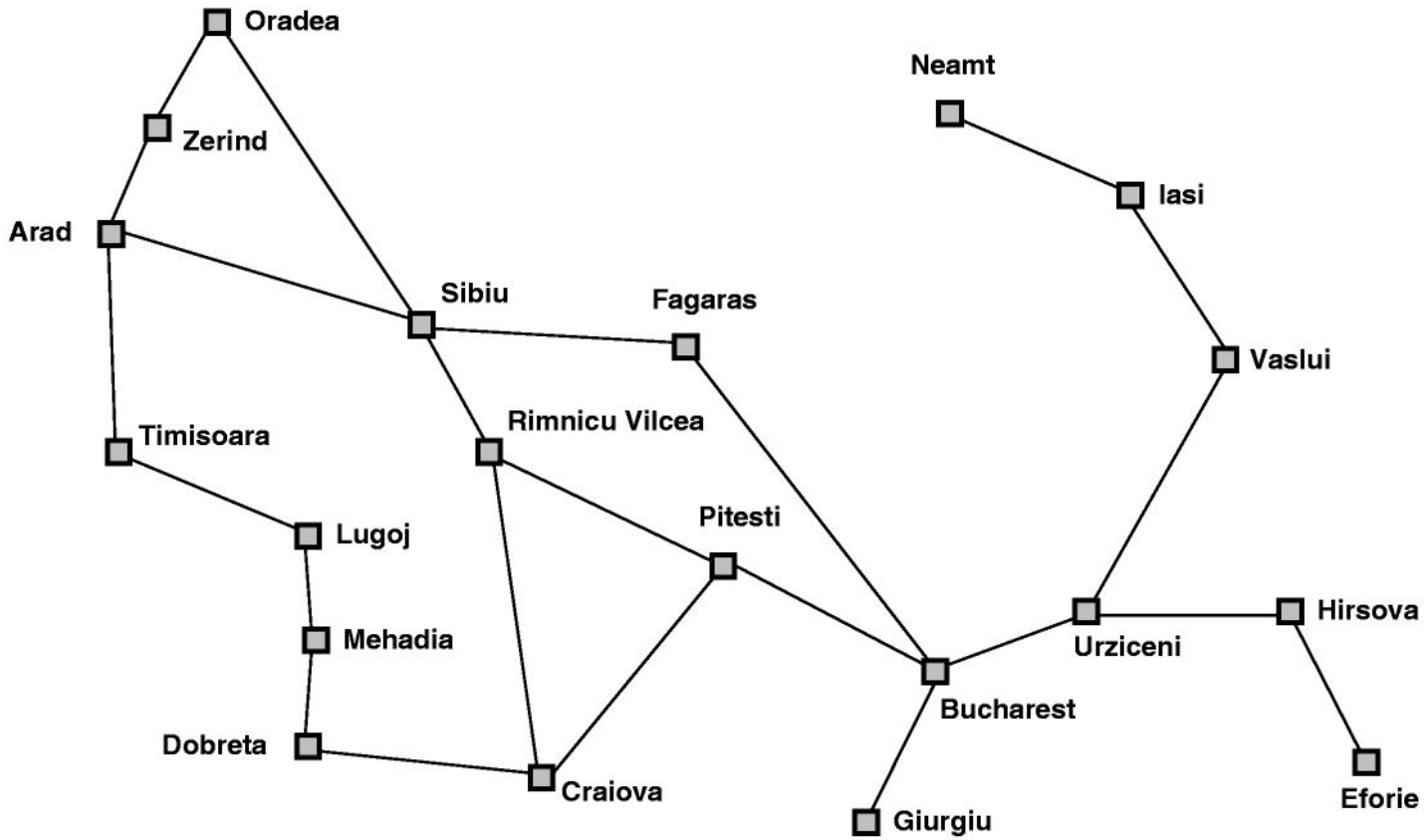
$$\text{RESULT}(In(Arad), Go(Zerind)) = In(Zerind) .$$



- The **goal test**, which determines whether a given state is a goal state. Sometimes there is an explicit set of possible goal states, and the test simply checks whether the given state is one of them. The agent's goal in Romania is the singleton set $\{In(Bucharest)\}$.



- A **path cost** function that assigns a numeric cost to each path. The problem-solving agent chooses a cost function that reflects its own performance measure. For the agent trying to get to Bucharest, time is of the essence, so the cost of a path might be its length in kilometers. In this chapter, we assume that the cost of a path can be described as the *sum* of the costs of the individual actions along the path.[3] The **step cost** of taking action a in state s to reach state s' is denoted by $c(s, a, s')$. The step costs for Romania are shown in Figure 3.2 as route distances. We assume that step costs are nonnegative.[4]



The preceding elements define a problem and can be gathered into a single data structure that is given as input to a problem-solving algorithm. A **solution** to a problem is an action sequence that leads from the initial state to a goal state. Solution quality is measured by the path cost function, and an **optimal solution** has the lowest path cost among all solutions.

Structure of Problem solving agent

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
              state, some description of the current world state
              goal, a goal, initially null
              problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
      goal ← FORMULATE-GOAL(state)
      problem ← FORMULATE-PROBLEM(state, goal)
      seq ← SEARCH(problem)
      if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

Explanation of problem solving agent

- Imagine a robot vacuum:
  1. Perceives room is dirty (percept)
  2. Updates state to "room dirty"
  3. Formulates goal "clean room"
  4. Creates problem (current location, actions: move, clean)
  5. Searches for action sequence (e.g., [move to center, clean])
  6. Executes first action (move to center)

Toy Example-1

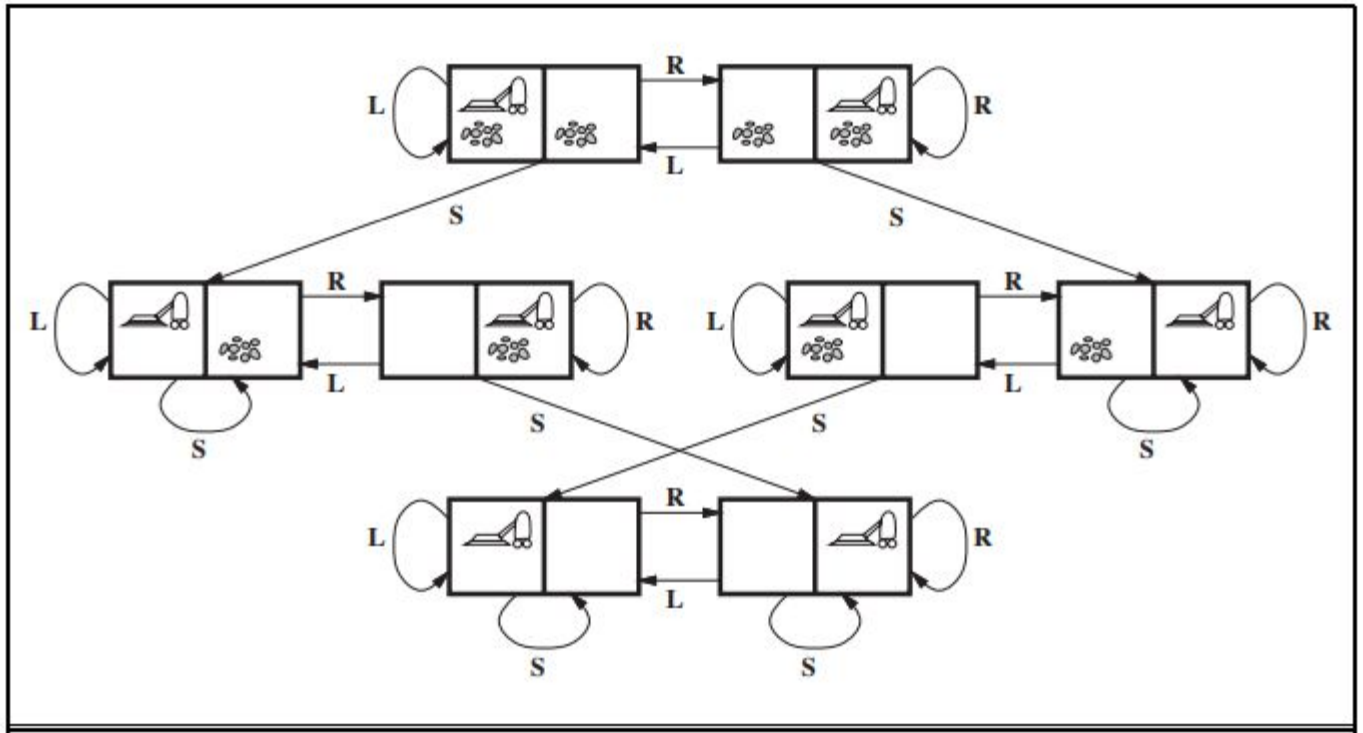
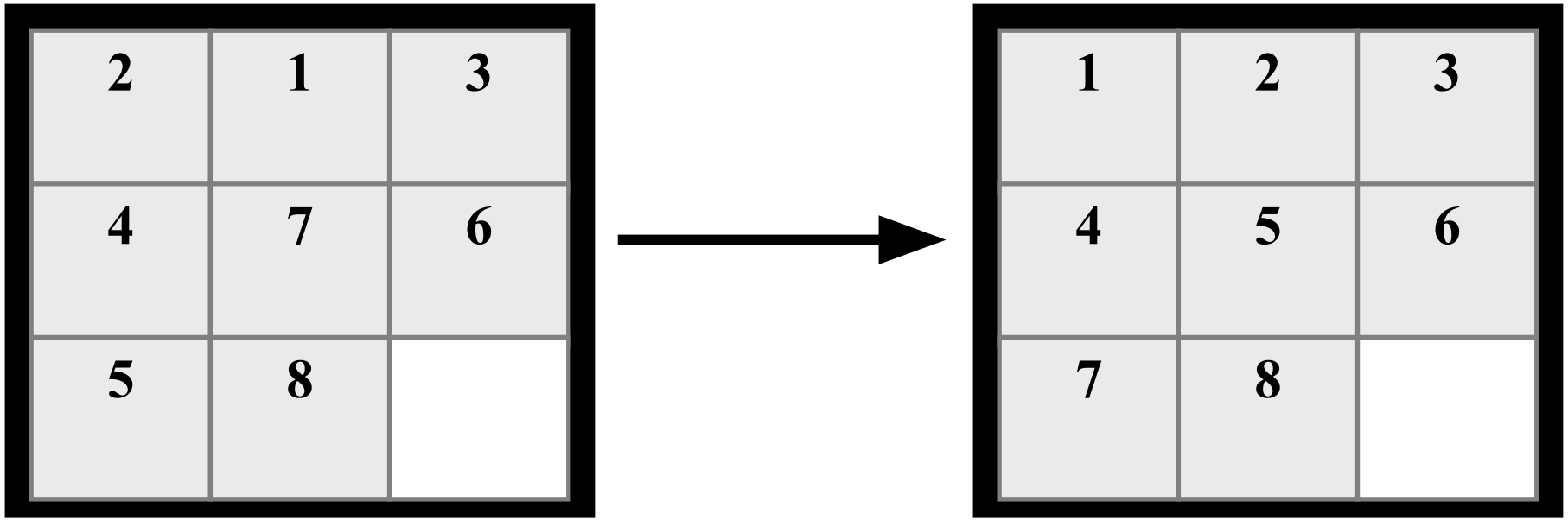


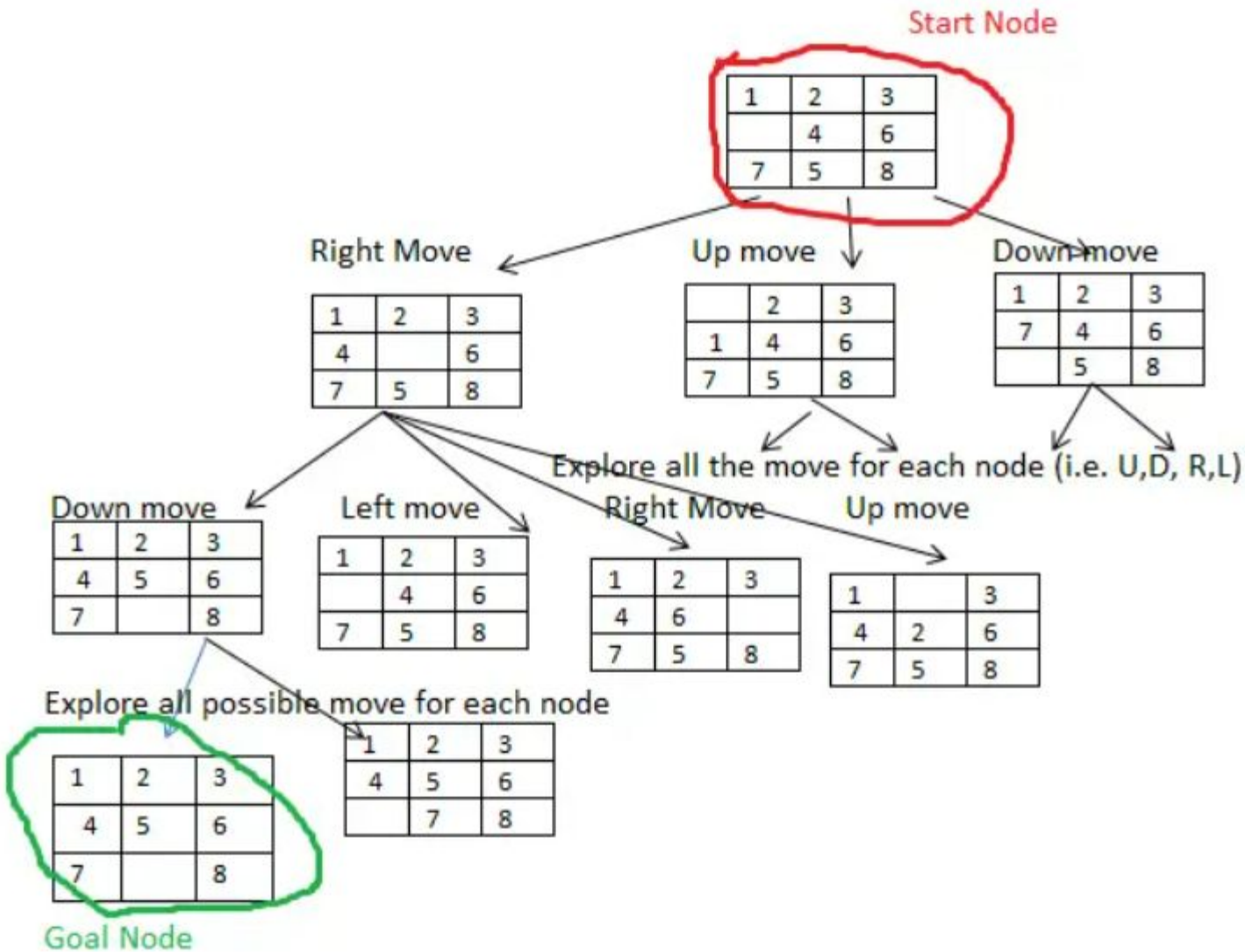
Figure 3.3 The state space for the vacuum world. Links denote actions: L = *Left*, R = *Right*, S = *Suck*.

- **States**: The state is determined by both the agent location and the dirt locations. The agent is in one of two locations, each of which might or might not contain dirt. Thus, there are $2 \times 2^2 = 8$ possible world states. A larger environment with n locations has $n \cdot 2^n$ states.
- **Initial state**: Any state can be designated as the initial state.
- **Actions**: In this simple environment, each state has just three actions: *Left*, *Right*, and *Suck*. Larger environments might also include *Up* and *Down*.
- **Transition model**: The actions have their expected effects, except that moving *Left* in the leftmost square, moving *Right* in the rightmost square, and *Suck*ing in a clean square have no effect. The complete state space is shown in Figure 3.3.
- **Goal test**: This checks whether all the squares are clean.
- **Path cost**: Each step costs 1, so the path cost is the number of steps in the path.

Classical AI Search Problem

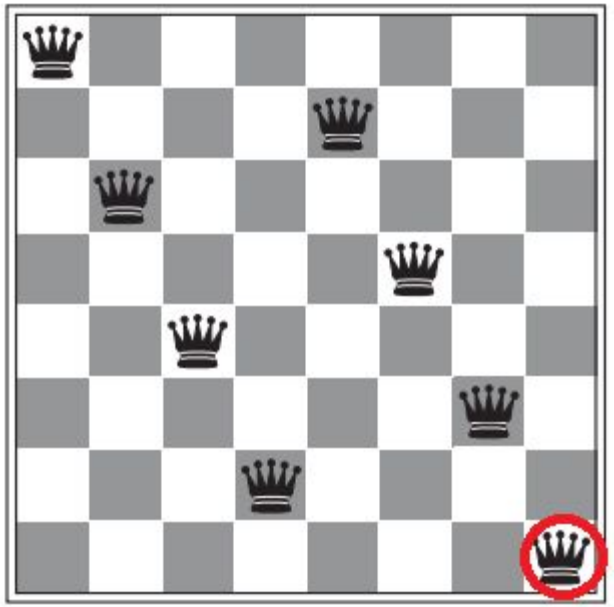
- 8-Puzzle




Start Node
1 2 3
4 6
7 5 8
Right Move
1 2 3
4 6
7 5 8
Up move
2 3
1 4 6
7 5 8
Down move
1 2 3
7 4 6
5 8
Explore all the move for each node (i.e. U,D, R,L)
Down move
1 2 3
4 5 6
7 8
Left move
1 2 3
4 6
7 5 8
Right Move
1 2 3
4 6
7 5 8
Up move
1 3
4 2 6
7 5 8
Explore all possible move for each node
1 2 3
4 5 6
7 8
1 2 3
4 5 6
7 8
Goal Node

- **States**: A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.
- **Initial state**: Any state can be designated as the initial state. Note that any given goal can be reached from exactly half of the possible initial states (Exercise 3.4).
- **Actions**: The simplest formulation defines the actions as movements of the blank space *Left*, *Right*, *Up*, or *Down*. Different subsets of these are possible depending on where the blank is.
- **Transition model**: Given a state and action, this returns the resulting state; for example, if we apply *Left* to the start state in Figure 3.4, the resulting state has the 5 and the blank switched.
- **Goal test**: This checks whether the state matches the goal configuration shown in Figure 3.4. (Other goal configurations are possible.)
- **Path cost**: Each step costs 1, so the path cost is the number of steps in the path.

8 Queen Problem



- In chess, a **queen** can move:
  - Horizontally (row)
  - Vertically (column)
  - Diagonally (both directions)
- So two queens are **attacking each other** if they are placed:
  1. In the **same row**
  2. In the **same column**
  3. On the **same diagonal**

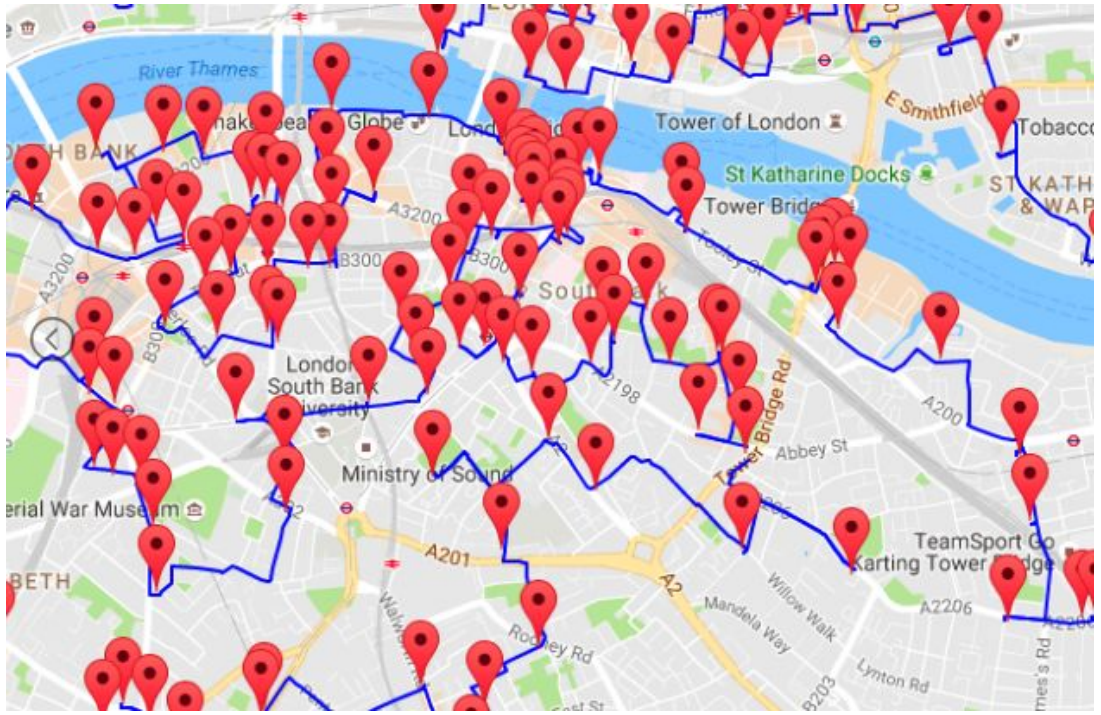
- **States**: Any arrangement of 0 to 8 queens on the board is a state.
- **Initial state**: No queens on the board.
- **Actions**: Add a queen to any empty square.
- **Transition model**: Returns the board with a queen added to the specified square.
- **Goal test**: 8 queens are on the board, none attacked.

In this formulation, we have $64 \cdot 63 \cdots 57 \approx 1.8 \times 10^{14}$ possible sequences to investigate. A better formulation would prohibit placing a queen in any square that is already attacked:

- **States**: All possible arrangements of n queens $(0 \leq n \leq 8)$, one per column in the leftmost n columns, with no queen attacking another.
- **Actions**: Add a queen to any square in the leftmost empty column such that it is not attacked by any other queen.

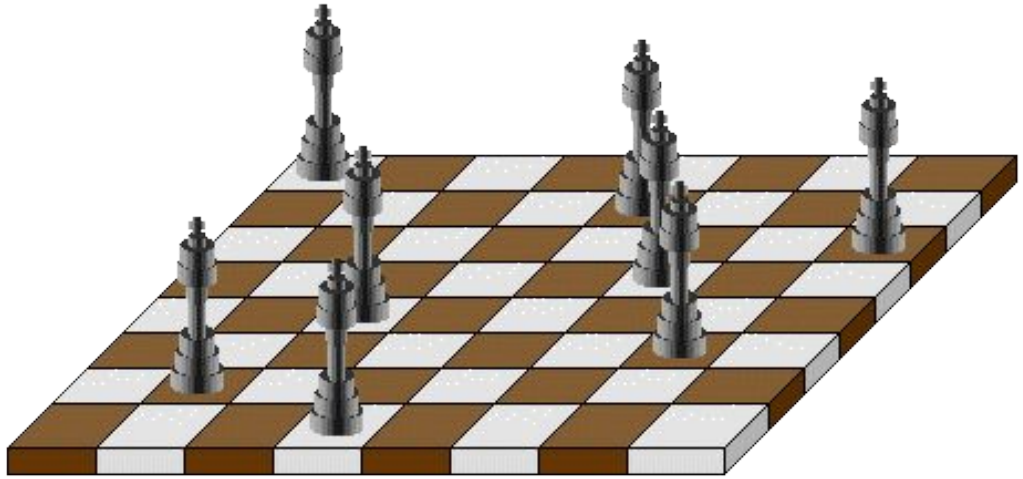
Classical AI Search Problem

- **Travelling Salesperson Problem (TSP)**
  - Suppose a salesman has five cities to visit and then must return home.
  - The goal of the problem is to find the shortest path for salesman to travel, visiting each city, and then returning to the starting city.

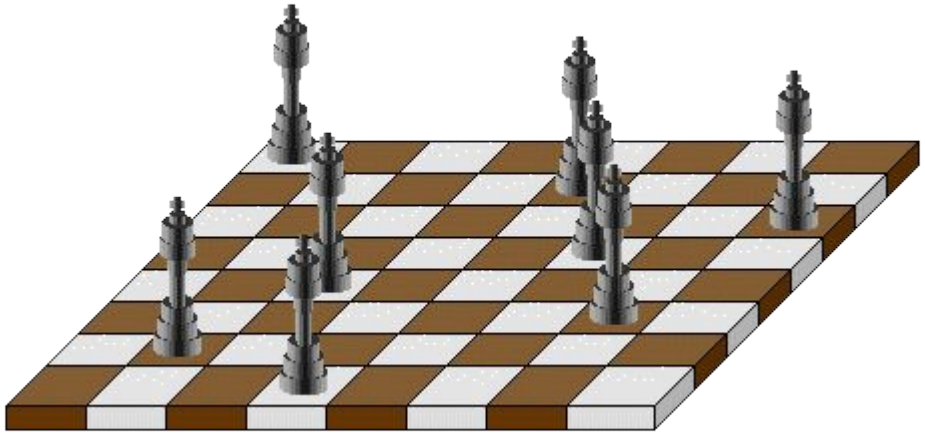


Classical AI Search Problem

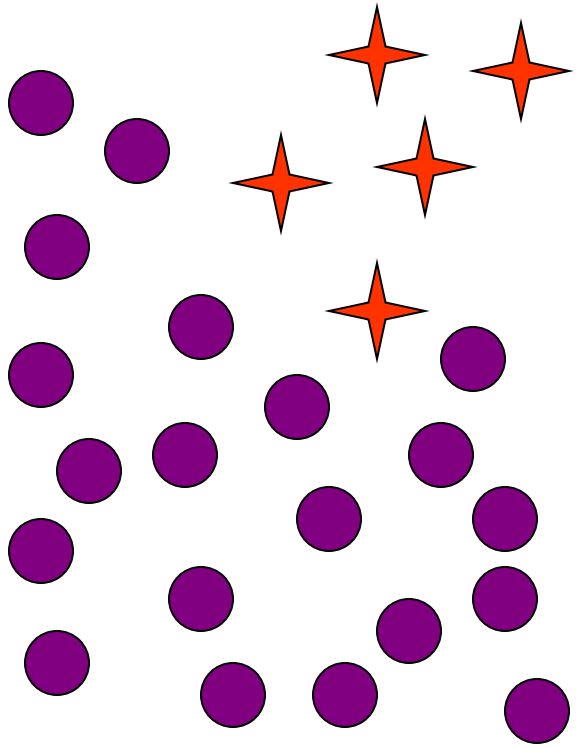
- N-Queens
  - Problem of placing *n* chess queens on an *n*×*n* chessboard so that no two queens attack each other
  - A solution requires that no two queens share the same row, column, or diagonal

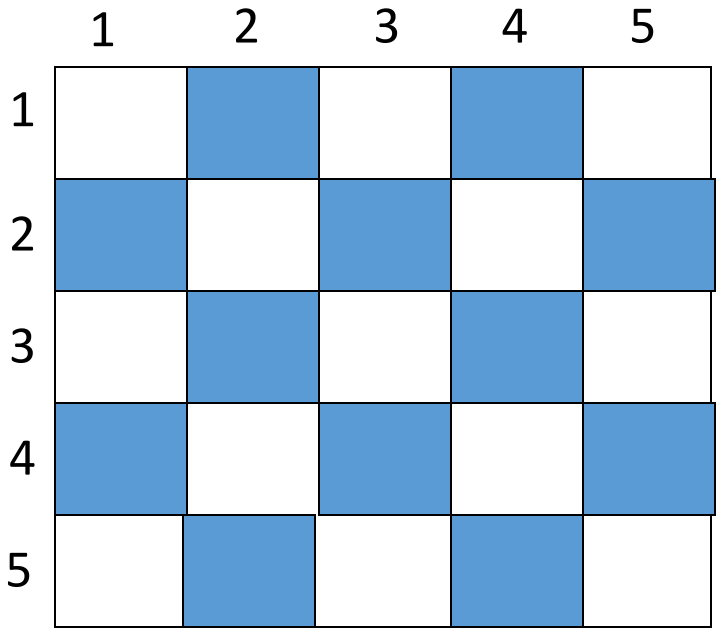


Classic AI Search Problems

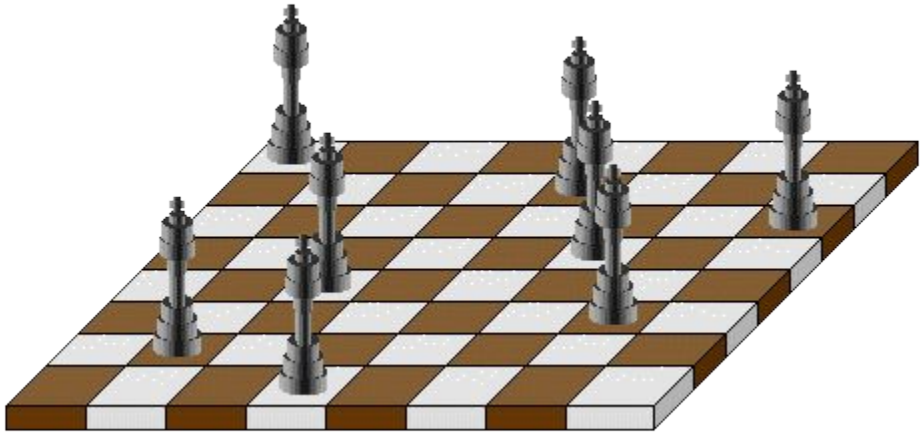


- 5-Queens:

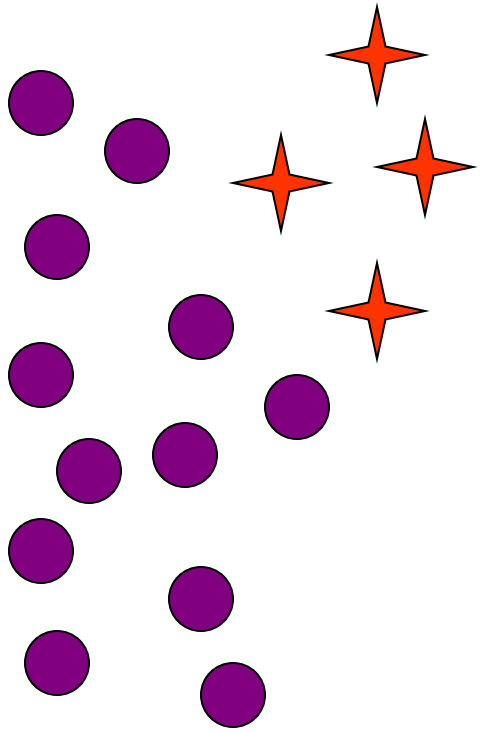


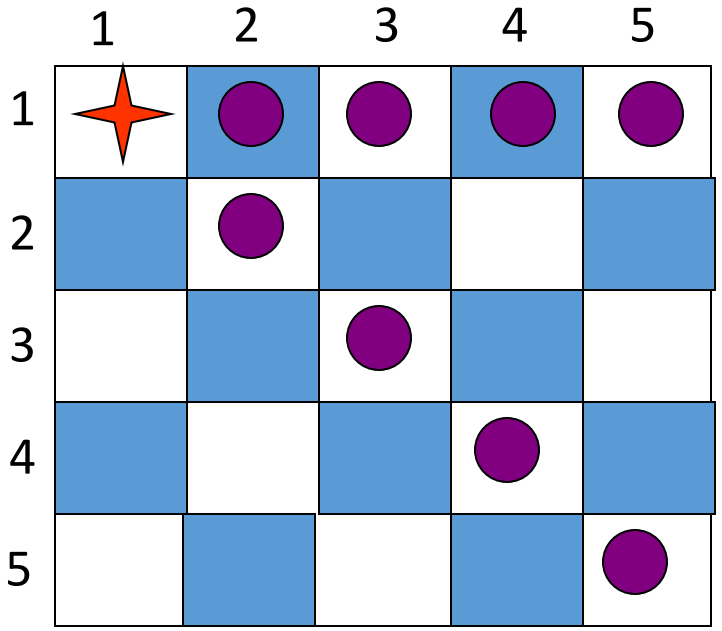


Classic AI Search Problems

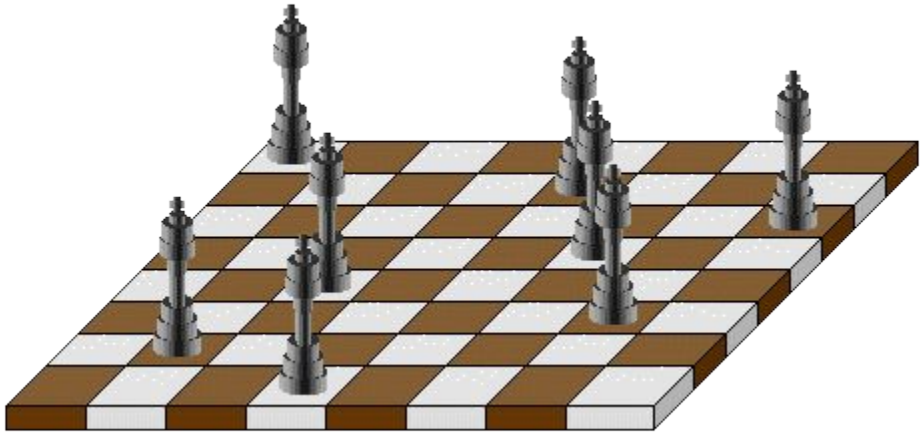


- 5-Queens:

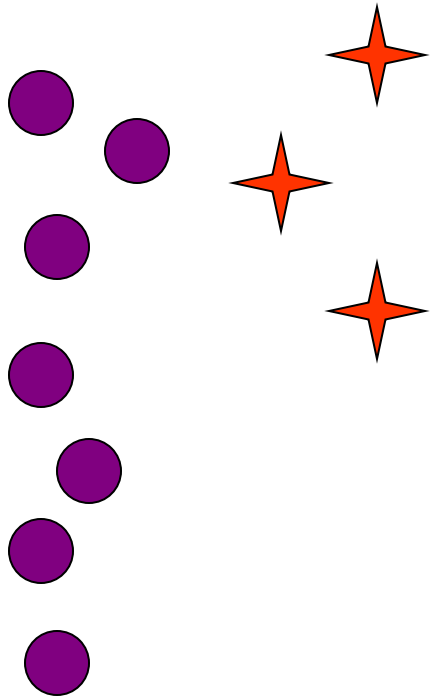


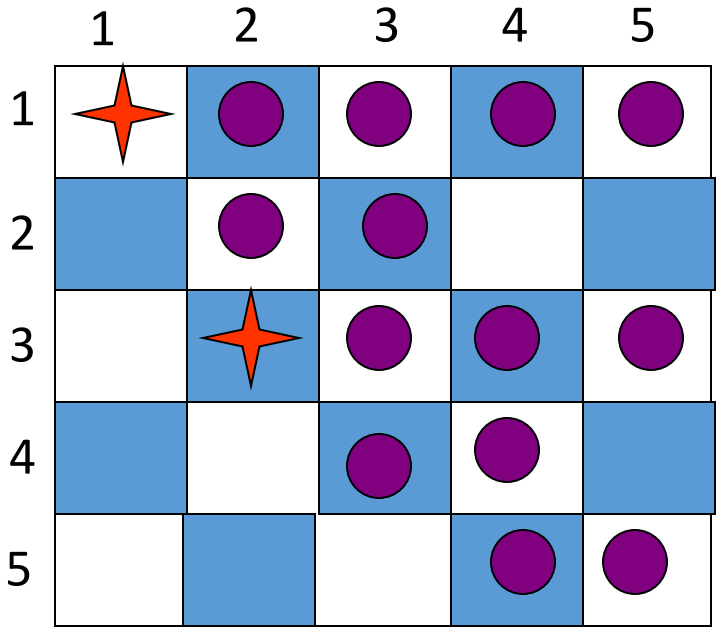


Classic AI Search Problems

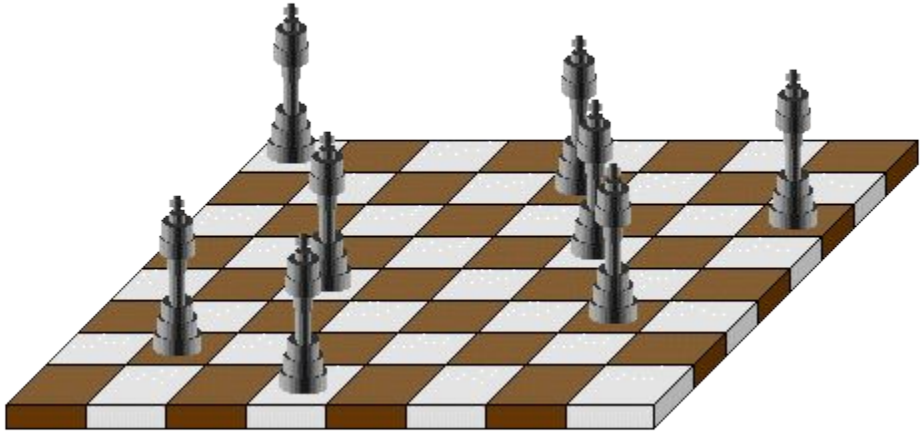


- 5-Queens:

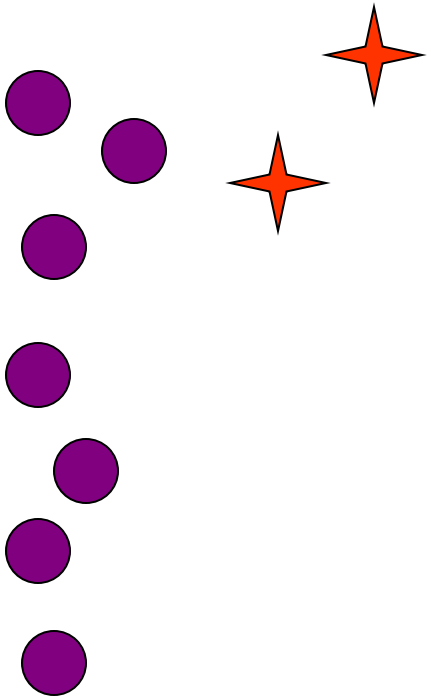


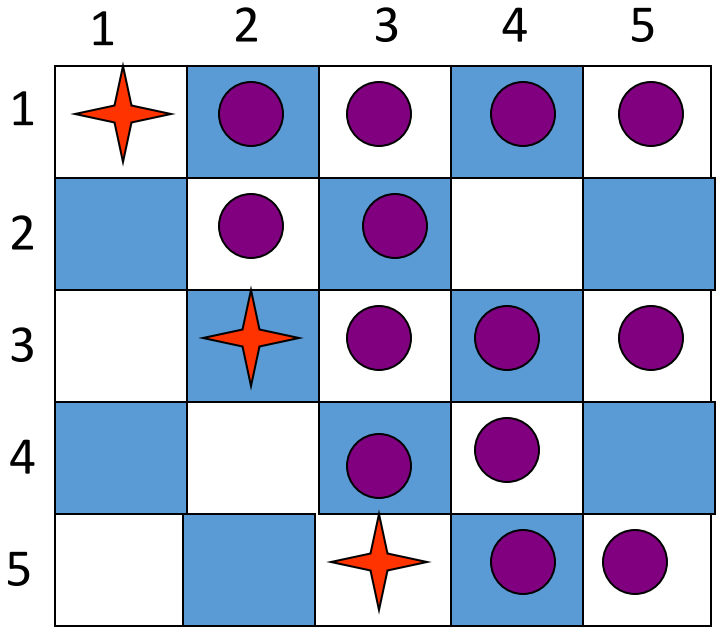


Classic AI Search Problems

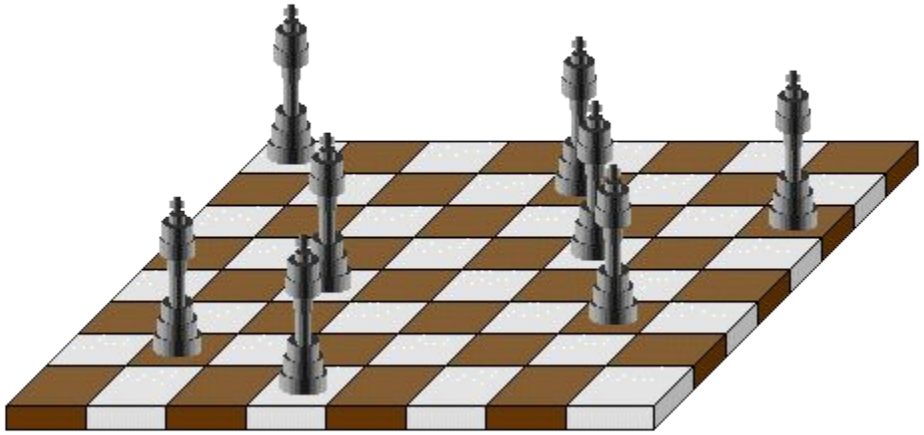


- 5-Queens:

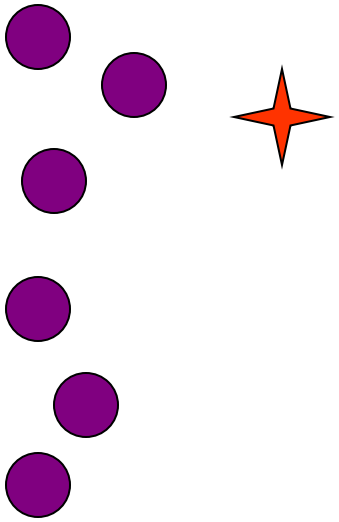


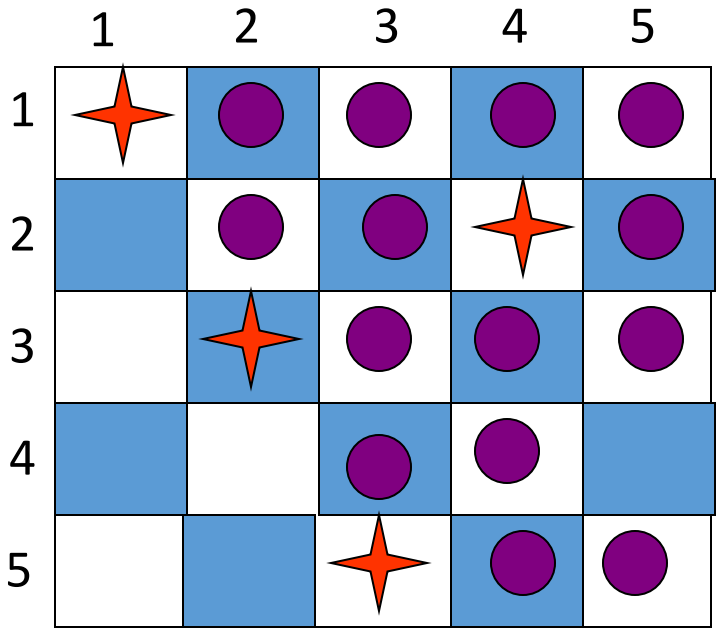


Classic AI Search Problems

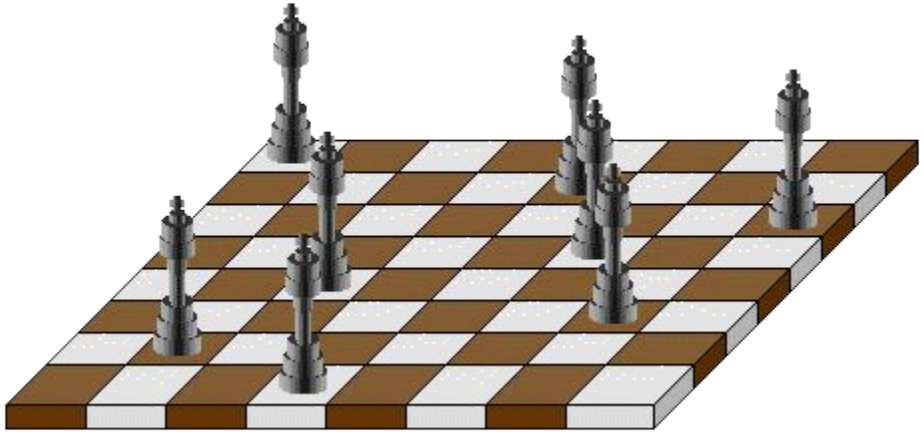


- 5-Queens:

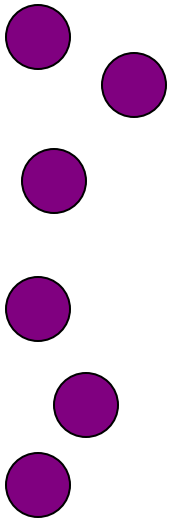




Classic AI Search Problems

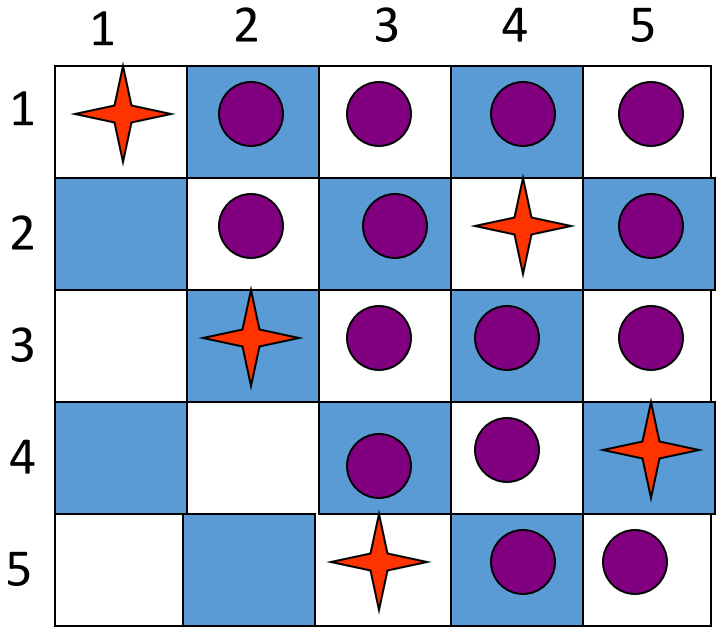


- 5-Queens:



Solution !!

No Queen is under Attack



River Crossing Problem

- A farmer wishes to carry a wolf, a goat and corn across a river, from the south to the north shore.
  - The farmer has a small rowing boat.
  - The boat can only carry at most the farmer and one other item.
  - If left unattended the wolf will eat the goat and the goat will eat the corn.

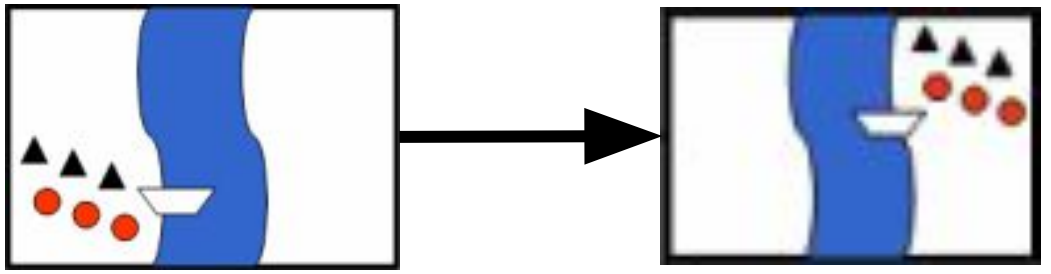
- How can the farmer safely transport the wolf, the goat and the corn to the opposite shore?



Missionaries and cannibals

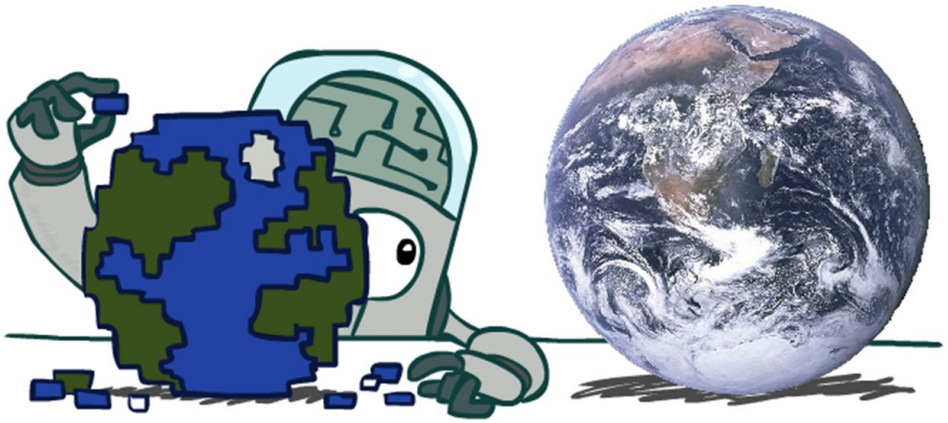
- Three missionaries and three cannibals are on the left bank of a river.
- There is one boat which can hold one or two people.
- Find a way to get everyone to the right bank, without ever leaving a group of missionaries in one place outnumbered by cannibals in that place.





Initial State

Goal State



Problem Solving by Searching

Problem Formulation

Search in AI

- Search in Data Structures
  - You're given an existent tree.
  - You search it in different orders.
  - It resides in memory.
- Search in Artificial Intelligence
  - The tree does not exist.
  - You have to generate it as you go.
  - For realistic problems, it does not fit in memory.

Problem Formulation(defining problem in proper way)

A **Problem Space** consists of

- The current state of the world (**initial state**)
- A description of the actions we can take to transform one state of the world into another (**operators or successor function**).
- A description of the desired state of the world (**goal state**), this could be implicit or explicit.
- A **solution** consists of the goal state, or a path to the goal state.

Problem Formulation

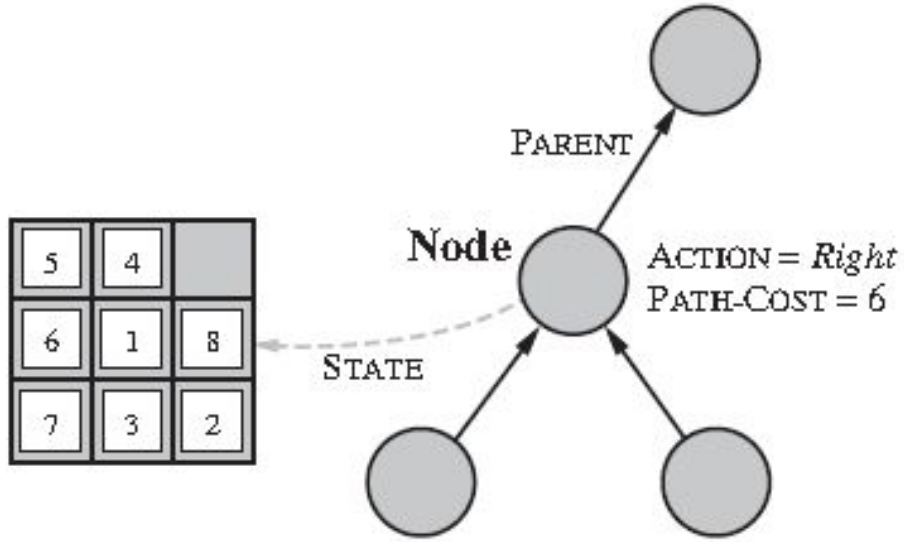
- Initial state
- Actions (operators, successor functions)
- State space
- Path
- Goal test
- Solution
- Path cost
- Total cost

Representation of a Search/State Space

- A convenient way of representing search spaces is as a graph

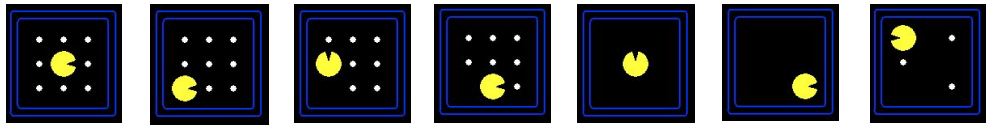
  - **States** are nodes
  - **Actions** are edges
  - **Initial state** is root
  - **Solution** is path from root to goal node

  - Edges sometimes have associated costs
  - States resulting from operator are children

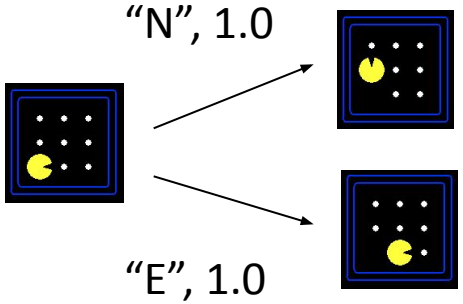


State Space Representation

- A search problem consists of:
  - A state space



  - A successor function (with actions, costs)



  - A start state and a goal test

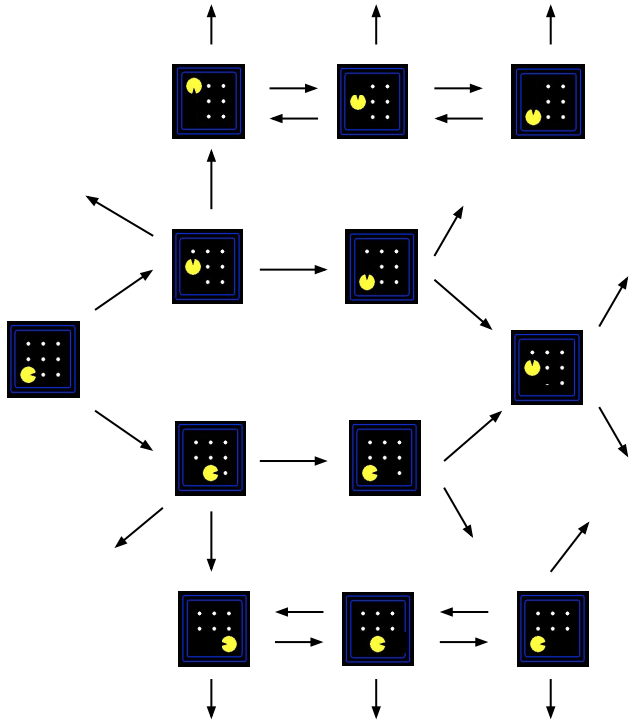
- A solution is a sequence of actions (a plan) which transforms the start state to a goal state

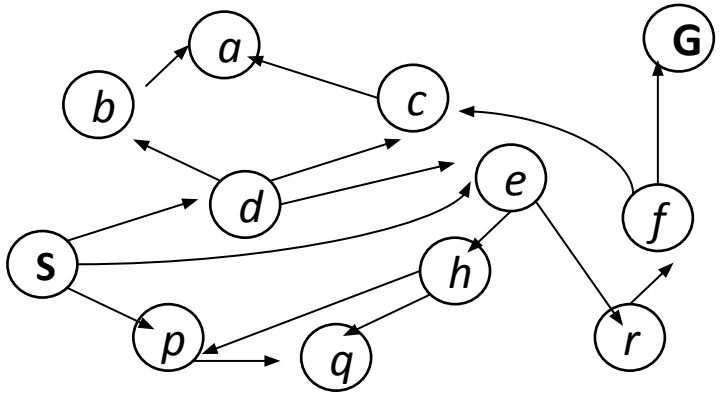
State Space Graphs

- State space graph: A mathematical representation of a search problem
  - Nodes are (abstracted) world configurations
  - Arcs represent successors (action results)
  - The goal test is a set of goal nodes (maybe only one)

- In a state space graph, each state occurs only once!

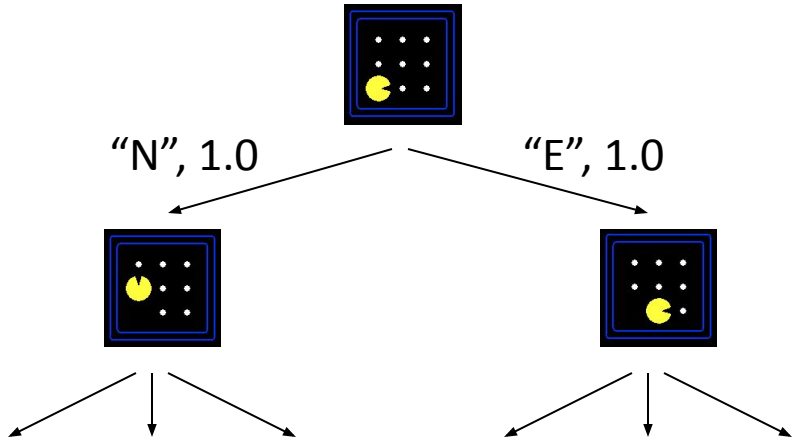
- We can rarely build this full graph in memory (it's too big), but it's a useful idea





Tiny search graph for a tiny search problem

Search Trees





This is now / start

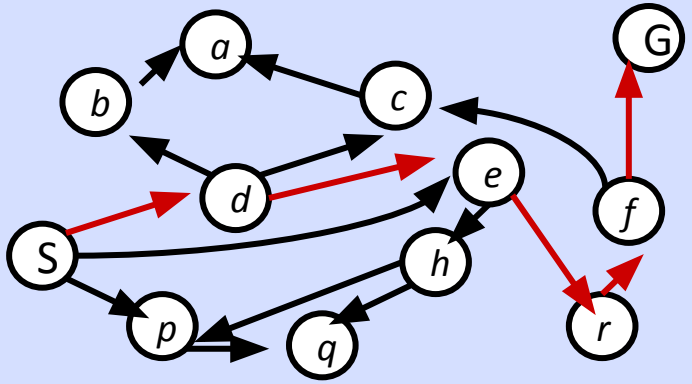


Possible futures

- A search tree:
  - A "what if" tree of plans and their outcomes
  - The start state is the root node
  - Children correspond to successors
  - Nodes show states, but correspond to PLANS that achieve those states
  - For most problems, we can never actually build the whole tree

State Space Graphs vs. Search Trees

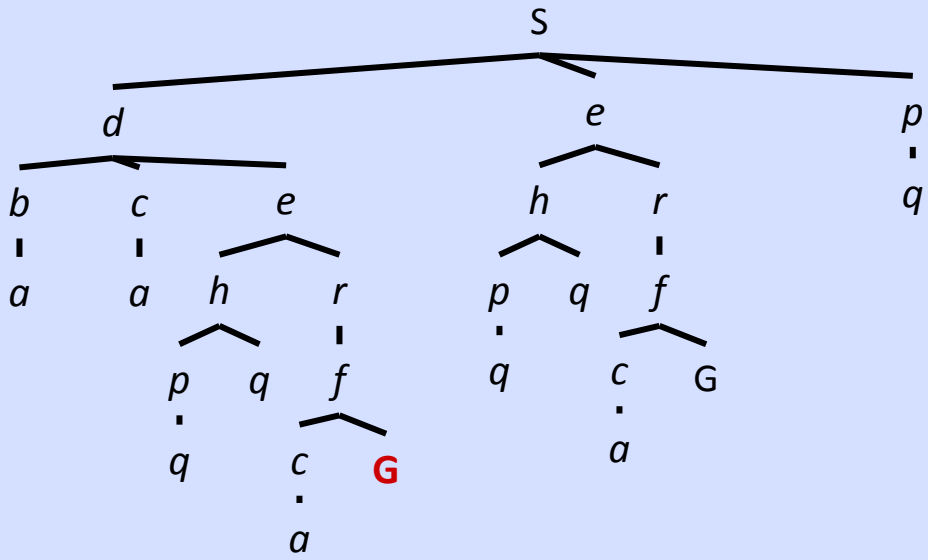
State Space Graph



Each NODE in the search tree is an entire PATH in the state space graph.

We construct both on demand – and we construct as little as possible.

Search Tree

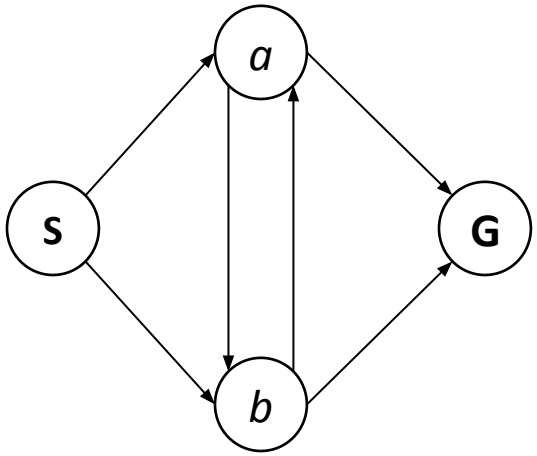


Approach

- Start with a **frontier** that contains the initial state.
- Repeat:
  - If the frontier is empty, then no solution.
  - Remove a node from the frontier.
  - If node contains goal state, return the solution.
  - **Expand** node, add resulting nodes to the frontier.

Quiz: State Space Graphs vs. Search Trees

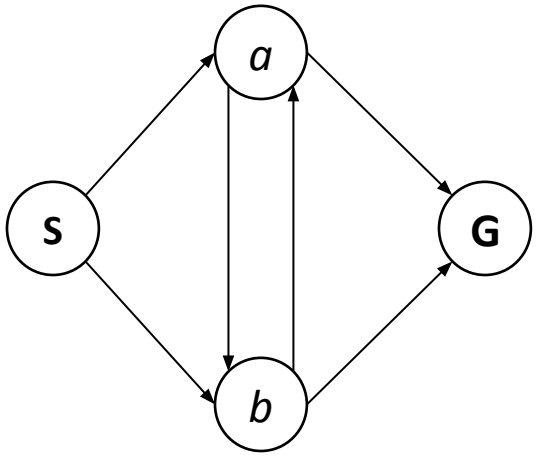
Consider this 4-state graph:



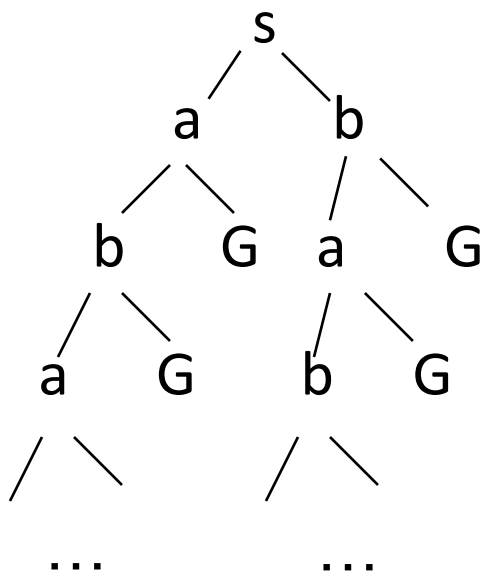
How big is its search tree (from S)?

Quiz: State Space Graphs vs. Search Trees

Consider this 4-state graph:



How big is its search tree (from S)?





Important: Lots of repeated structure in the search tree!

Revised Approach

- Start with a **frontier** that contains the initial state.
- Start with an empty **explored set**.
- Repeat:
  - If the frontier is empty, then no solution.
  - Remove a node from the frontier.
  - If node contains goal state, return the solution.
  - Add the node to the explored set.
  - **Expand** node, add resulting nodes to the frontier if they aren't already in the frontier or the explored set.

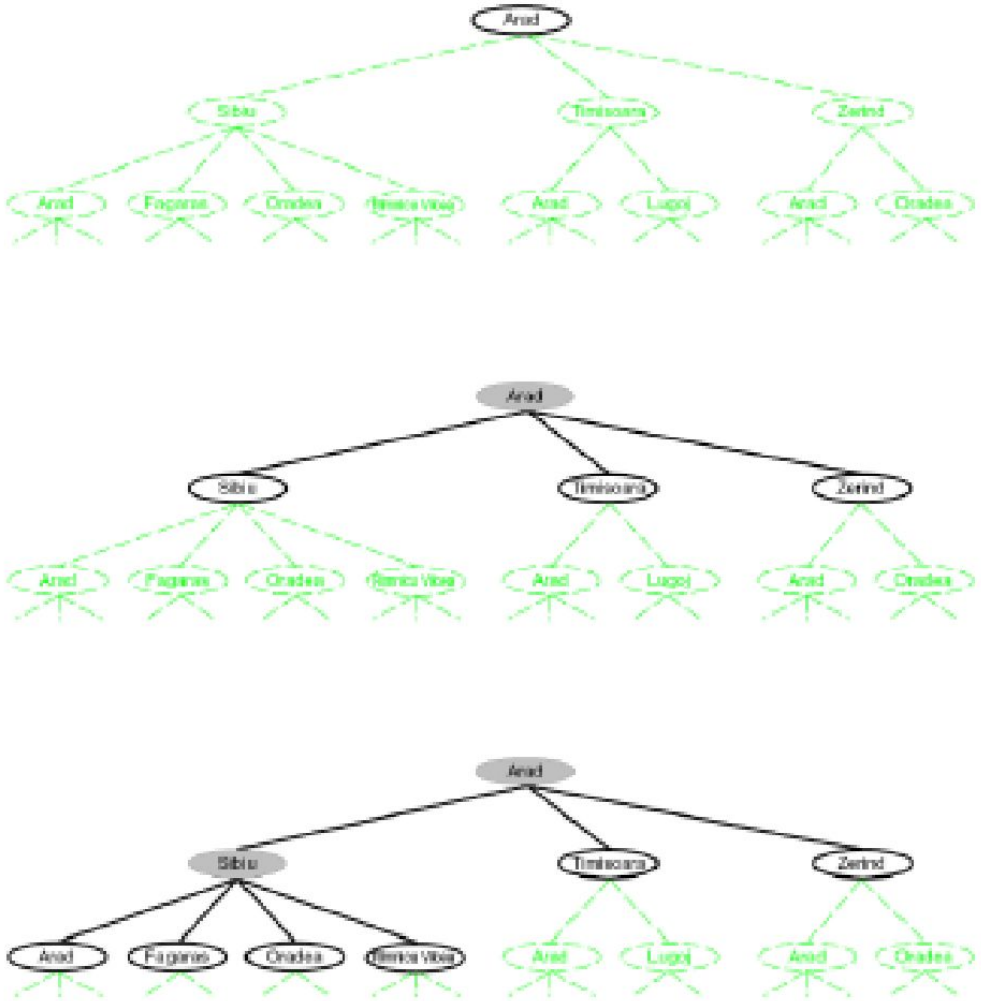
Searching with a Search Tree

- Search:

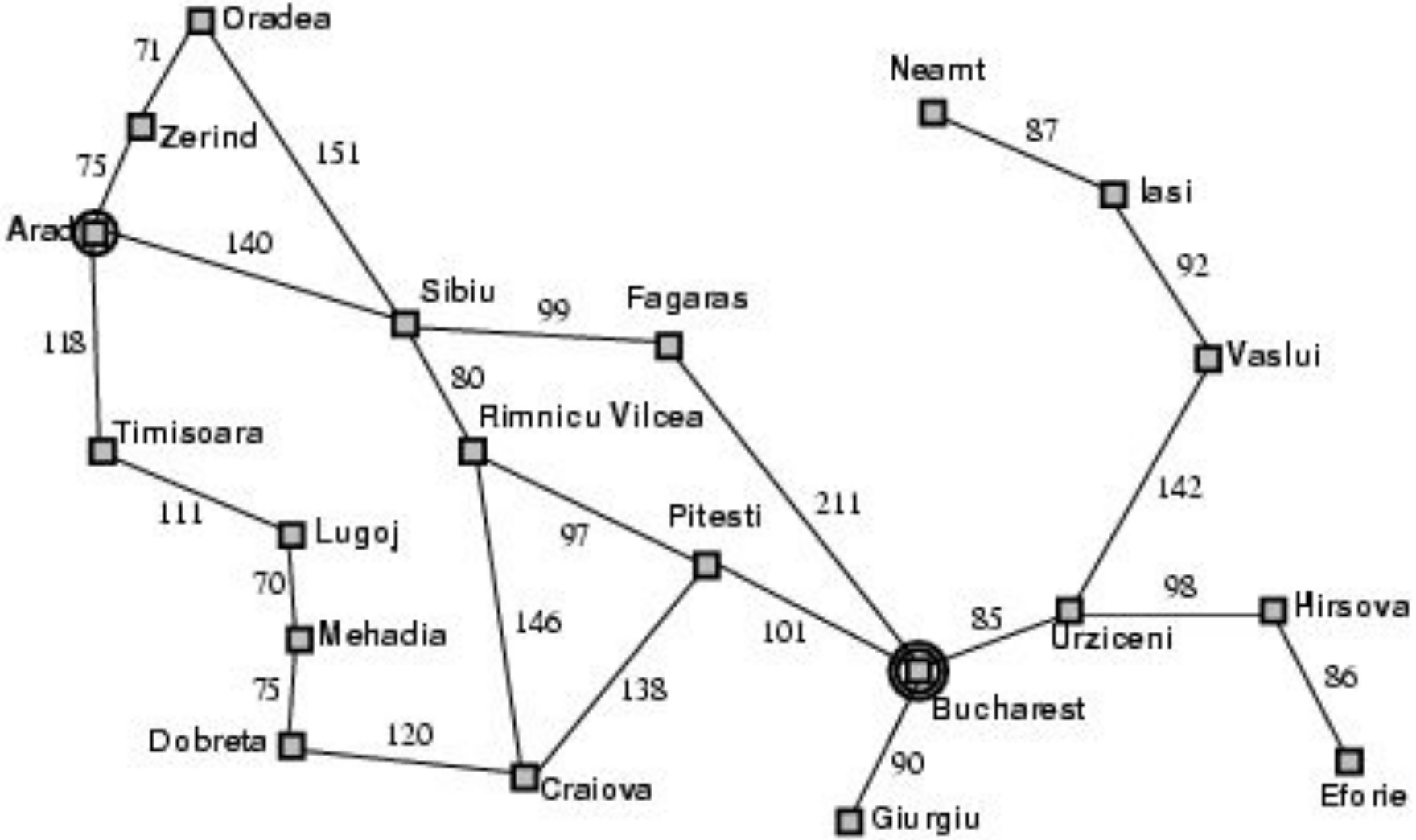
(1) Expand out potential plans (tree nodes)

(2) Maintain fringe (peripheral) partial plans under consideration.

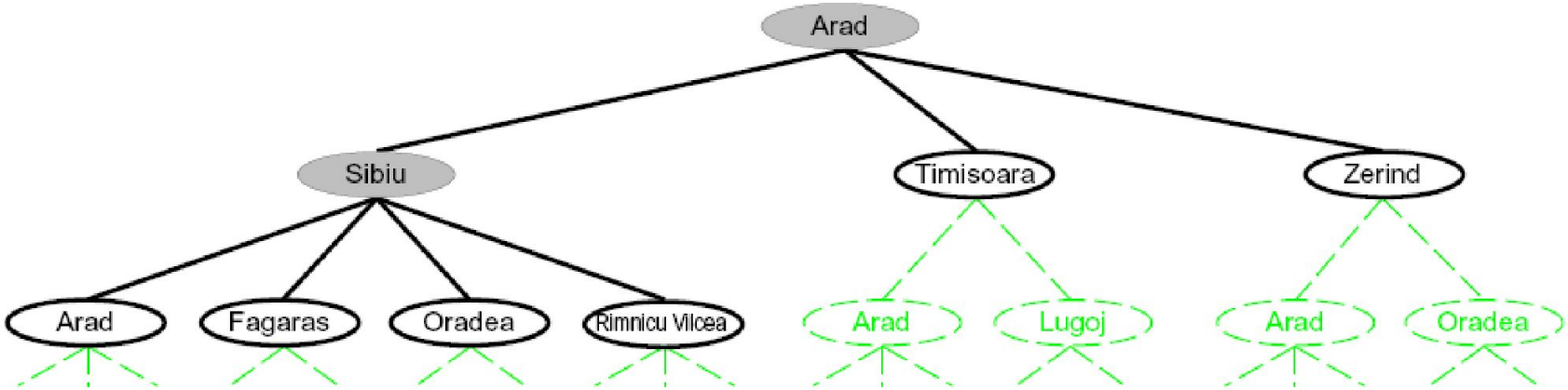
(3) Try to expand as few tree nodes as possible



Example: Romania



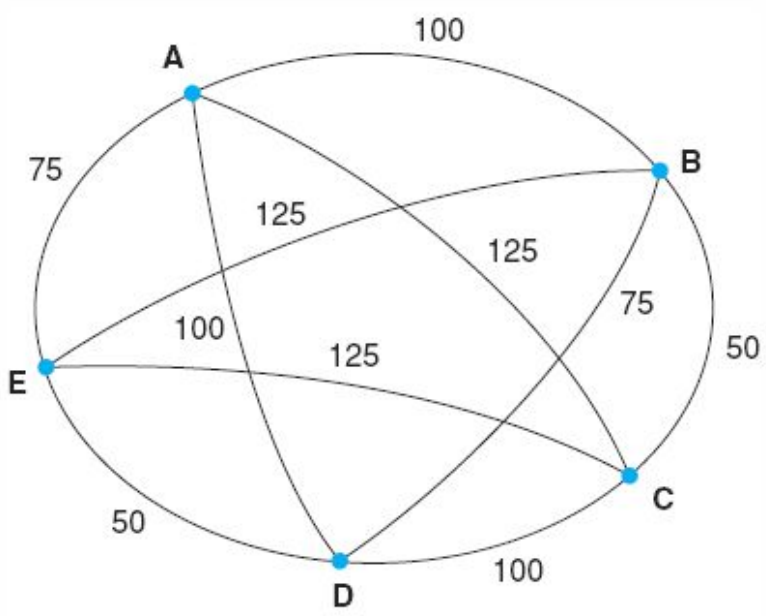
Searching with a Search Tree



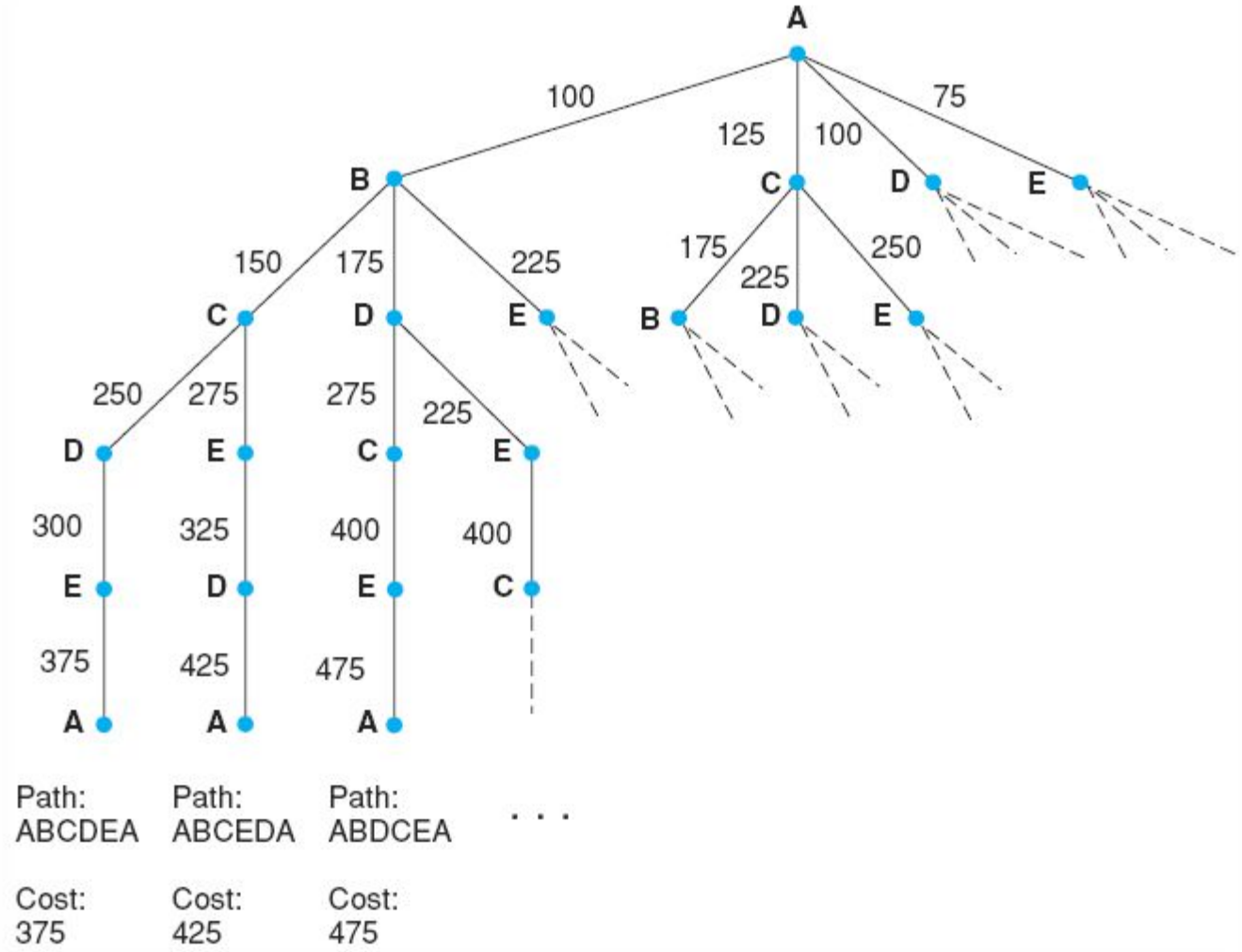
- Search:
  - Expand out potential plans (tree nodes)
  - Maintain a fringe (The collection of generated nodes that have not yet been expanded) of partial plans under consideration
  - Try to expand as few tree nodes as possible

Problem Formulation: TSP

- **Travelling Salesperson Problem (TSP)**
  - Suppose a salesman has four cities to visit and then must return home.
  - The goal of the problem is to find the shortest path for salesman to travel, visiting each city, and then returning to the starting city.



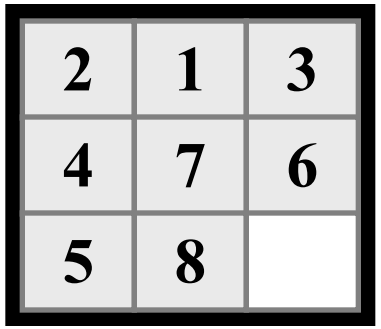
Search for the travelling salesperson problem. Each arc is marked with the total weight of all paths from the start node (A) to its endpoint.



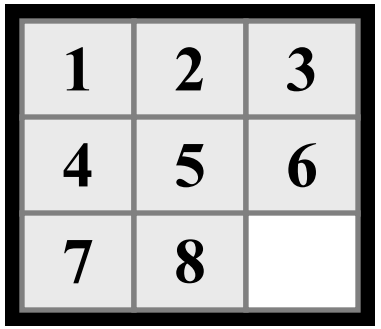
Luger: Artificial Intelligence, 6th edition. © Pearson Education Limited, 2009

Problem Formulation: **8-Puzzle Problem**

Initial State	Operators	Goal State



Slide blank square left.
Slide blank square right.
….



Problem Formulation: **8-Puzzle Problem**

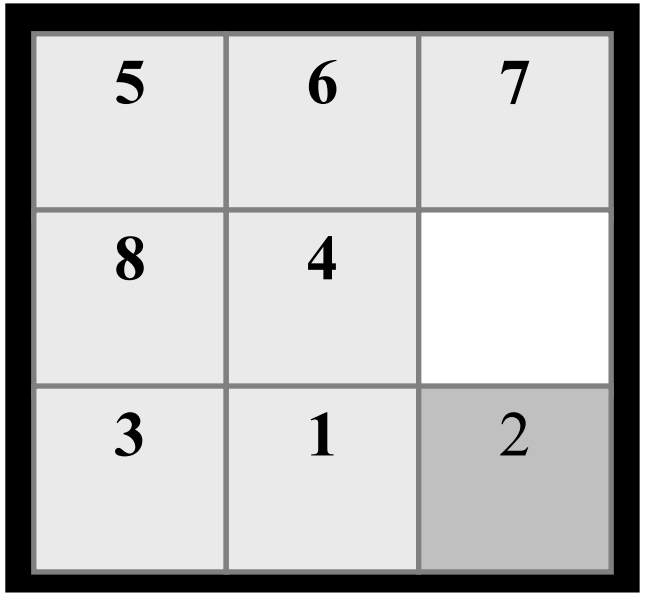
Representing states:

- **For the 8-puzzle**

- 3 by 3 array
  - 5, 6, 7
  - 8, 4, BLANK
  - 3, 1, 2

- A vector of length nine
  - 5, 6, 7, 8, 4, BLANK, 3, 1, 2

- A list of facts
  - Upper_left = 5
  - Upper_middle = 6
  - Upper_right = 7
  - Middle_left = 8

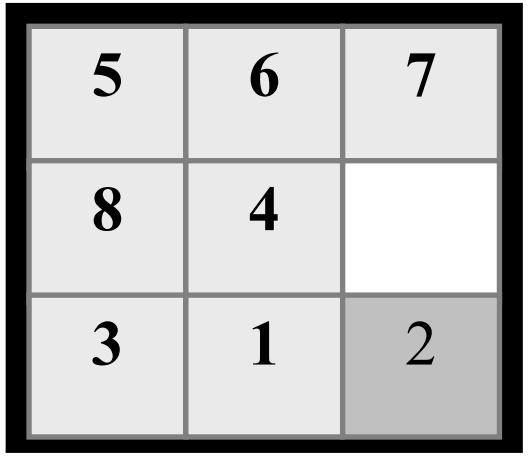


Problem Formulation: **8-Puzzle Problem**

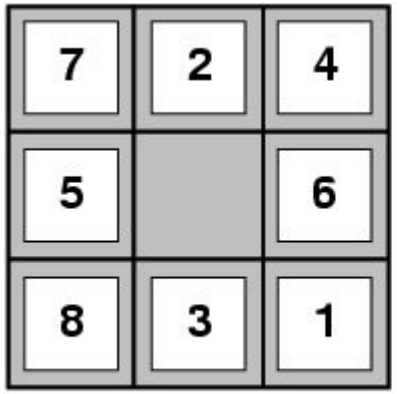
- Specifying operators
  - There are often many ways to specify the operators, some will be much easier to implement...

- **Move 1 left**
- **Move 1 right**
- **Move 1 up**
- **Move 1 down**
- **Move 2 left**
- **Move 2 right**
- **Move 2 up**
- **Move 2 down**
- **Move 3 left**
- **Move 3 right**
- **Move 3 up**
- **Move 3 down**
- **Move 4 left**
- …

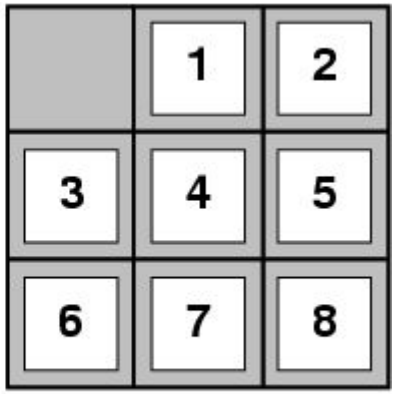
- **Move Blank left**
- **Move Blank right**
- **Move Blank up**
- **Move Blank down**



Problem Formulation: **8-Puzzle Problem**



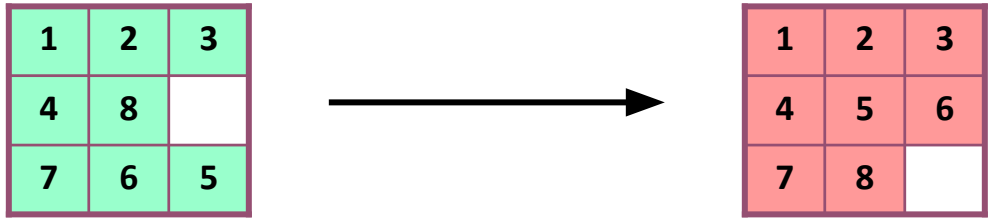
Start State



Goal State

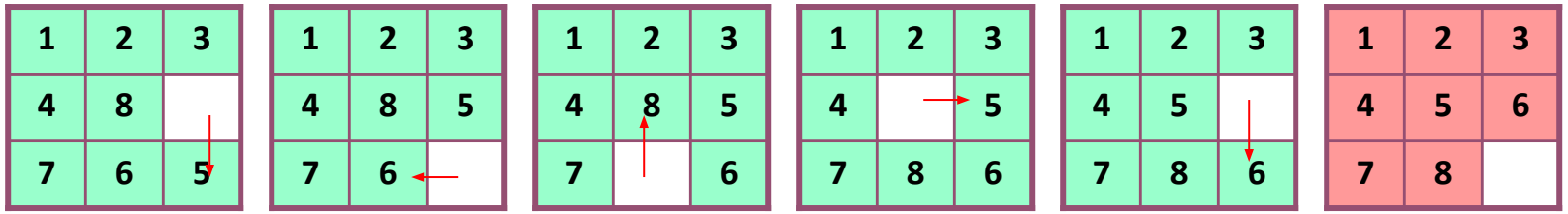
- states locations of tiles
- actions move left, right, up, down
- goal test goal state (given)
- path cost 1 per move

Problem Formulation: **8-Puzzle Problem**



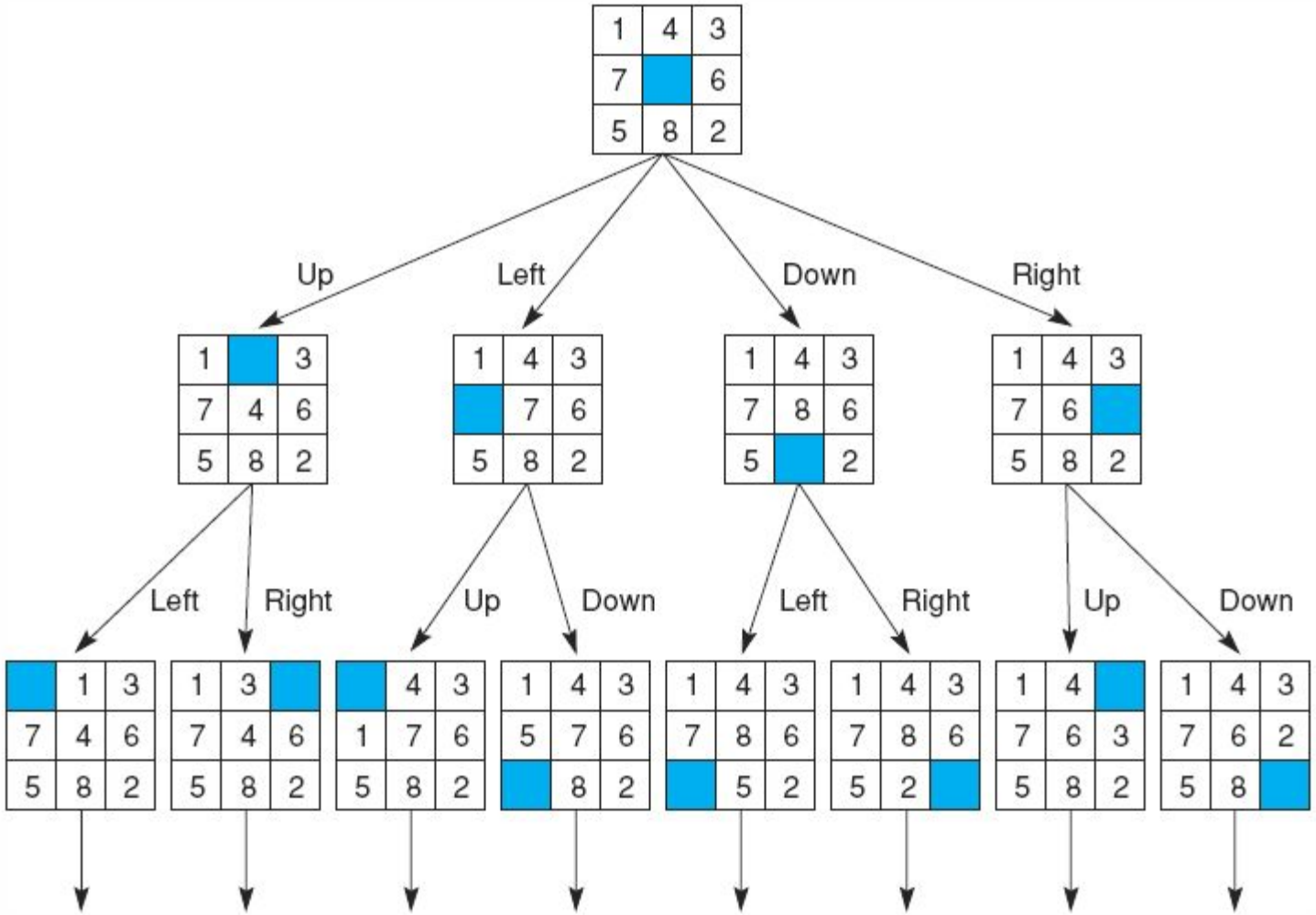
Initial state **Goal state**

- Operators: ***slide blank up, slide blank down, slide blank left, slide blank right***

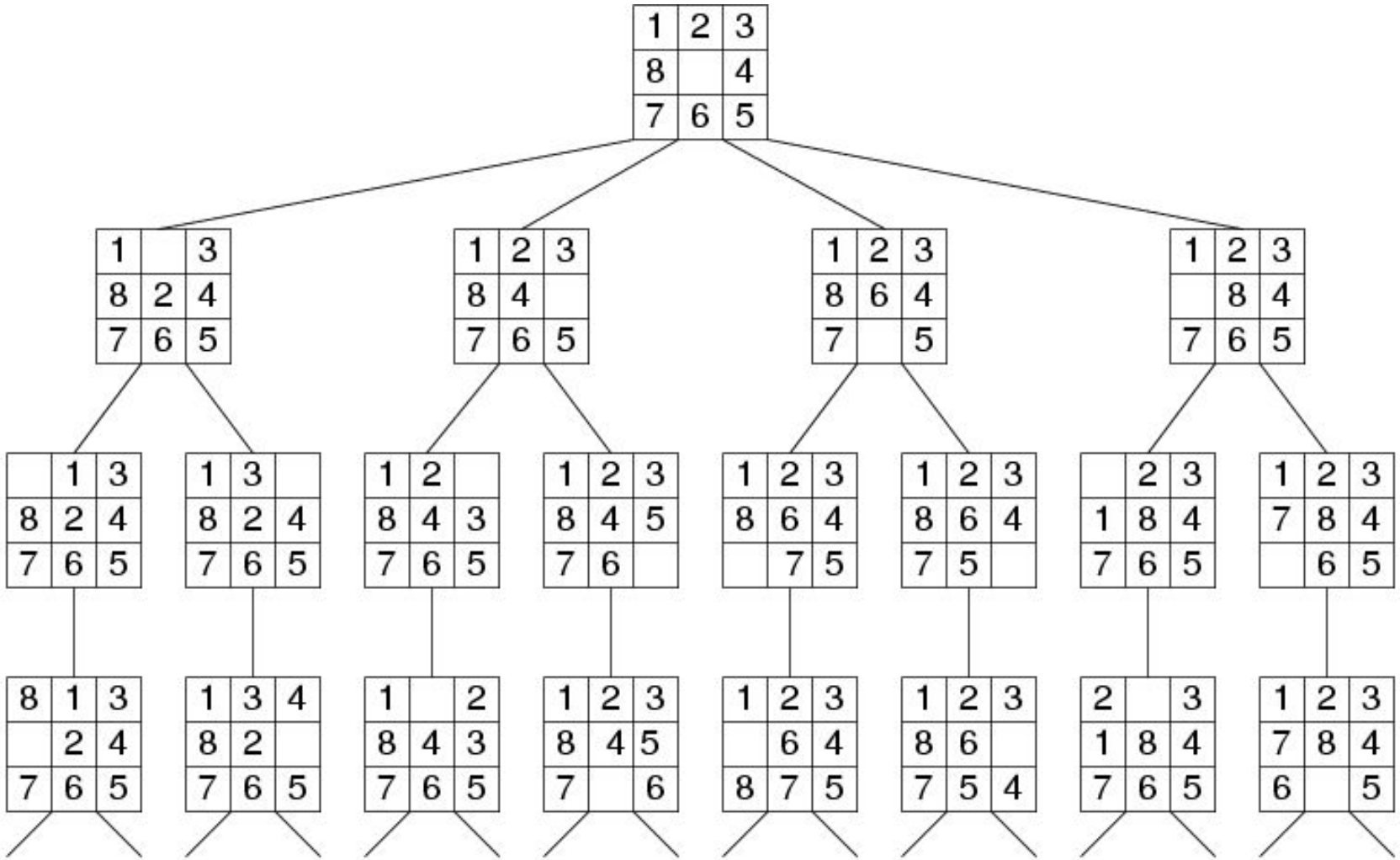


- Solution: ***swap-down, swap-left, swap-up, swap-right, swap-down***
- *Path cost: **5** steps to reach the goal*

State space of the 8-puzzle generated by “move blank” operations

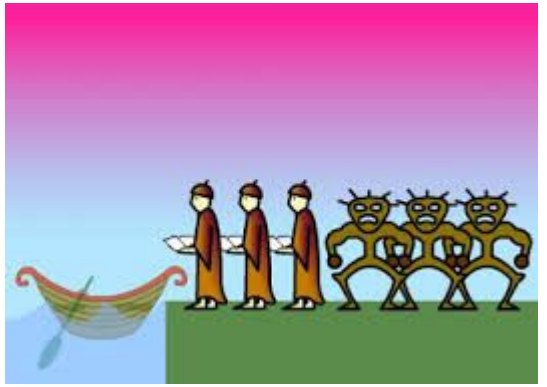


Fragment of 8-Puzzle Problem Space



Problem Formulation: Missionaries and Cannibals

- Three missionaries and three cannibals are on the left bank of a river.
- There is one boat which can hold one or two people.
- Find a way to get everyone to the right bank, without ever leaving a group of missionaries in one place outnumbered by cannibals in that place.



Missionaries and cannibals

- States: three numbers (i, j, k) representing the number of missionaries, cannibals, and boats on the left bank of the river.
- Initial state: (3, 3, 1)
- Operators: take one missionary, one cannibal, two missionaries, two cannibals, one missionary and one cannibal across the river in a given direction
- Goal Test: reached state (0, 0, 0)?
- Path Cost: Number of crossings.

Formalization of the M&C Problem

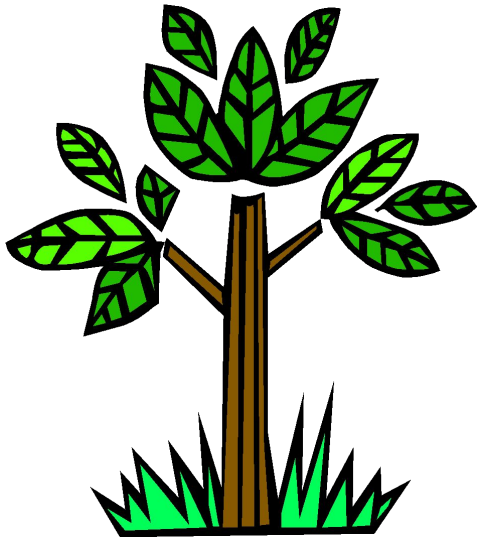
- **State space**: triple (x,y,z), where x,y, and z represent the number of missionaries, cannibals and boats currently on the original bank.
- **Initial State**: (3,3,1)
- **Successor function**: From each state, either bring one missionary, one cannibal, two missionaries, two cannibals, or one of each type to the other bank.
- **Note**: Not all states are attainable (e.g., (0,0,1)), and some are illegal.
- **Goal State**: (0,0,0)
- **Path Costs**: 1 unit per crossing

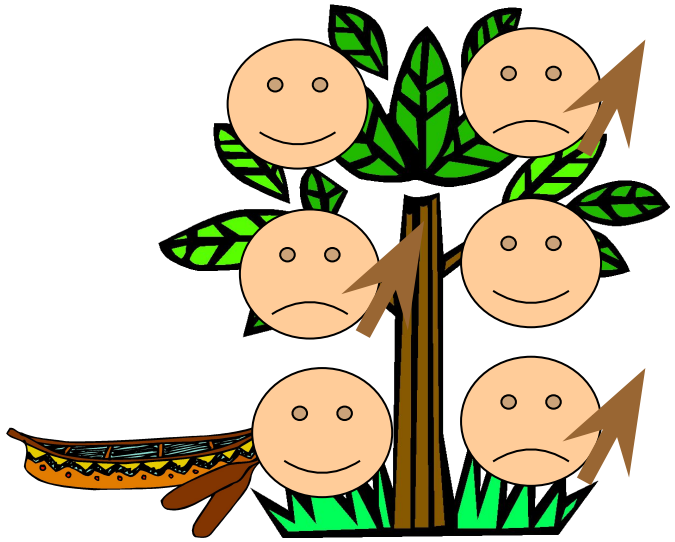
Missionaries and Cannibals

Solution = the sequence of actions within the path :

[(3,3,1)→ (2,2,0)→(3,2,1) →(3,0,0) →(3,1,1) →(1,1,0) →(2,2,1) →(0,2,0) →(0,3,1) →(0,1,0) → (0,2,1) →(0,0,0)]

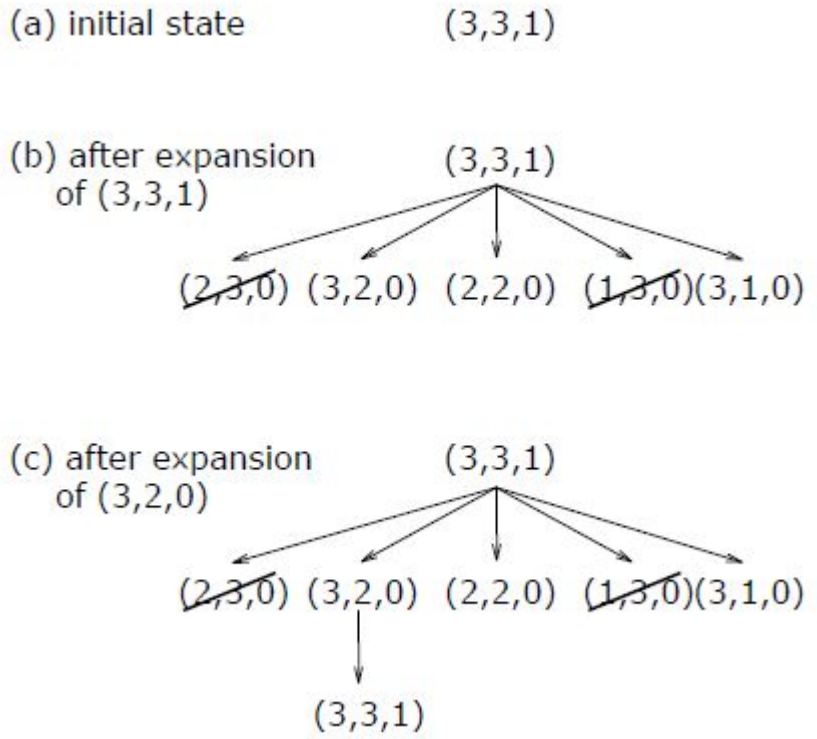
Cost = 11 crossings





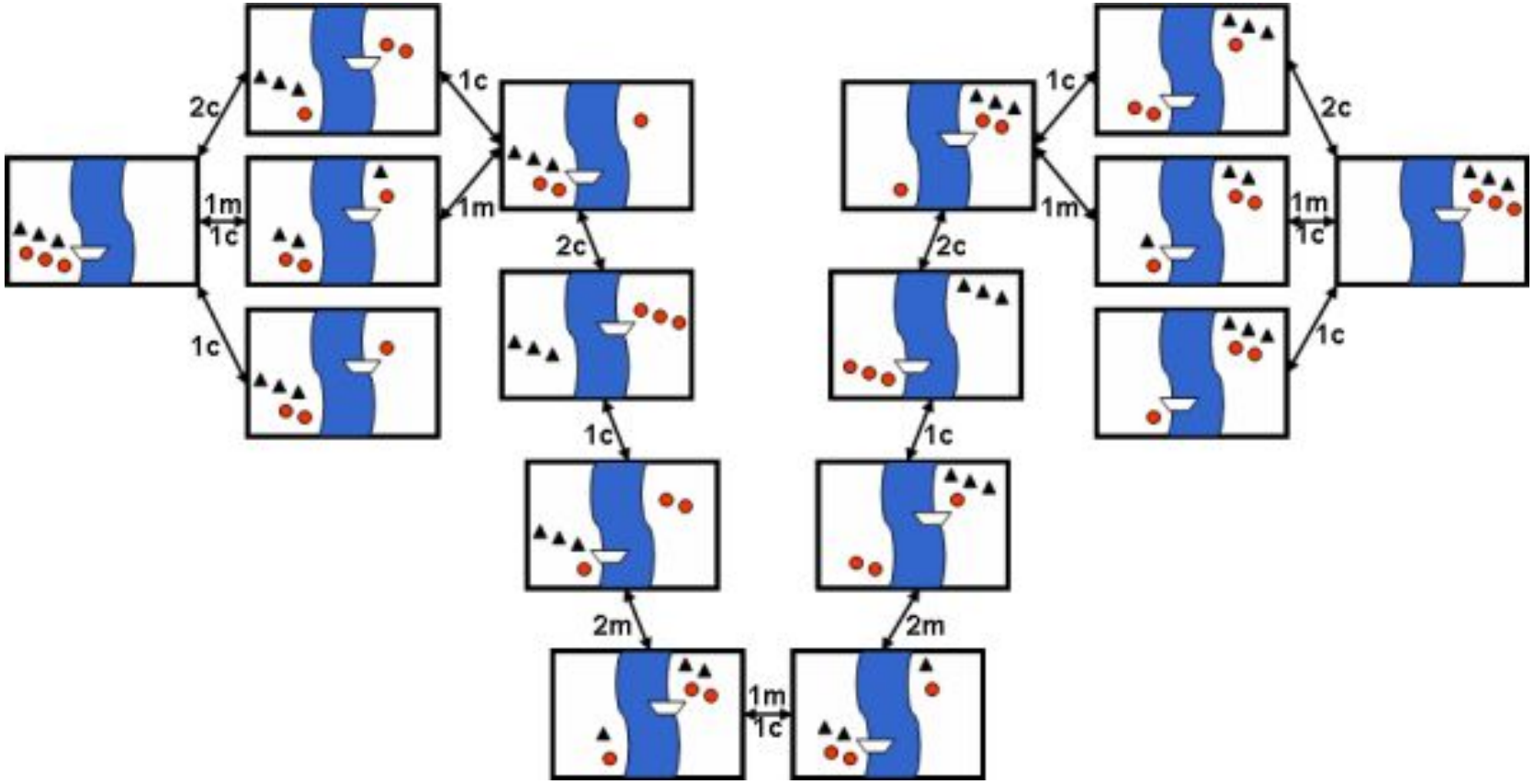
Missionaries and Cannibals

- From the initial state, produce all successive states step by step search tree.



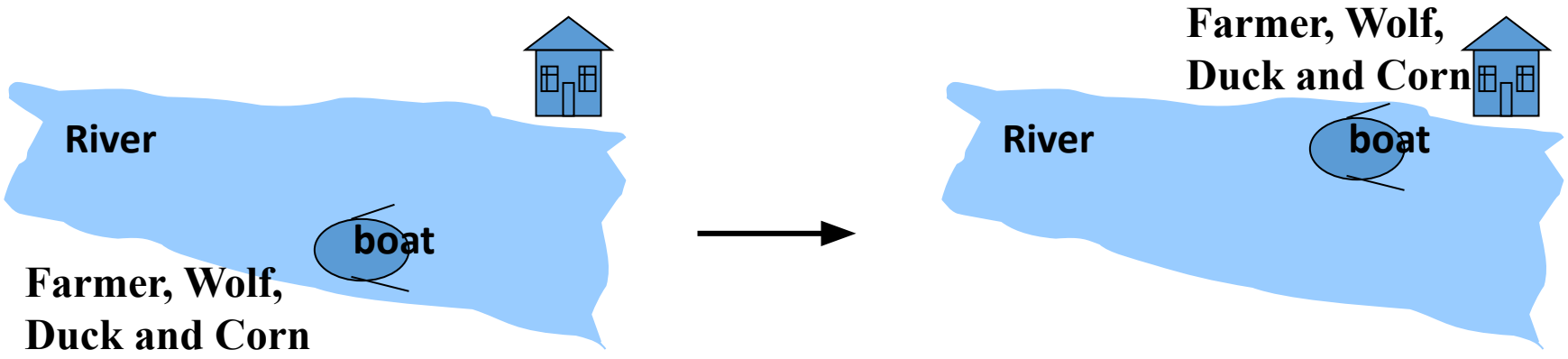
Missionaries and Cannibals

- *Search-space for the Missionaries and Cannibals problem*



The River Problem

- A farmer wishes to carry a wolf, a duck and corn across a river, from the south to the north shore. The farmer has a small rowing boat. The boat can only carry at most the farmer and one other item.
- If left unattended the wolf will eat the duck and the duck will eat the corn.

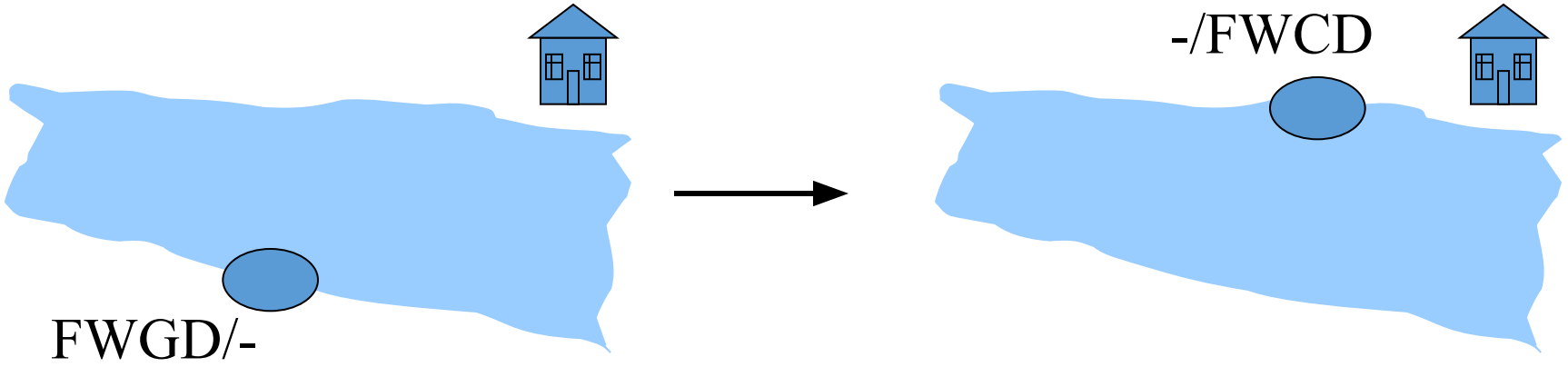


How can the farmer safely transport the wolf, the duck and the corn to the opposite shore?

The River Problem

- The River Problem:

F=Farmer W=Wolf G=Goat C=Corn /=River



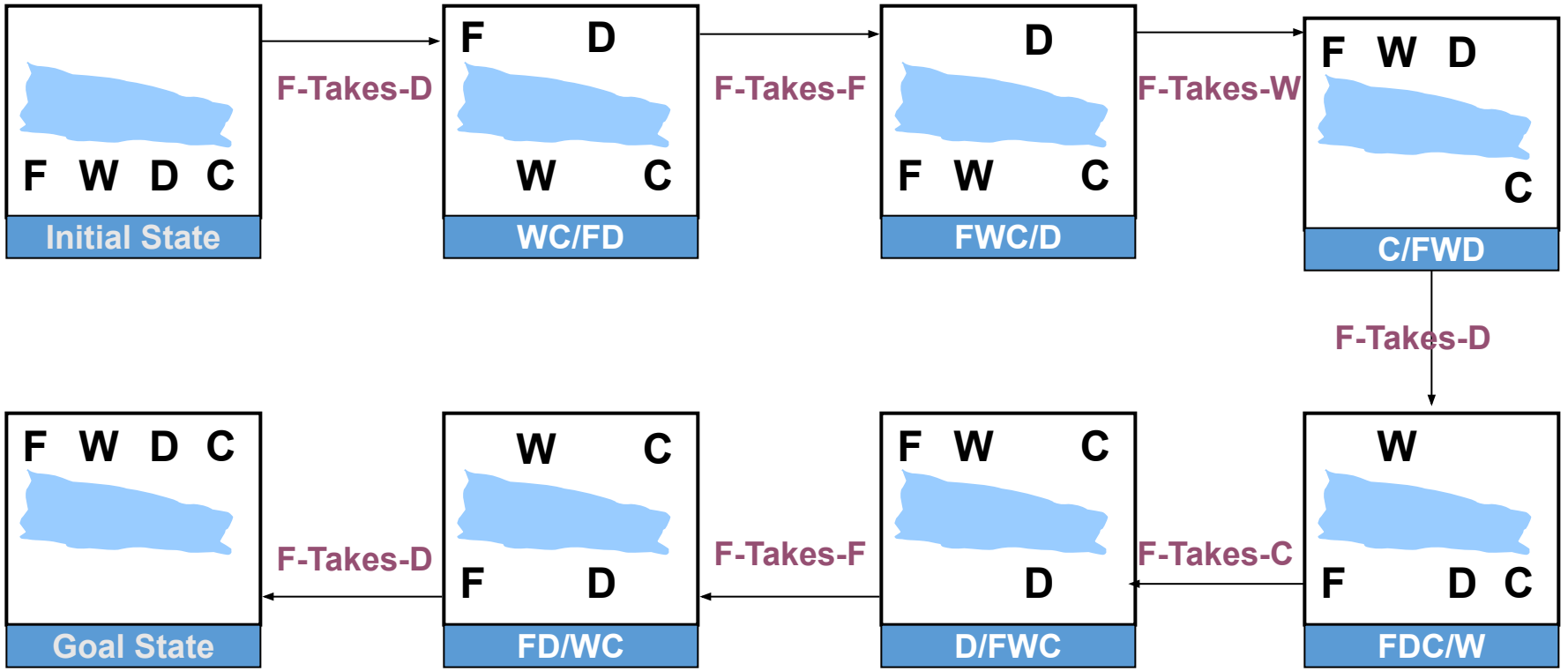
How can the farmer safely transport the wolf, the duck and the corn to the opposite shore?

The River Problem

- Problem formulation:
  - **State representation**: location of farmer and items in both sides of river
    [items in South shore / items in North shore] : (FWDC/-, FD/WC, C/FWD …)
  - **Initial State**: farmer, wolf, duck and corn in the south shore FWDC/-
  - **Goal State**: farmer, duck and corn in the north shore -/FWDC
  - **Operators**: the farmer takes in the boat at most one item from one side to the other side
    (F-Takes-W, F-Takes-D, F-Takes-C, F-Takes-Self [himself only])
  - **Path cost**: the number of crossings

The River Problem

- **Problem solution**: (path Cost = 7)
  - While there are other possibilities here is one **7** step solution to the river problem



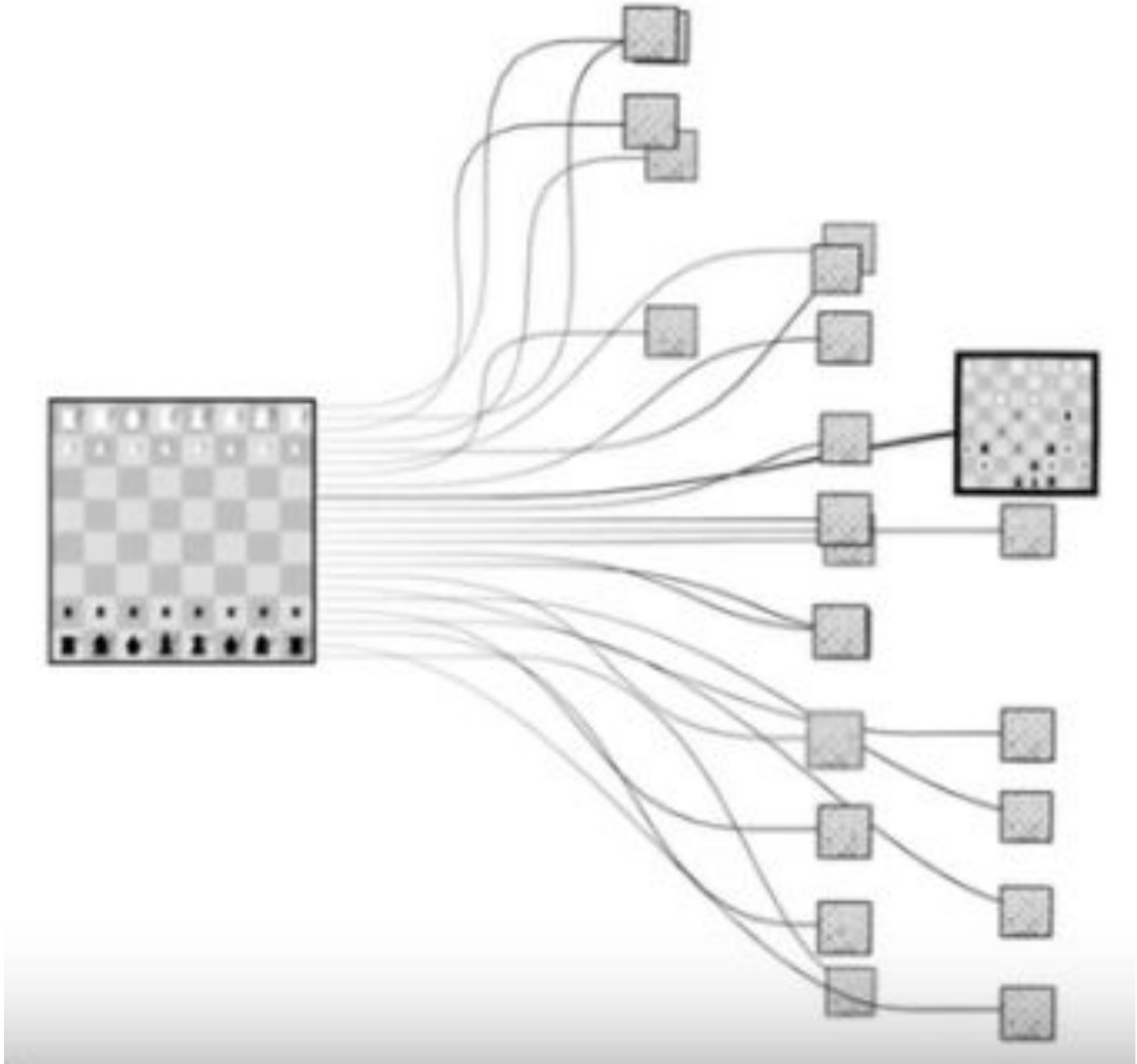
Summary

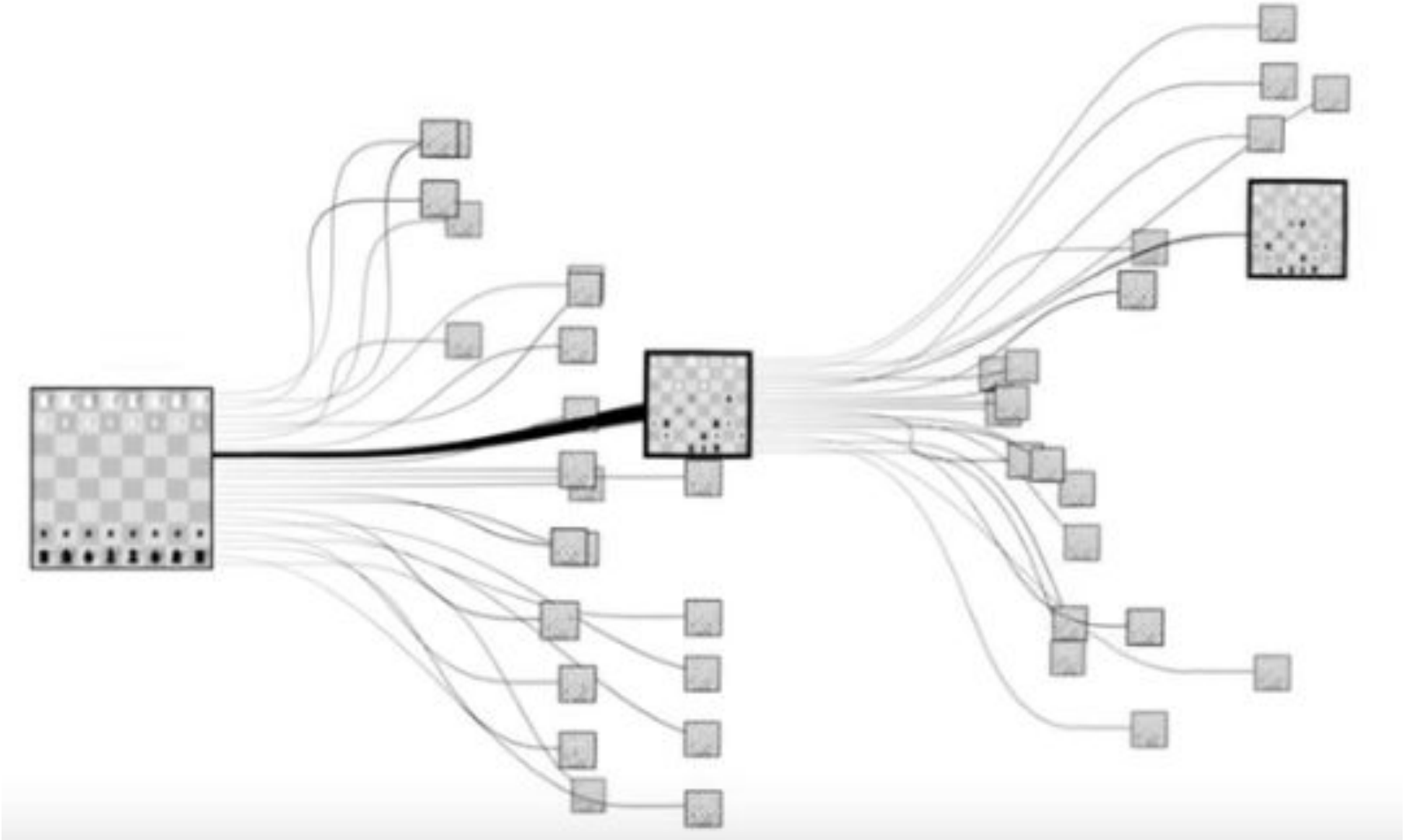
- **Search**: process of constructing sequences of actions that achieve a goal given a problem.
- It is assumed that the environment is **observable, deterministic, static and completely known**.
- **Goal formulation** is the first step in solving problems by searching. It facilitates problem formulation.
- **Formulating a problem** requires specifying five components:
  - State representation,
  - Initial state,
  - Goal state,
  - Operators (actions), and
  - Path cost function.

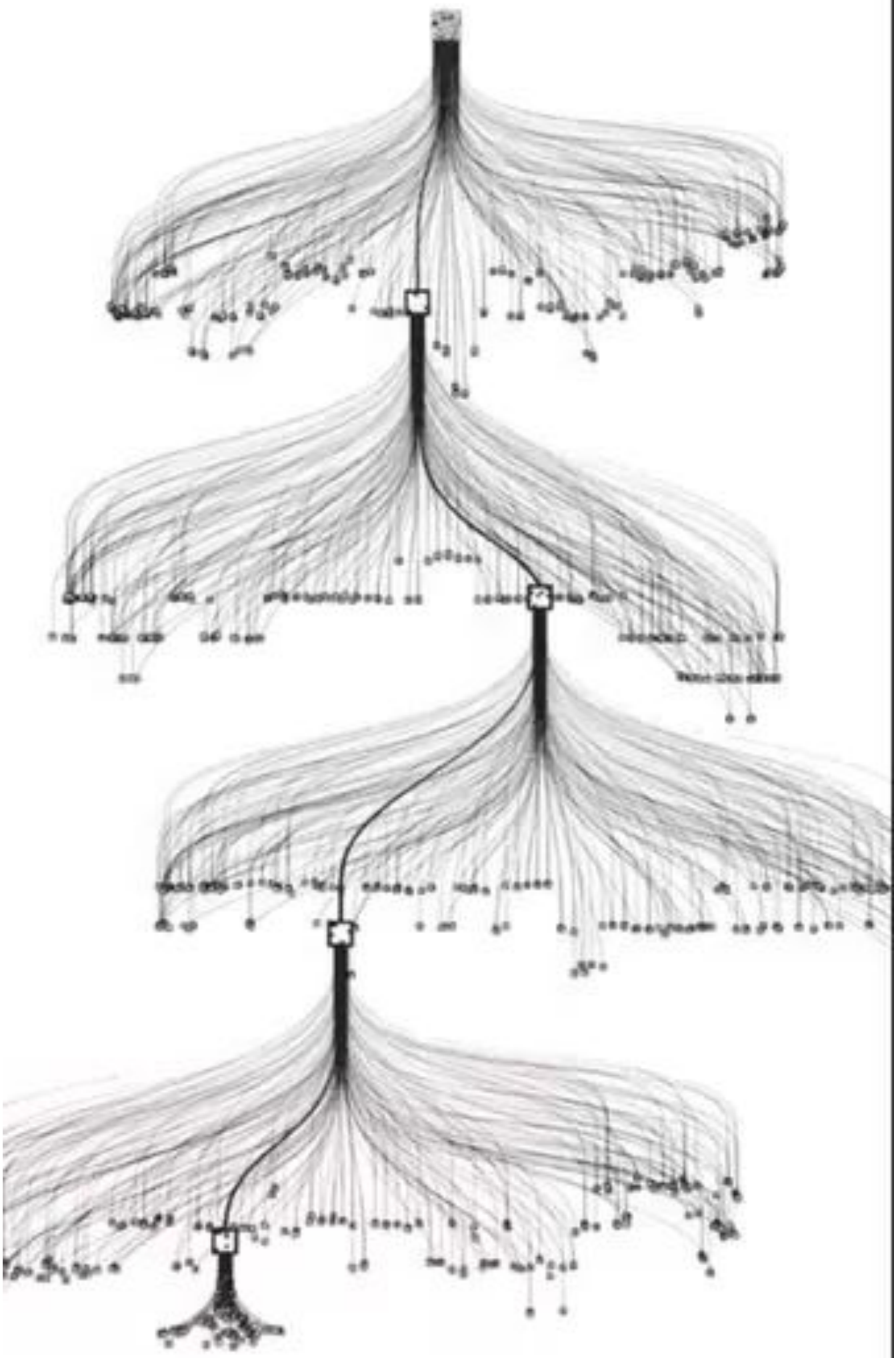
Search Strategies

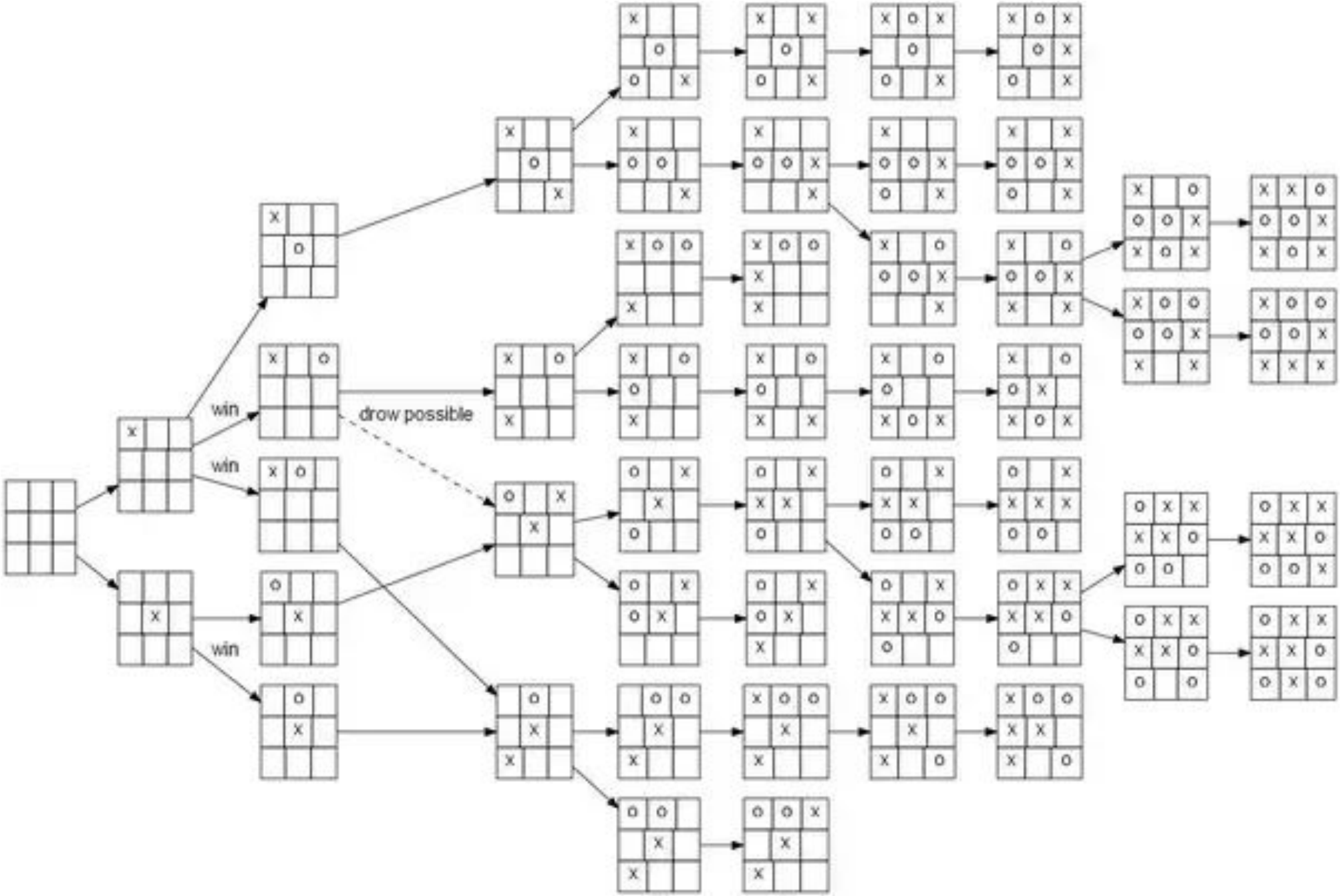
- Several general-purpose search algorithms that can be used to solve these problems.

1. Uninformed search algorithms
   - algorithms that are given no information about the problem other than its definition
   - Although some of these algorithms can solve any solvable problem, none of them can do so efficiently
2. Informed search algorithms
   - can do quite well given some guidance on where to look for solutions



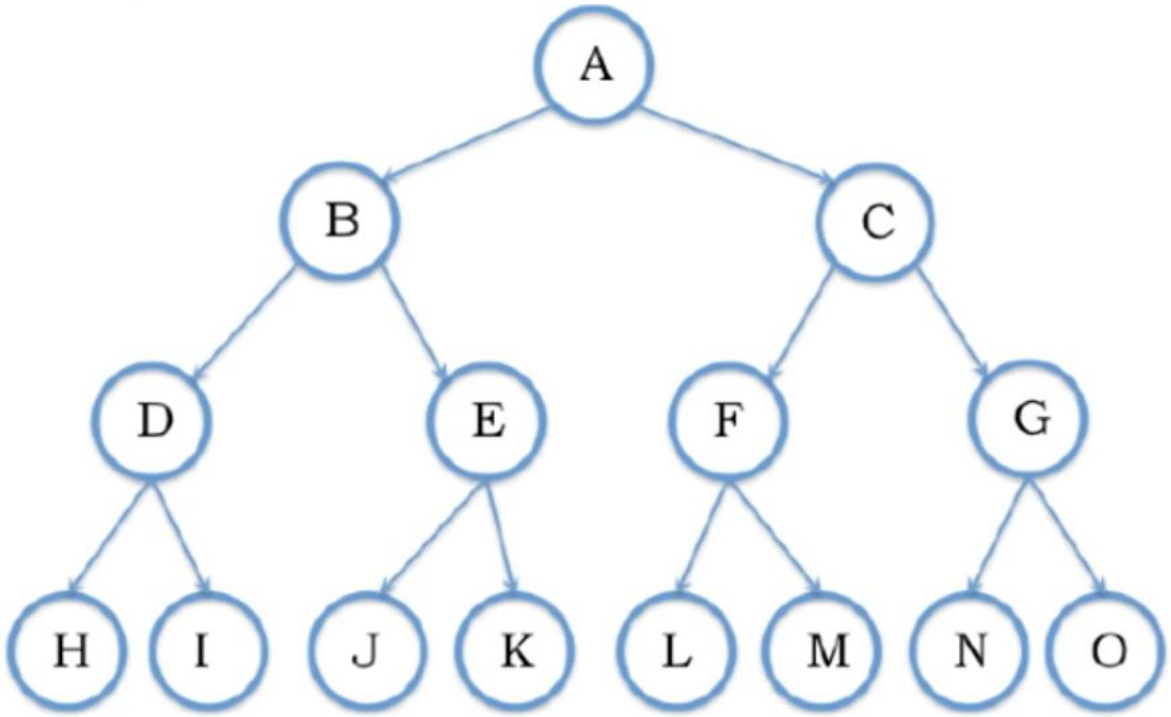




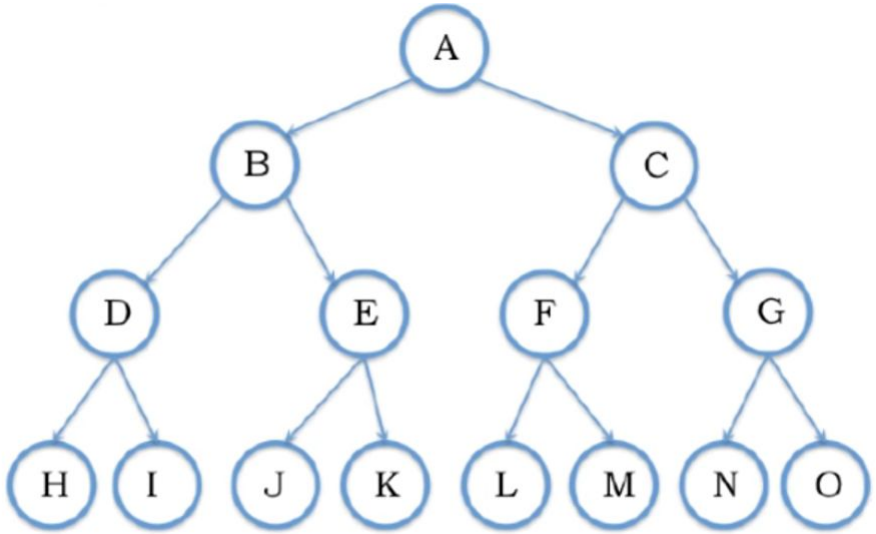

win
win
win
draw possible

Uninformed Search

Depth First Search (DFS)



Depth First Search (DFS)

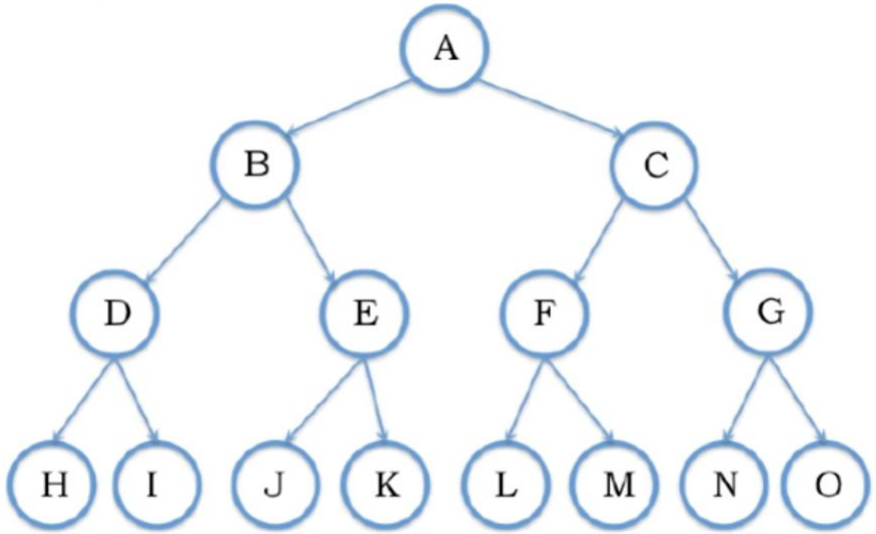


Elements Popped From Stack:

Elements pushed on stack

Stack
A

Depth First Search (DFS)



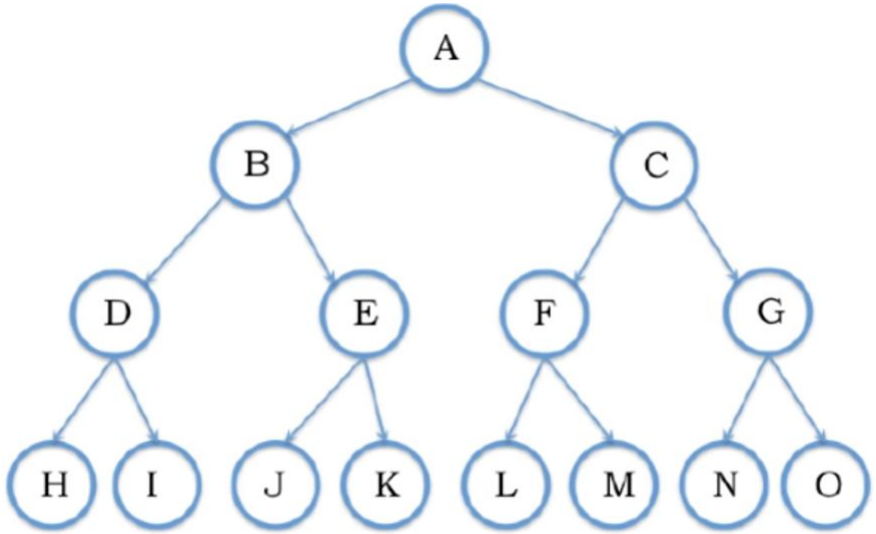
Elements Popped From Stack:

A □

Elements pushed on stack

Stack
C
B
~~A~~

Depth First Search (DFS)

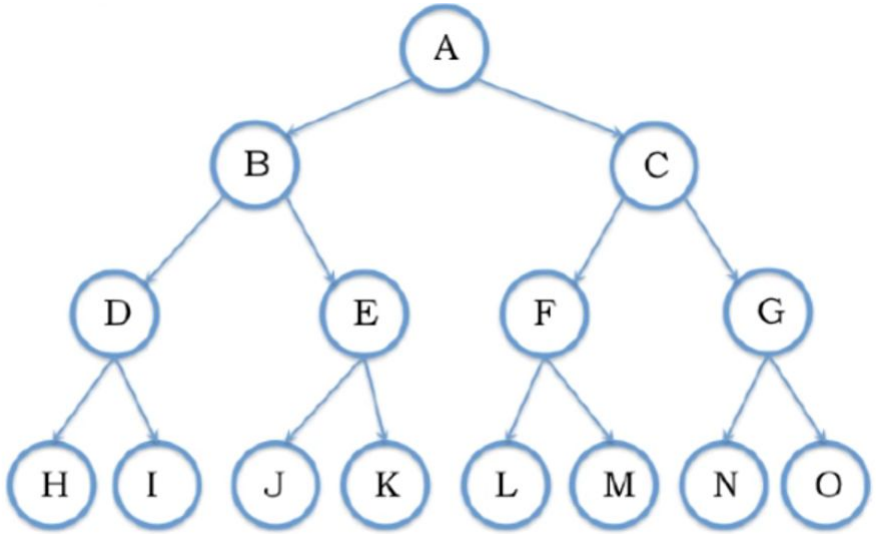


Elements Popped From Stack:
A □ C □

Elements pushed on stack

Stack
G
F
~~C~~
B
~~A~~

Depth First Search (DFS)

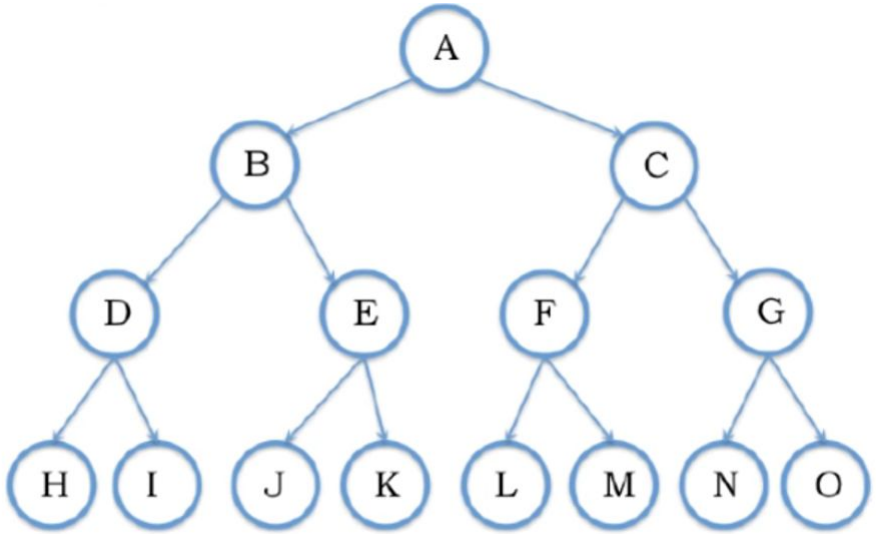


Elements Popped From Stack:
A □ C □ G

Elements pushed on stack

Stack
O
N
~~G~~
F
~~C~~
B
~~A~~

Depth First Search (DFS)

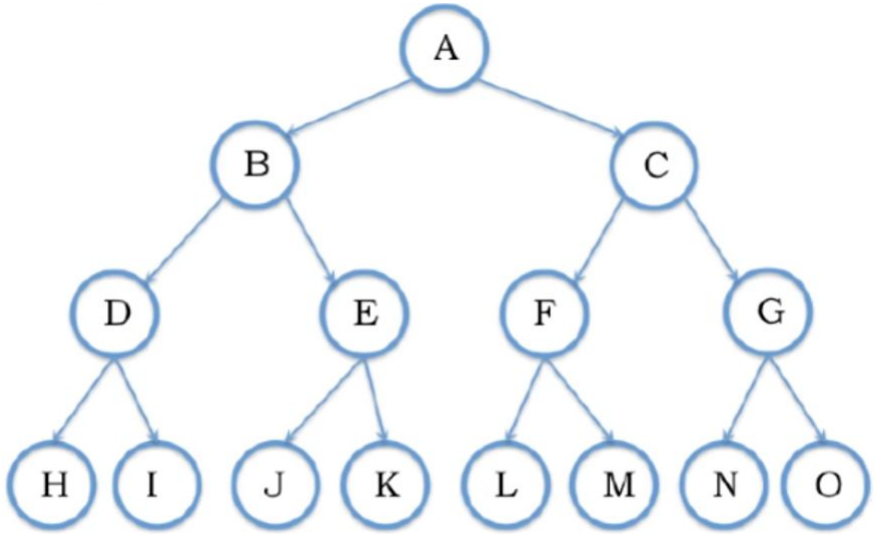


Elements Popped From Stack:
A □ C □ G □ O

Elements pushed on stack

Stack
~~O~~
N
~~G~~
F
~~C~~
B
~~A~~

Depth First Search (DFS)

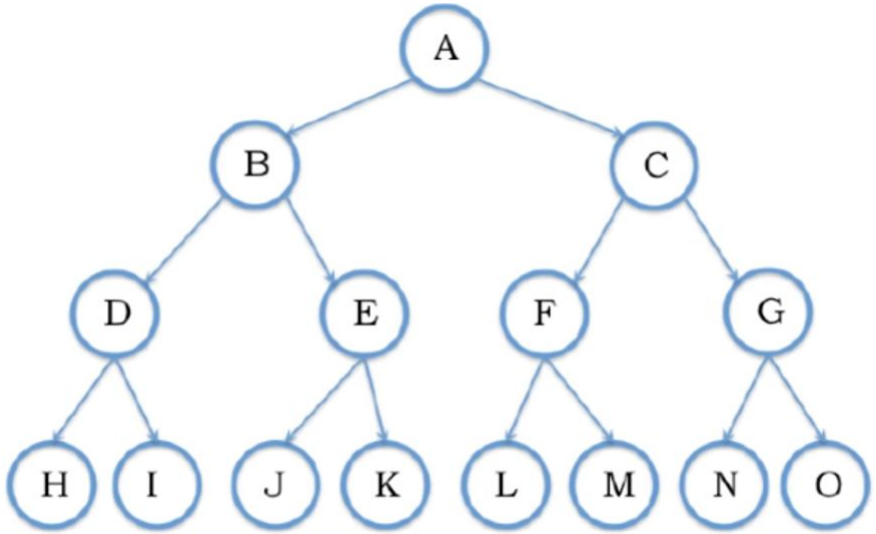


Elements Popped From Stack:
A □ C □ G □ O □ N

Elements pushed on stack

Stack
~~O~~
~~N~~
~~G~~
F
~~C~~
B
~~A~~

Depth First Search (DFS)

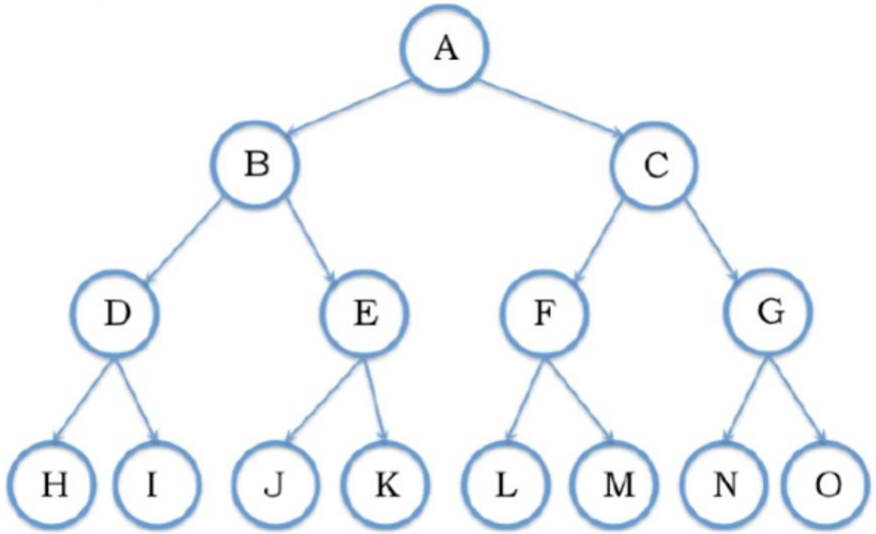


Elements Popped From Stack:
A □ C □ G □ O □ N □ F

Elements pushed on stack

Stack
M
L
~~O~~
~~N~~
~~G~~
~~F~~
~~C~~
B
~~A~~

Depth First Search (DFS)

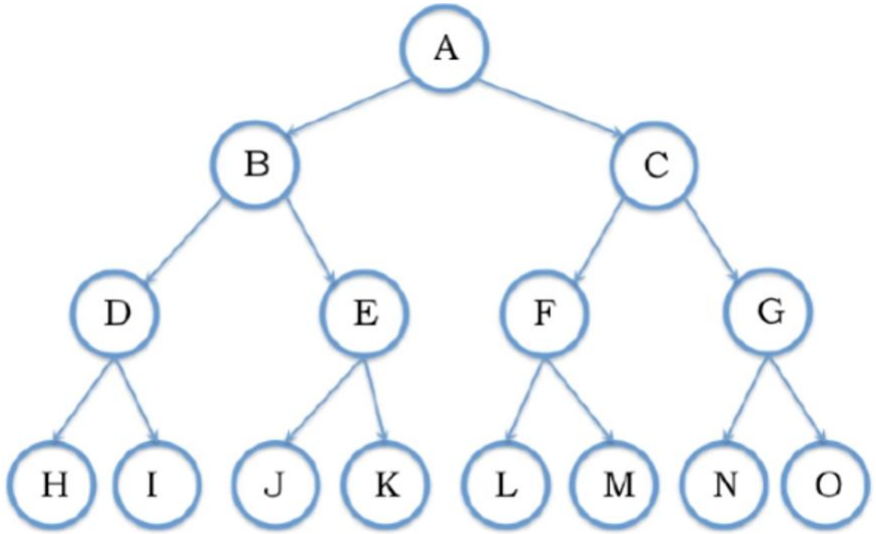


Elements Popped From Stack:
A □ C □ G □ O □ N □ F □ M

Elements pushed on stack

Stack
~~M~~
L
~~O~~
~~N~~
~~G~~
~~F~~
~~C~~
B
~~A~~

Depth First Search (DFS)

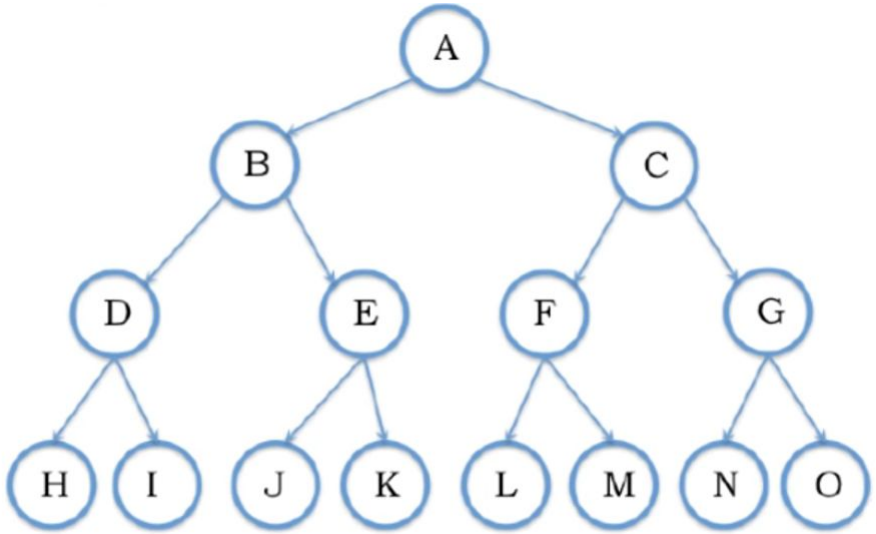


Elements Popped From Stack:
A □ C □ G □ O □ N □ F □ M □ L

Elements pushed on stack

Stack
~~M~~
~~L~~
~~O~~
~~N~~
~~G~~
~~F~~
~~C~~
B
~~A~~

Depth First Search (DFS)

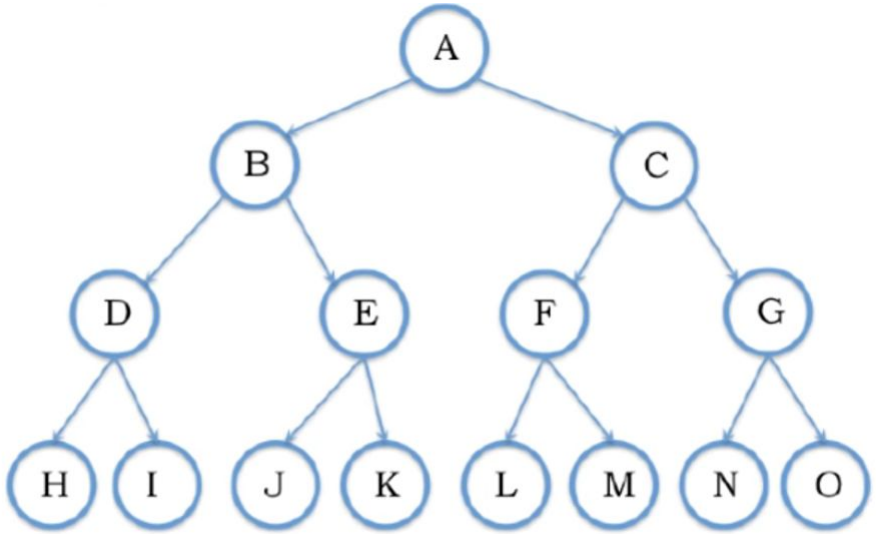


Elements Popped From Stack:
A □ C □ G □ O □ N □ F □ M □ B

Elements pushed on stack

Stack
E
D
~~M~~
~~L~~
~~O~~
~~N~~
~~G~~
~~F~~
~~C~~
~~B~~
~~A~~

Depth First Search (DFS)



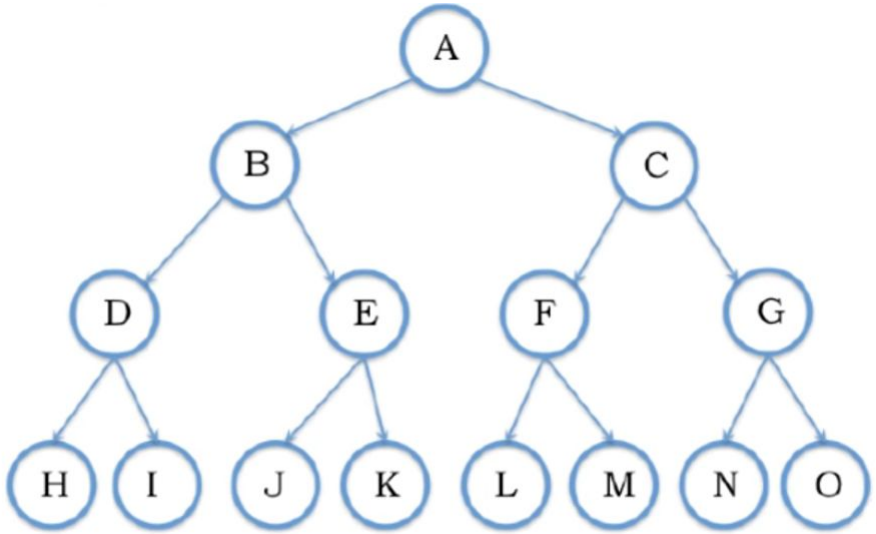
Elements Popped From Stack:

A ☐ C ☐ G ☐ O ☐ N ☐ F ☐ M ☐ B ☐ E

Elements pushed on stack

Stack
K
J
~~E~~
D
~~M~~
~~L~~
~~O~~
~~N~~
~~G~~
~~F~~
~~C~~
~~B~~
~~A~~

Depth First Search (DFS)



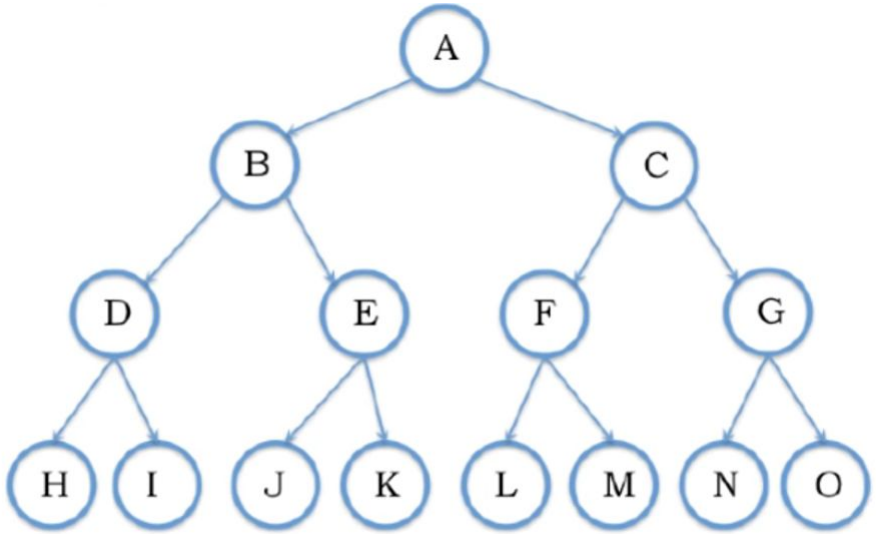
Elements Popped From Stack:

A ☐ C ☐ G ☐ O ☐ N ☐ F ☐ M ☐ B ☐ E ☐ K

Elements pushed on stack

Stack
~~K~~
J
~~E~~
D
~~M~~
~~L~~
~~O~~
~~N~~
~~G~~
~~F~~
~~C~~
~~B~~
~~A~~

Depth First Search (DFS)



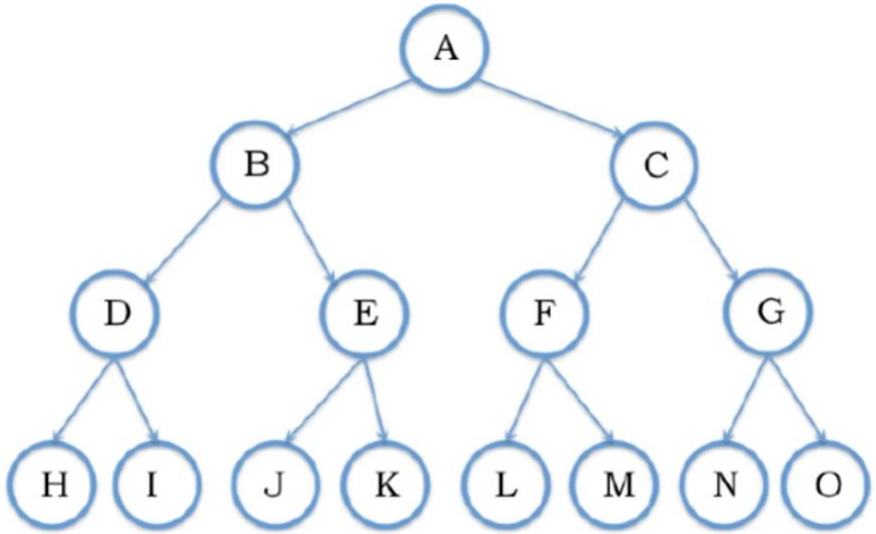
Elements Popped From Stack:

A ☐ C ☐ G ☐ O ☐ N ☐ F ☐ M ☐ B ☐ E ☐ K ☐ J

Elements pushed on stack

Stack
~~K~~
~~J~~
~~E~~
D
~~M~~
~~L~~
~~O~~
~~N~~
~~G~~
~~F~~
~~C~~
~~B~~
~~A~~

Depth First Search (DFS)



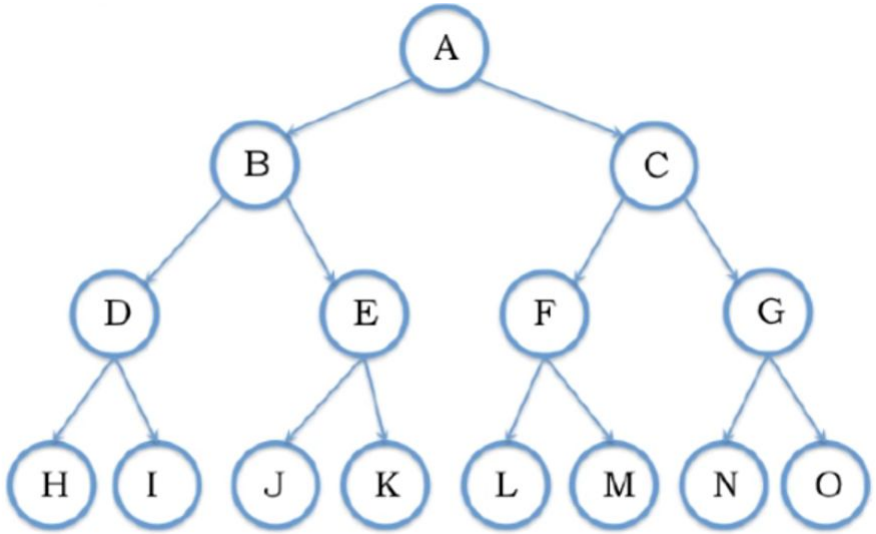
Elements Popped From Stack:

A □ C □ G □ O □ N □ F □ M □ B □ E □ K □ J □D

Elements pushed on stack

Stack
I
H
~~K~~
~~J~~
~~E~~
~~D~~
~~M~~
~~L~~
~~O~~
~~N~~
~~G~~
~~F~~
~~C~~
~~B~~
~~A~~

Depth First Search (DFS)



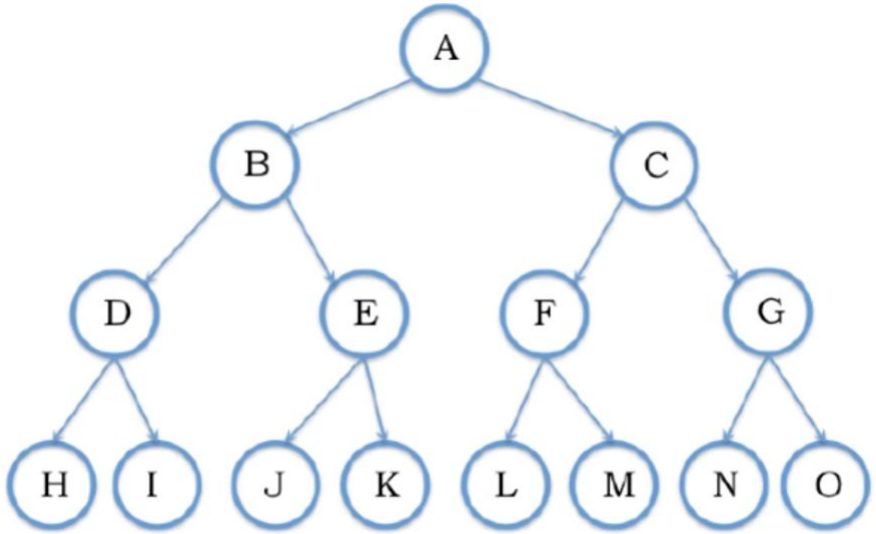
Elements Popped From Stack:

A ☐ C ☐ G ☐ O ☐ N ☐ F ☐ M ☐ B ☐ E ☐ K ☐ J ☐ D ☐ I

Elements pushed on stack

Stack
~~I~~
H
~~K~~
~~J~~
~~E~~
~~D~~
~~M~~
~~L~~
~~O~~
~~N~~
~~G~~
~~F~~
~~C~~
~~B~~
~~A~~

Depth First Search (DFS)



Elements Popped From Stack:

A ☐ C ☐ G ☐ O ☐ N ☐ F ☐ M ☐ B ☐ E ☐ K ☐ J ☐ D ☐ I ☐ H

Elements pushed on stack

Stack
~~I~~
~~H~~
~~K~~
~~J~~
~~E~~
~~D~~
~~M~~
~~L~~
~~O~~
~~N~~
~~G~~
~~F~~
~~C~~
~~B~~
~~A~~

Depth First Search (DFS)

•

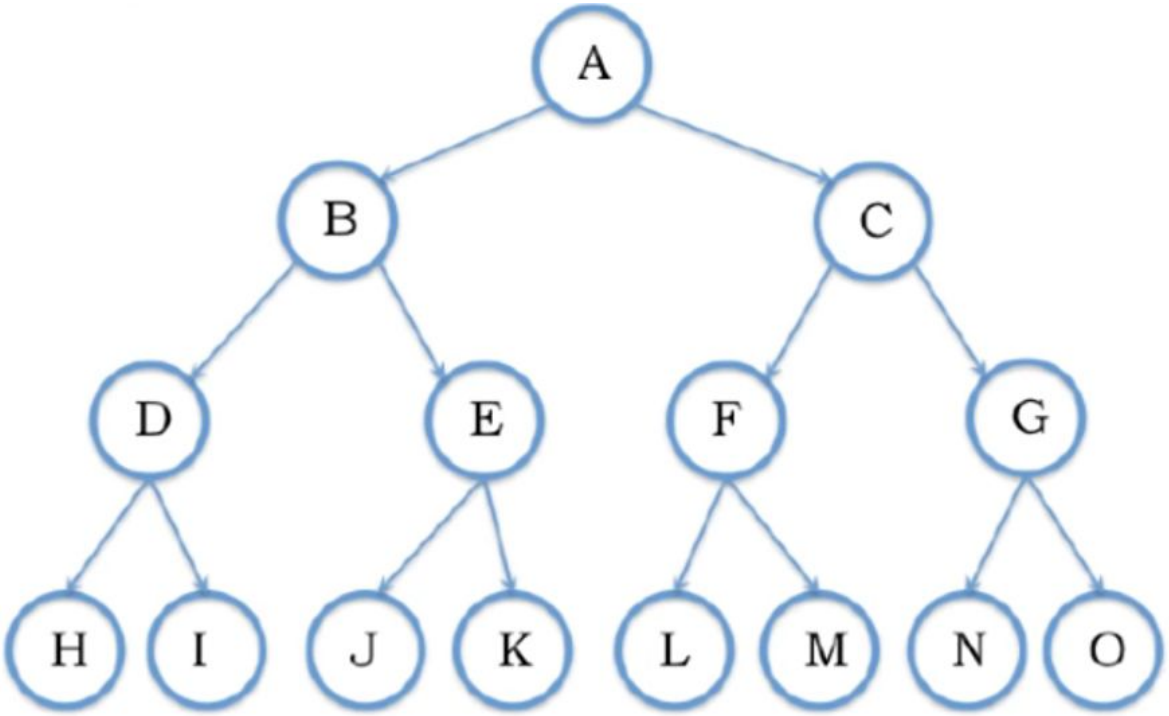
Depth-Limited Search (DLS)

- **DFS with a depth bound**
  - Searching is not permitted beyond the depth bound.

- Works well if we know what is the depth of the solution.

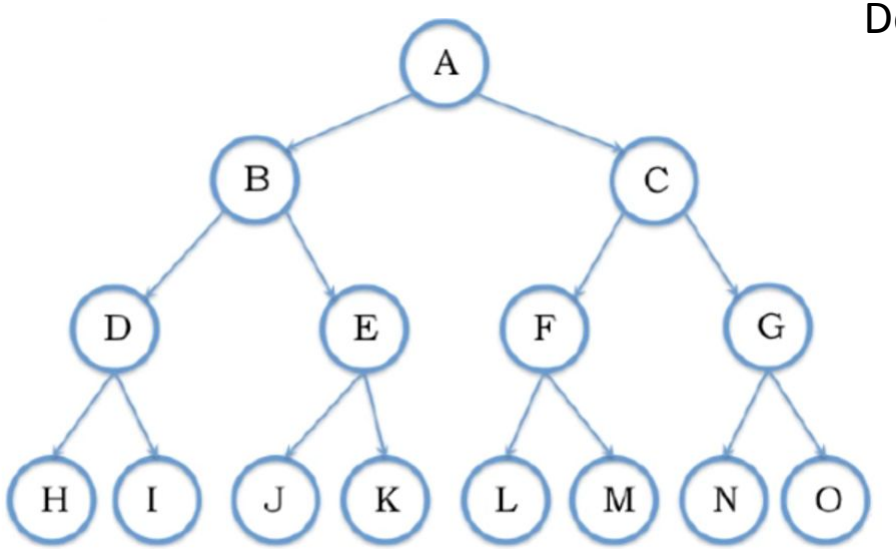
- If the solution is beneath the depth bound, the search cannot find the goal (hence this search algorithm is not complete).

Depth Limited Search (DLS)



Depth Limited Search (DLS)

Depth Limit = 2



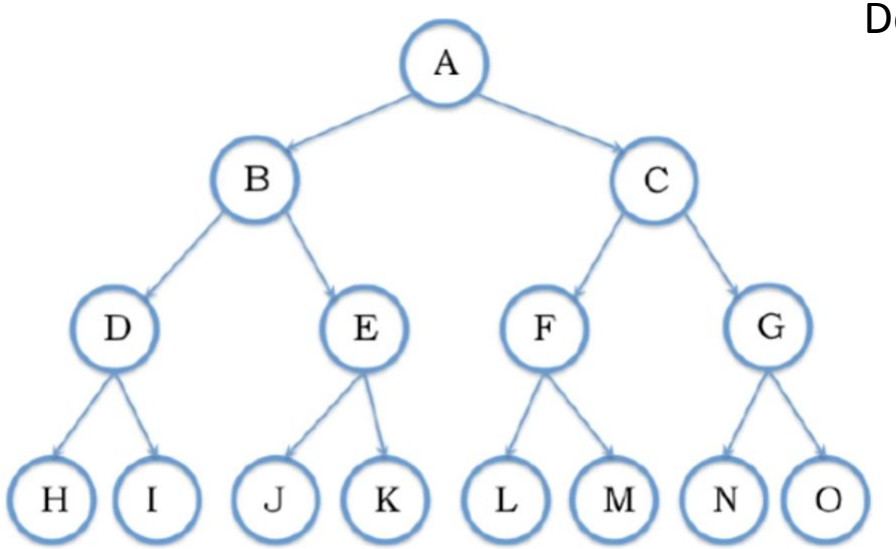
Elements Popped From Stack:

Elements pushed on stack

Stack
A

Depth Limited Search (DLS)

Depth Limit = 2



Elements Popped From Stack:

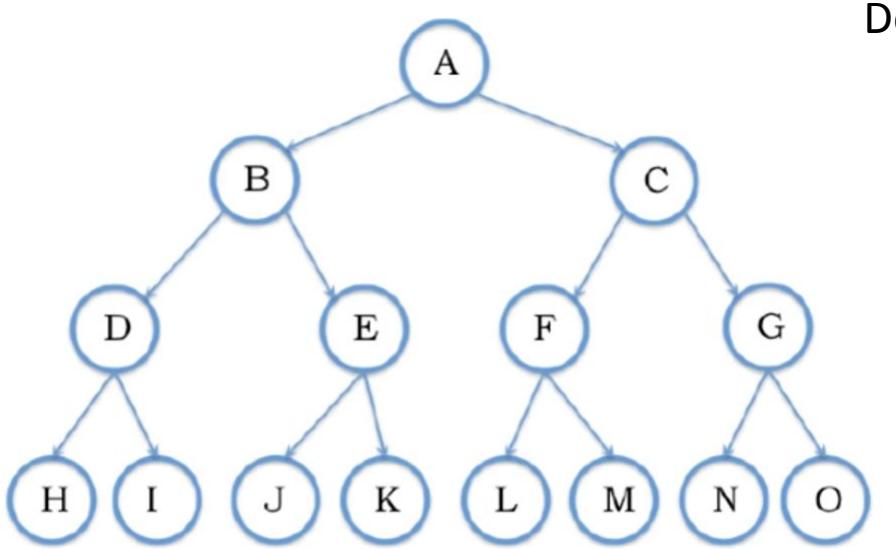
A

Elements pushed on stack

Stack
C
B
~~A~~

Depth Limited Search (DLS)

Depth Limit = 2



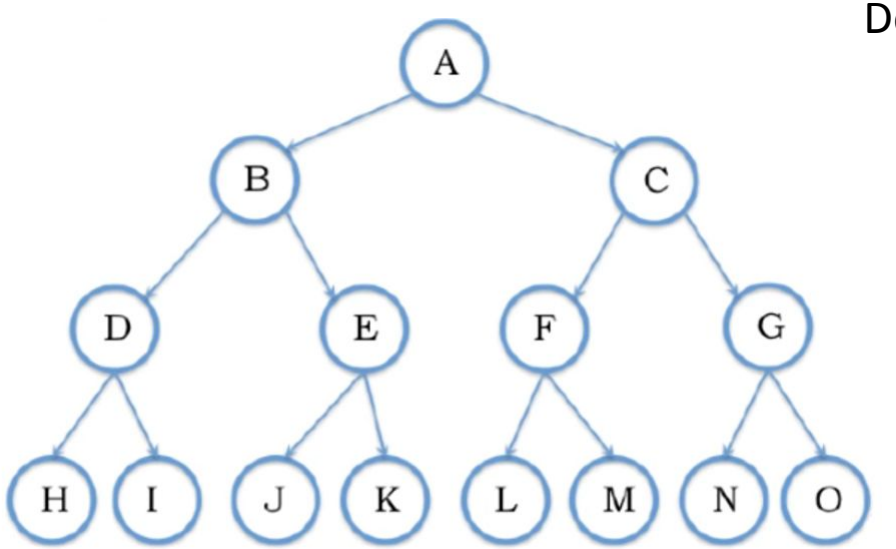
Elements Popped From Stack:
A □ C □

Elements pushed on stack

Stack
G
F
~~C~~
B
~~A~~

Depth Limited Search (DLS)

Depth Limit = 2



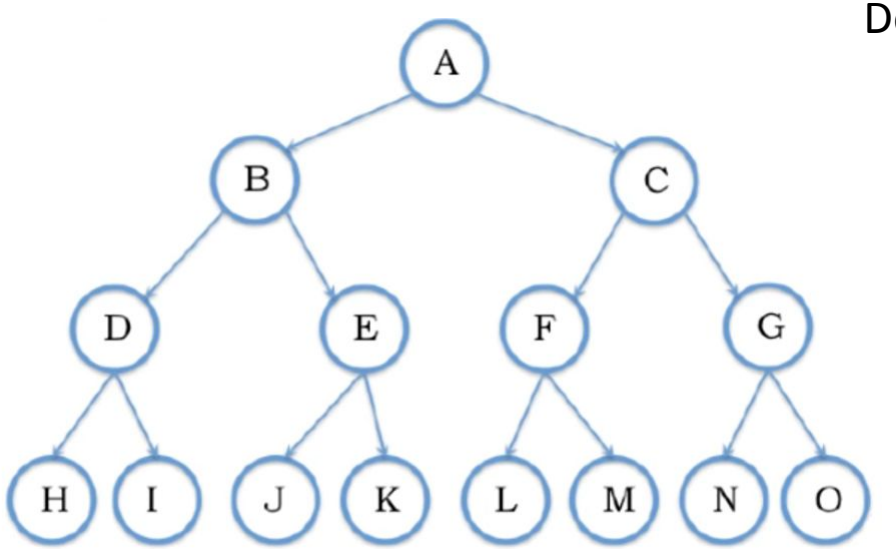
Elements Popped From Stack:
A □ C □ G

Elements pushed on stack

Stack
~~G~~
F
~~C~~
B
~~A~~

Depth Limited Search (DLS)

Depth Limit = 2



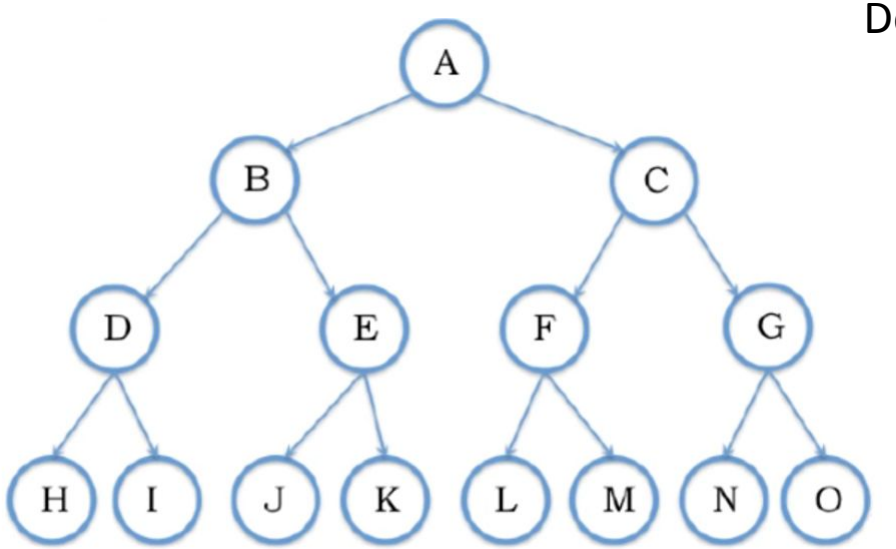
Elements Popped From Stack:
A □ C □ G □F

Elements pushed on stack

Stack
~~G~~
~~F~~
~~C~~
B
~~A~~

Depth Limited Search (DLS)

Depth Limit = 2



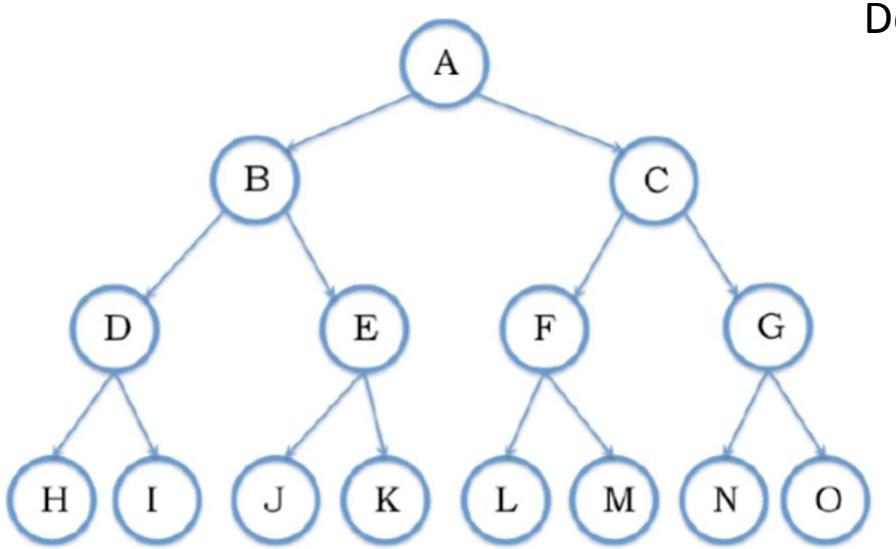
Elements Popped From Stack:
A ☐ C ☐ G ☐ F ☐ B

Elements pushed on stack

Stack
E
D
~~G~~
~~F~~
~~C~~
~~B~~
~~A~~

Depth Limited Search (DLS)

Depth Limit = 2



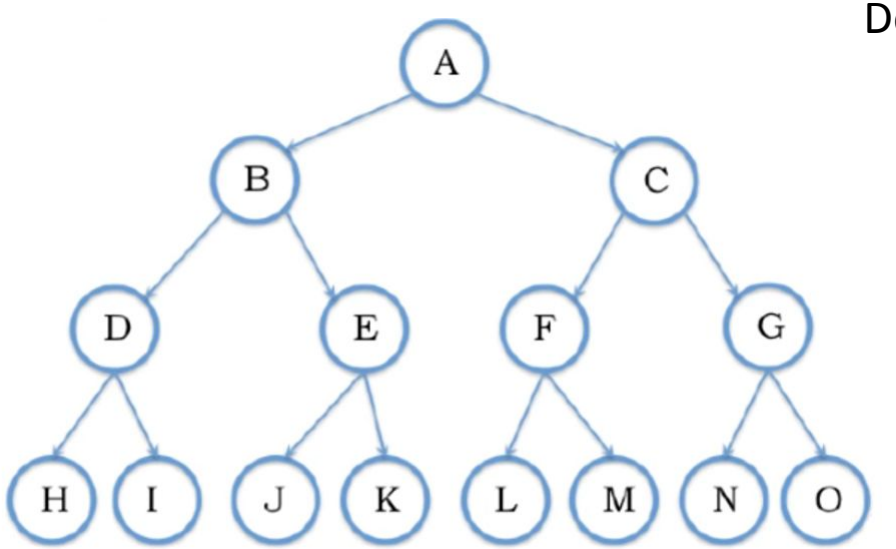
Elements Popped From Stack:
A ☐ C ☐ G ☐ F ☐ B ☐ E

Elements pushed on stack

Stack
~~E~~
D
~~G~~
~~F~~
~~C~~
~~B~~
~~A~~

Depth Limited Search (DLS)

Depth Limit = 2



Elements Popped From Stack:
A ☐ C ☐ G ☐ F ☐ B ☐ E ☐D

Elements pushed on stack

Stack
~~E~~
~~D~~
~~G~~
~~F~~
~~C~~
~~B~~
~~A~~

Depth-Limited Search (DLS)

•

Iterative Deepening Search

- To avoid the infinite depth problem of DFS:
  - Only search until depth L
  - i.e, don’t expand nodes beyond depth L
  - Depth-Limited Search
- What if solution is deeper than L?
  - Increase depth iteratively
  - Iterative Deepening Search
- IDS
  - Inherits the memory advantage of depth-first search
  - Has the completeness property of breadth-first search

Depth Bound = 0

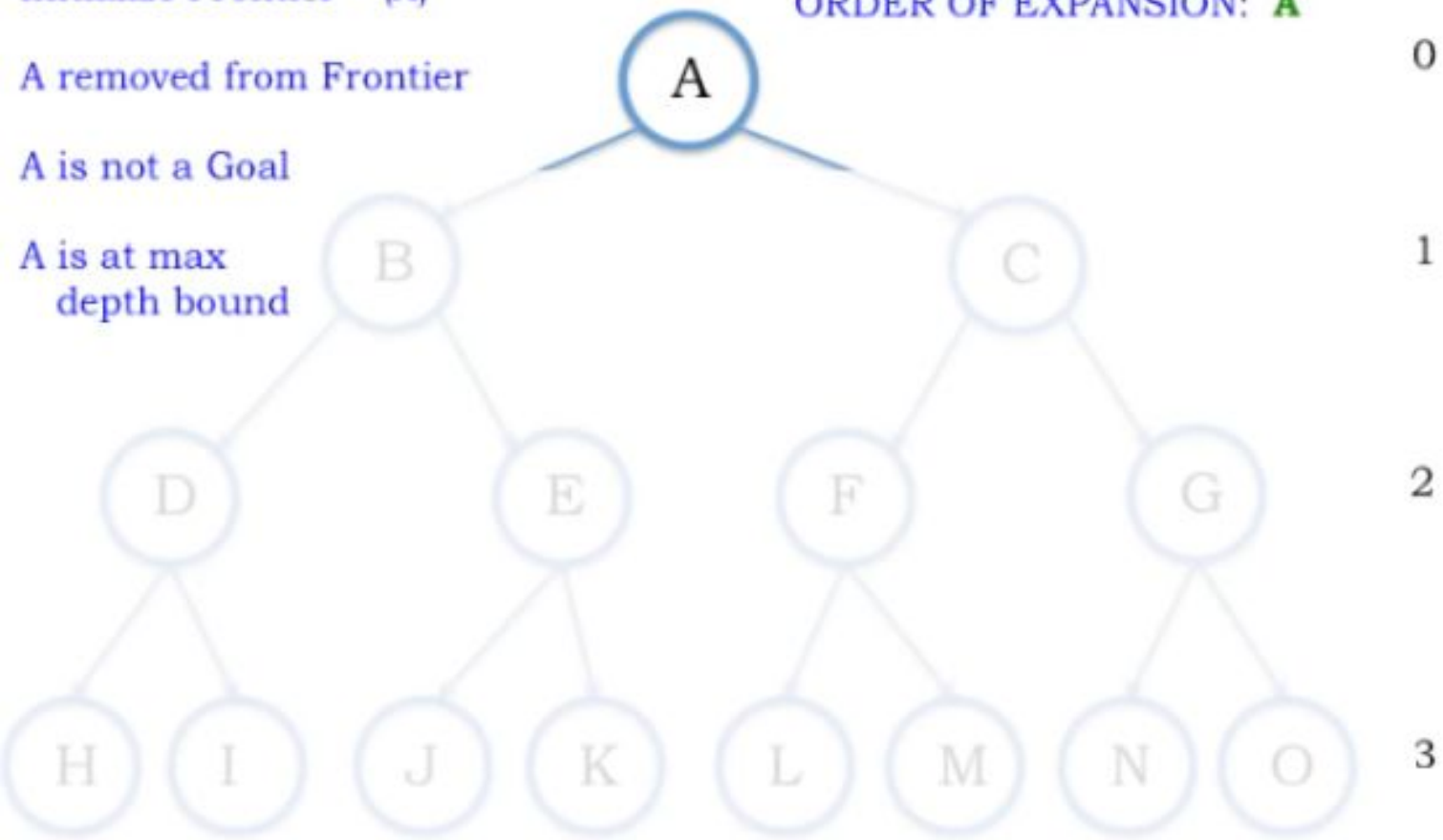
Initialize Frontier = {A}

A removed from Frontier

A is not a Goal

A is at max depth bound

ORDER OF EXPANSION: **A**



Elements pushed on stack

Stack
A

Depth Bound = 0

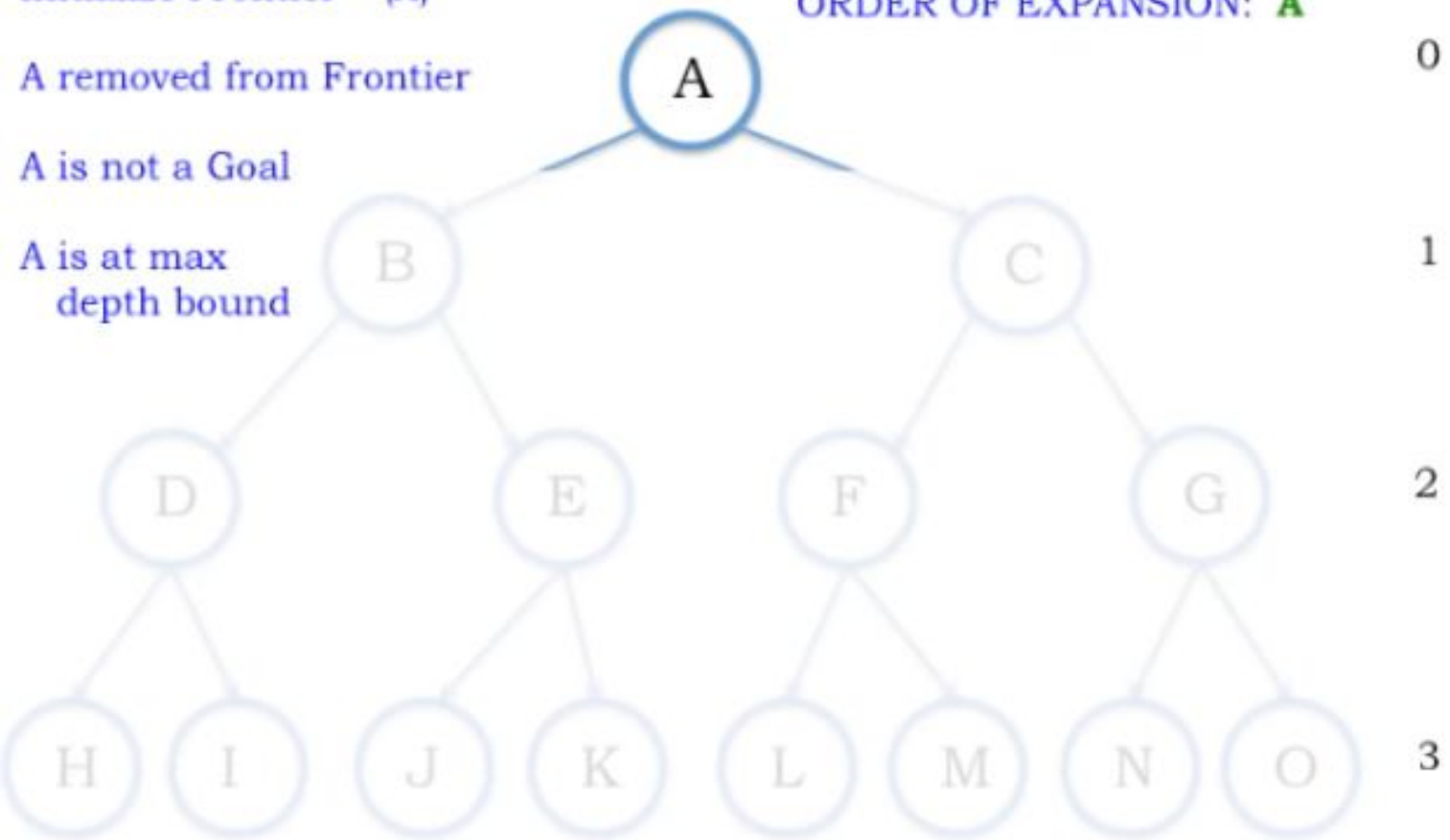
Initialize Frontier = {A}

A removed from Frontier

A is not a Goal

A is at max depth bound

ORDER OF EXPANSION: A



Elements pushed on stack

Stack
~~A~~

Depth Bound = 1

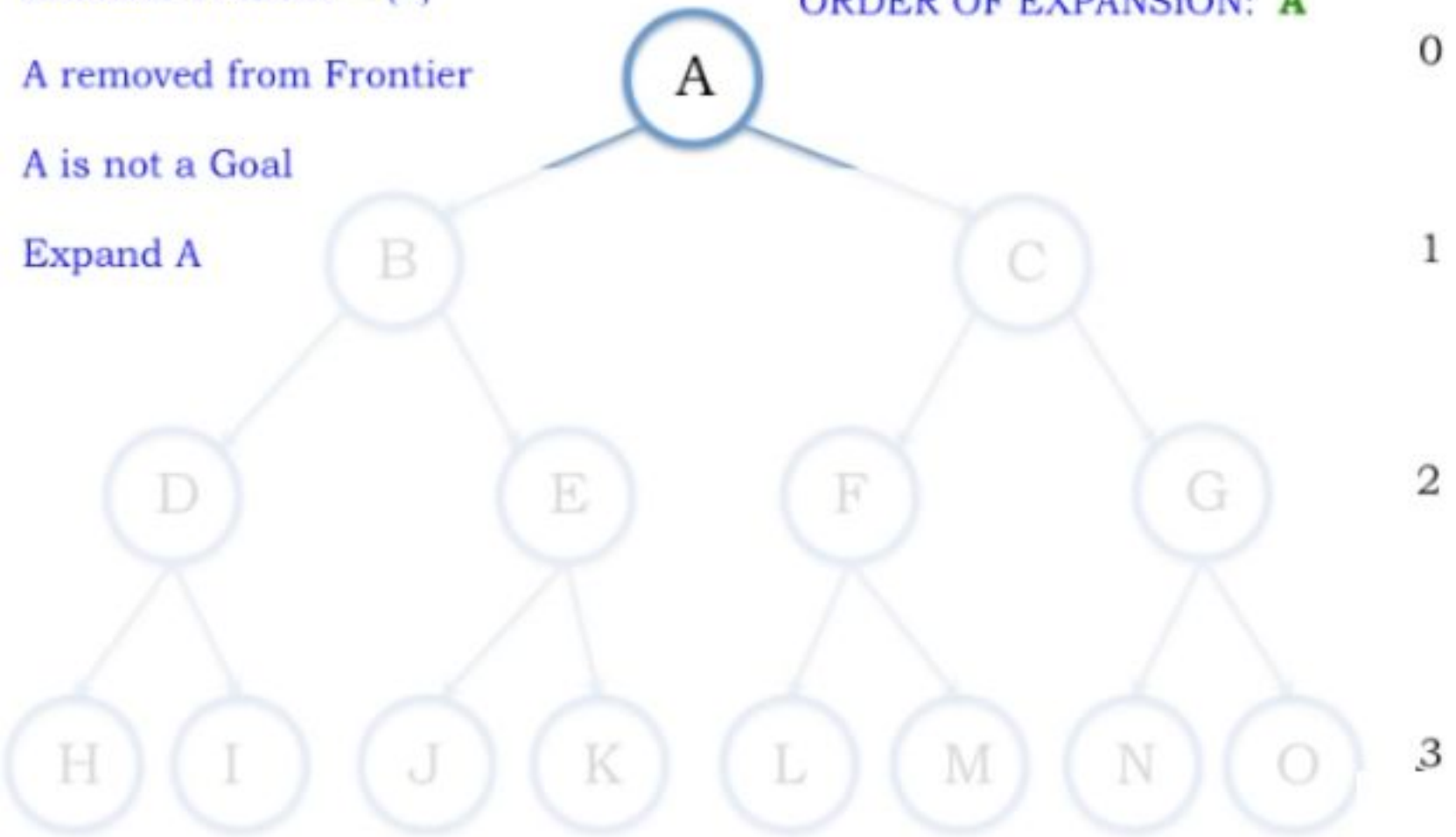
Initialize Frontier = {A}

A removed from Frontier

A is not a Goal

Expand A

ORDER OF EXPANSION: **A**



Elements pushed on stack

Stack
A

Depth Bound = 1

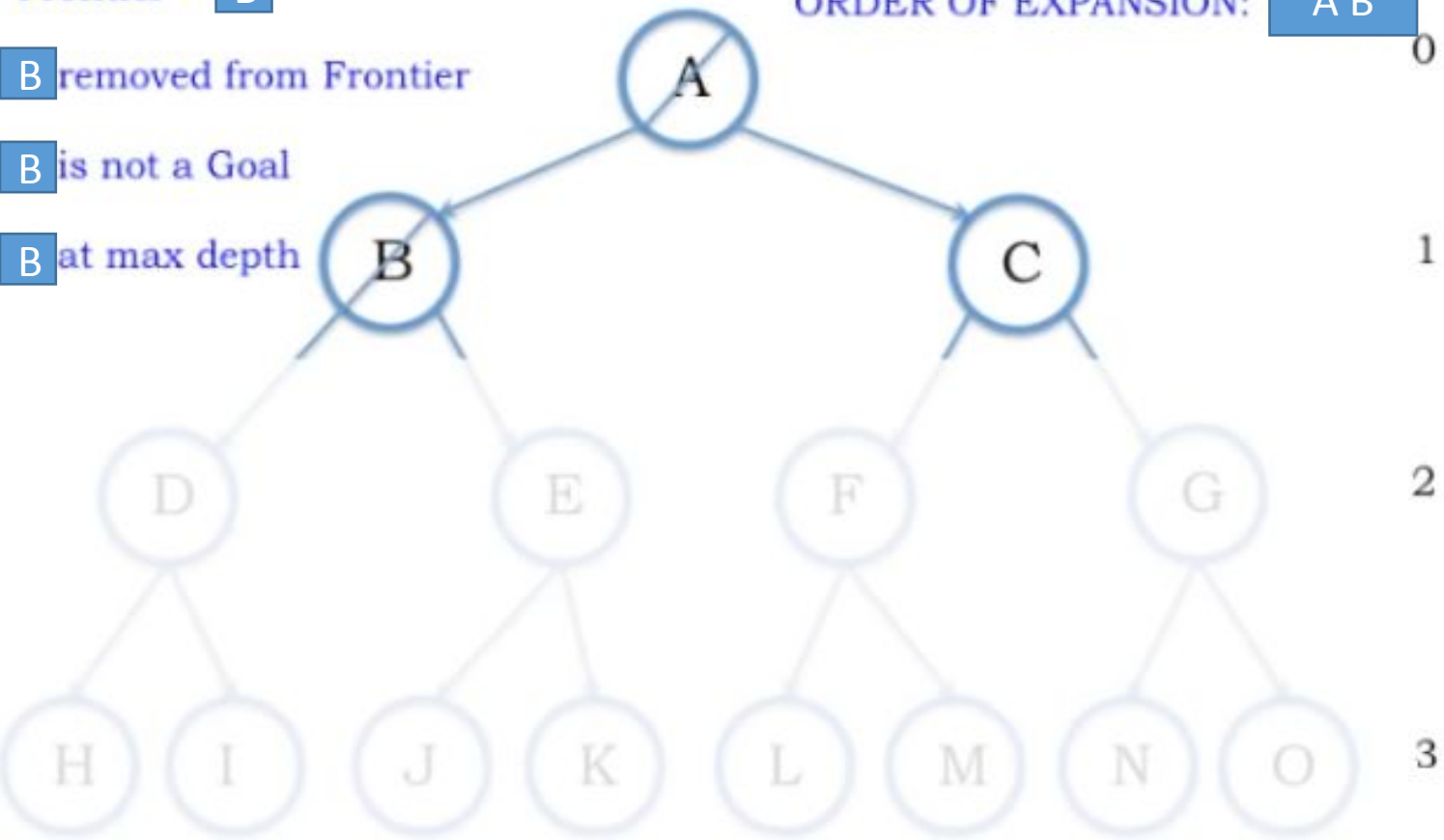
Frontier = B

B removed from Frontier

B is not a Goal

B at max depth

ORDER OF EXPANSION: A B



Elements pushed on stack

Stack
~~B~~
C
~~A~~

Depth Bound = 1

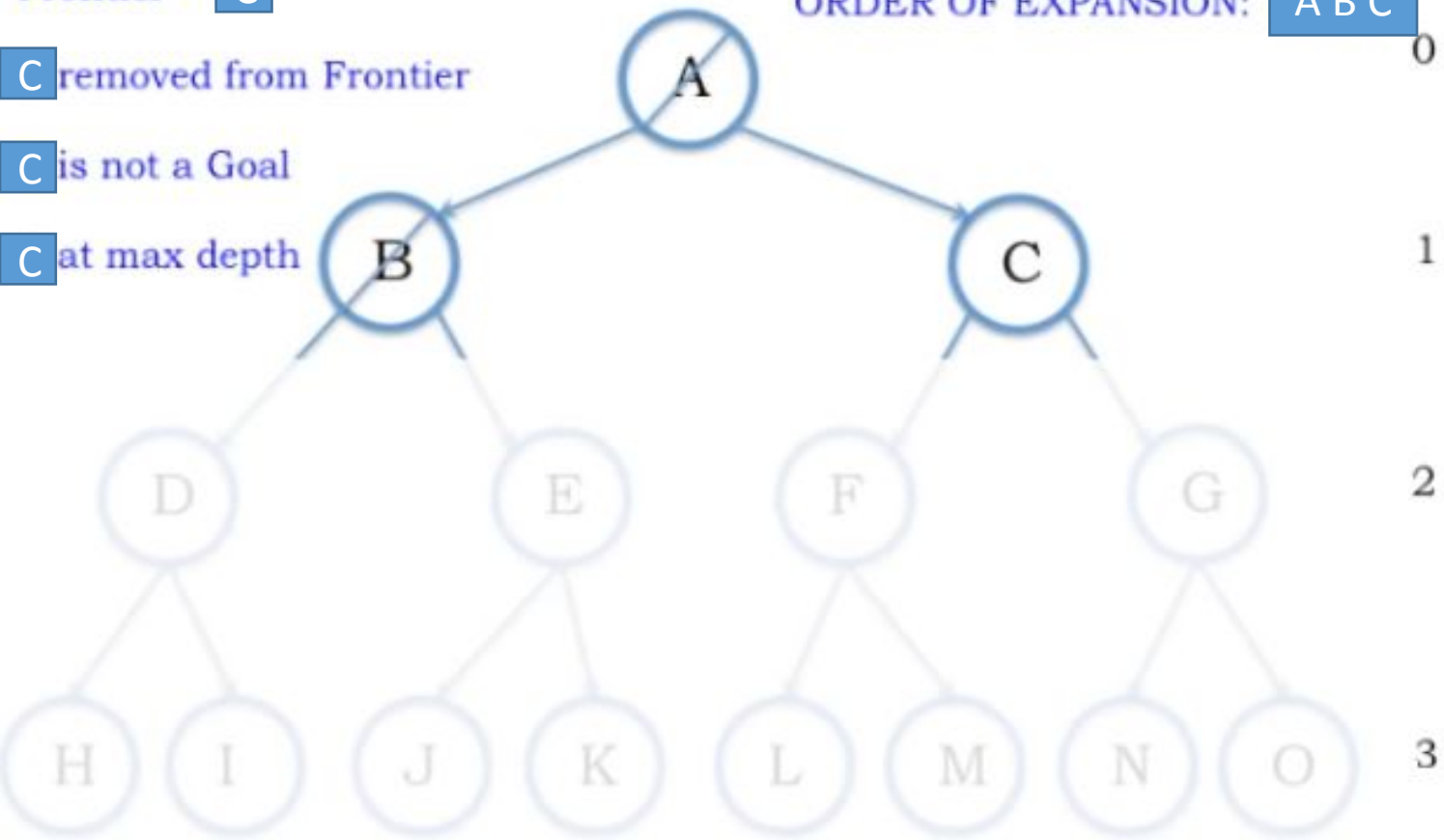
Frontier = C

C removed from Frontier

C is not a Goal

C at max depth

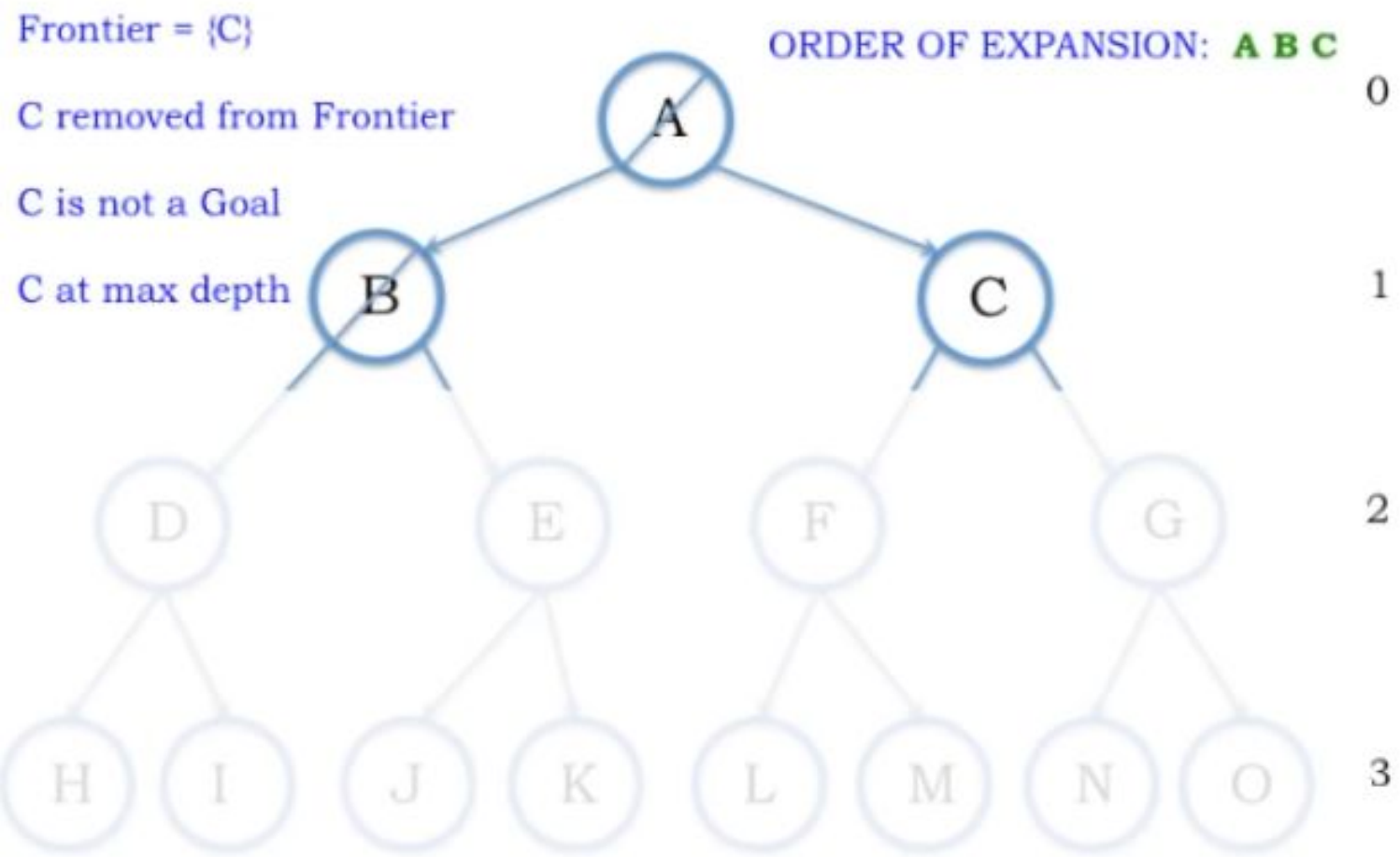
ORDER OF EXPANSION: A B C



Elements pushed on stack

Stack
~~B~~
~~C~~
~~A~~

Depth Bound = 1



Elements pushed on stack

Stack
~~B~~
~~C~~
~~A~~

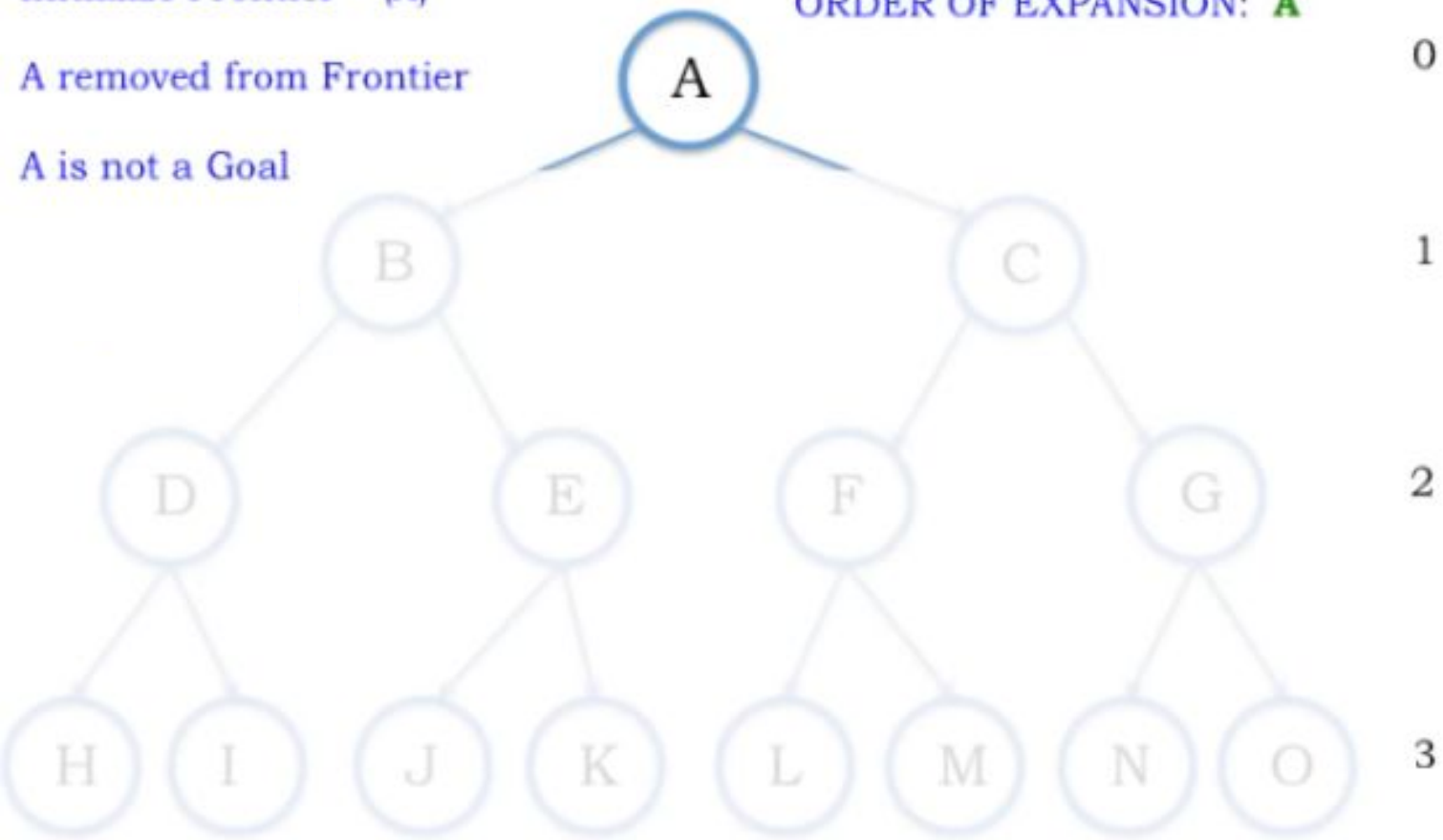
Depth Bound = 2

Initialize Frontier = {A}

A removed from Frontier

A is not a Goal

ORDER OF EXPANSION: A



Elements pushed on stack

Stack
A

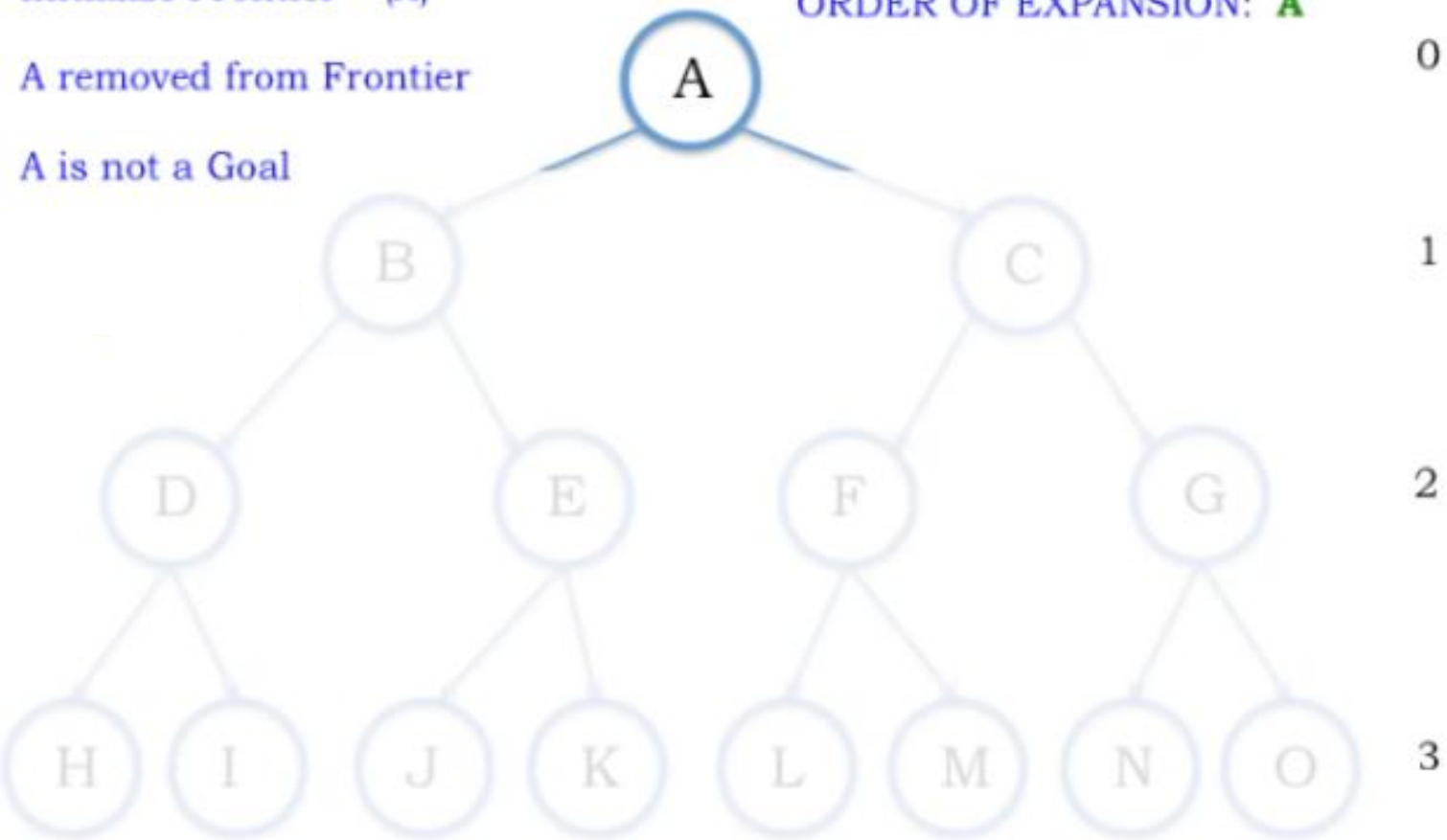
Depth Bound = 2

Initialize Frontier = {A}

A removed from Frontier

A is not a Goal

ORDER OF EXPANSION: **A**



Elements pushed on stack

Stack
B
C
~~A~~

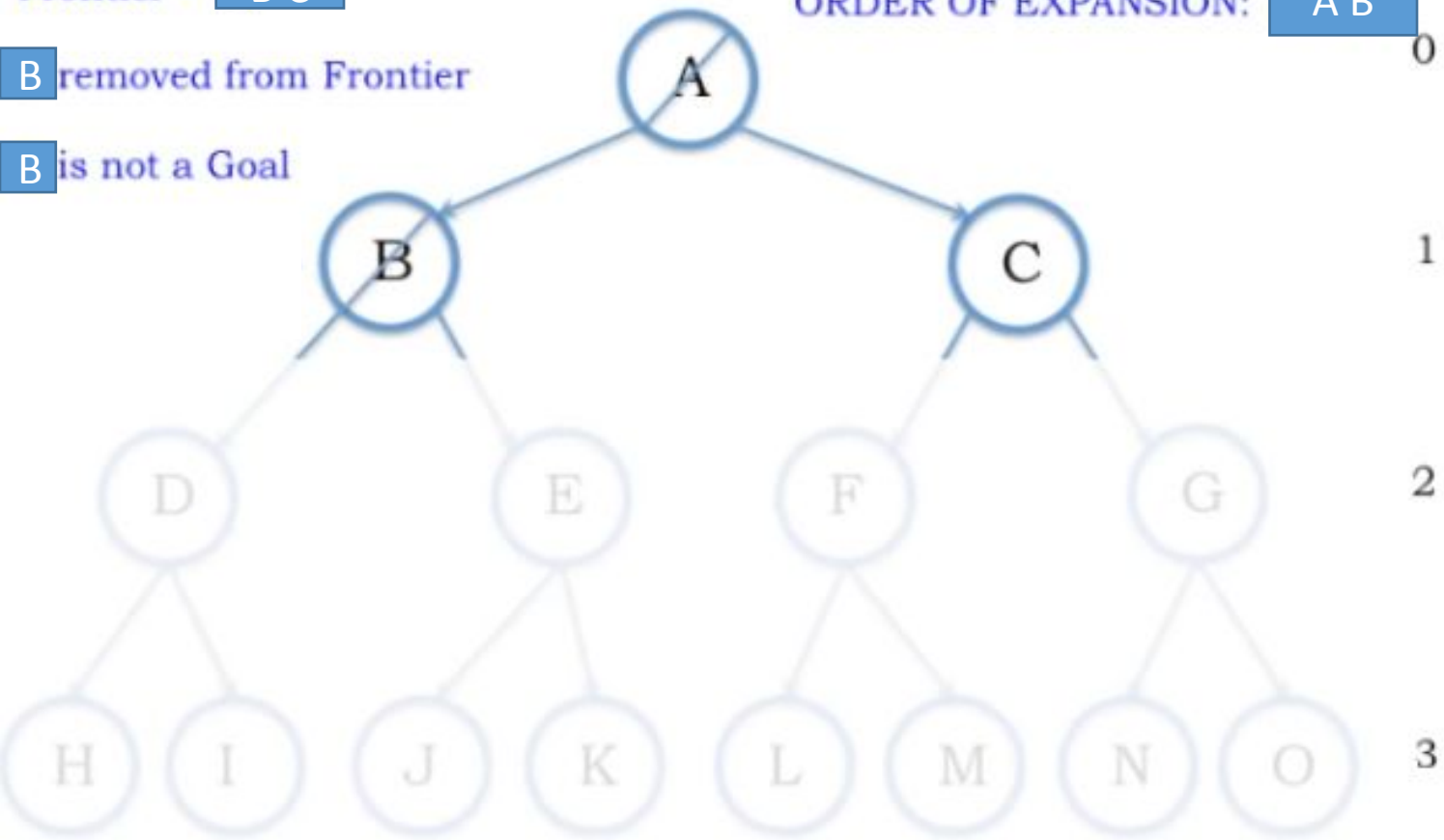
Depth Bound = 2

Frontier = B C

ORDER OF EXPANSION: A B

B removed from Frontier

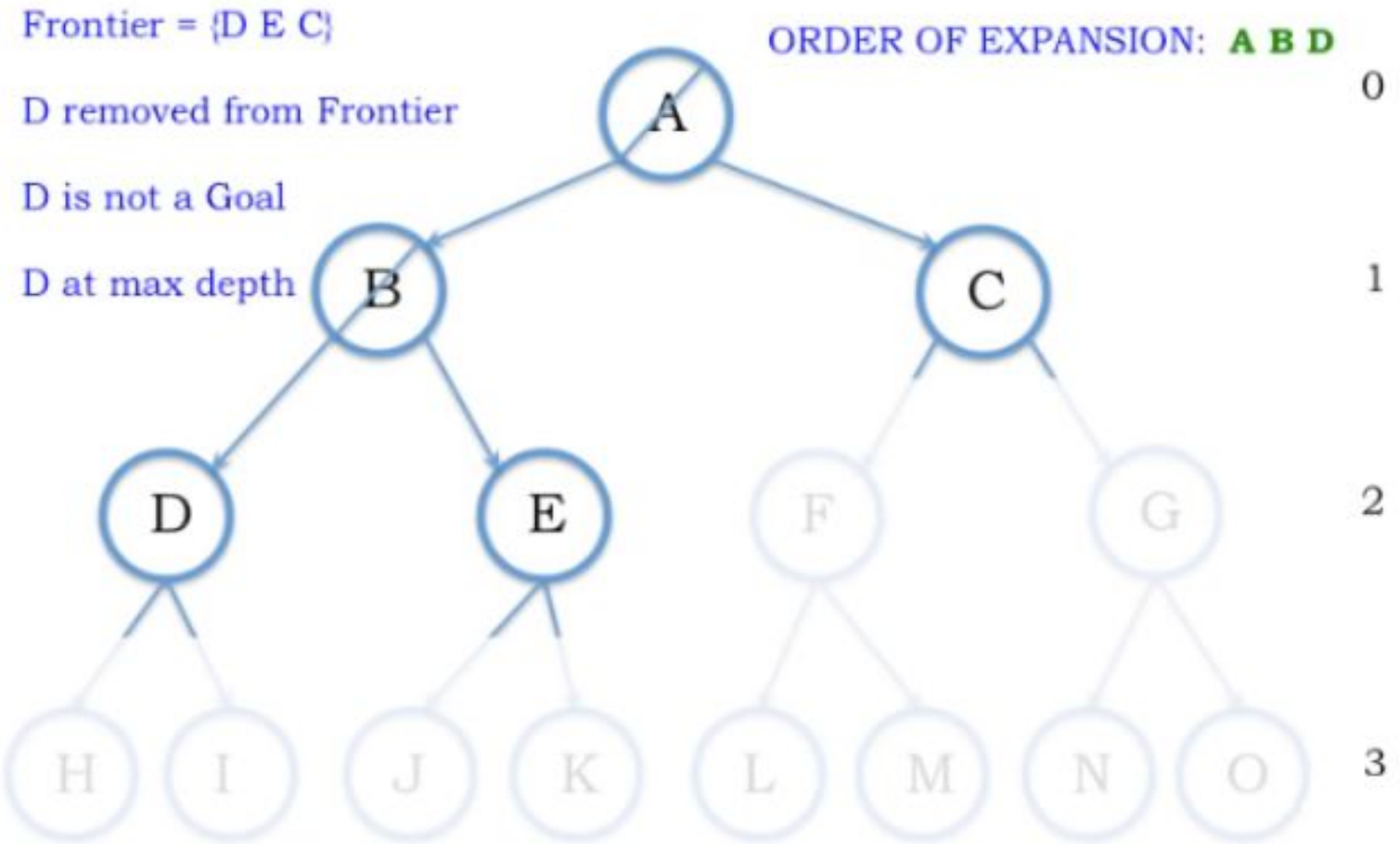
B is not a Goal



Elements pushed on stack

Stack
D
E
~~B~~
C
~~A~~

Depth Bound = 2



Elements pushed on stack

Stack
~~D~~
E
~~B~~
C
~~A~~

Depth Bound = 2

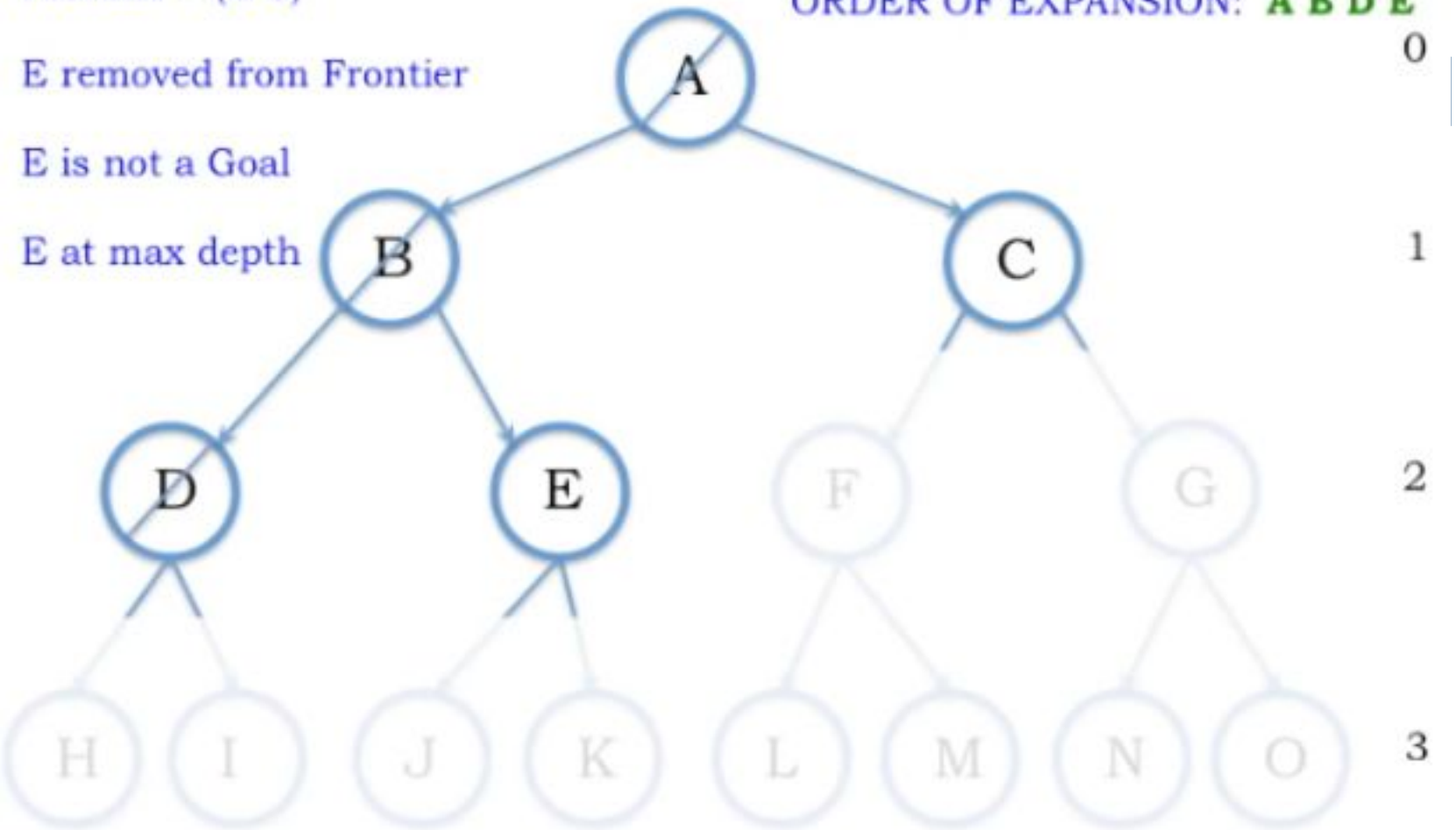
Frontier = {E C}

E removed from Frontier

E is not a Goal

E at max depth

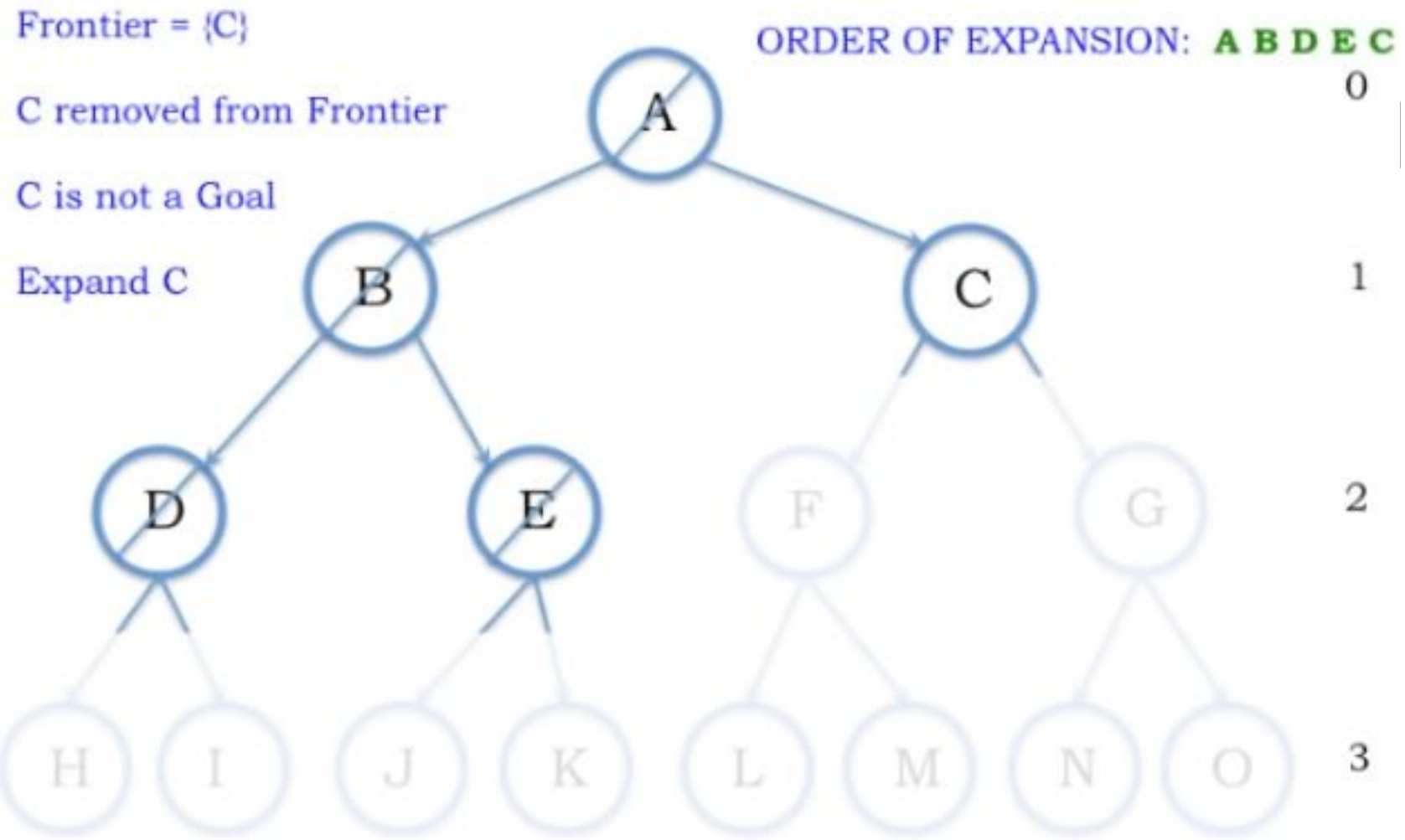
ORDER OF EXPANSION: **A B D E**



Elements pushed on stack

Stack
~~D~~
~~E~~
~~B~~
C
~~A~~

Depth Bound = 2



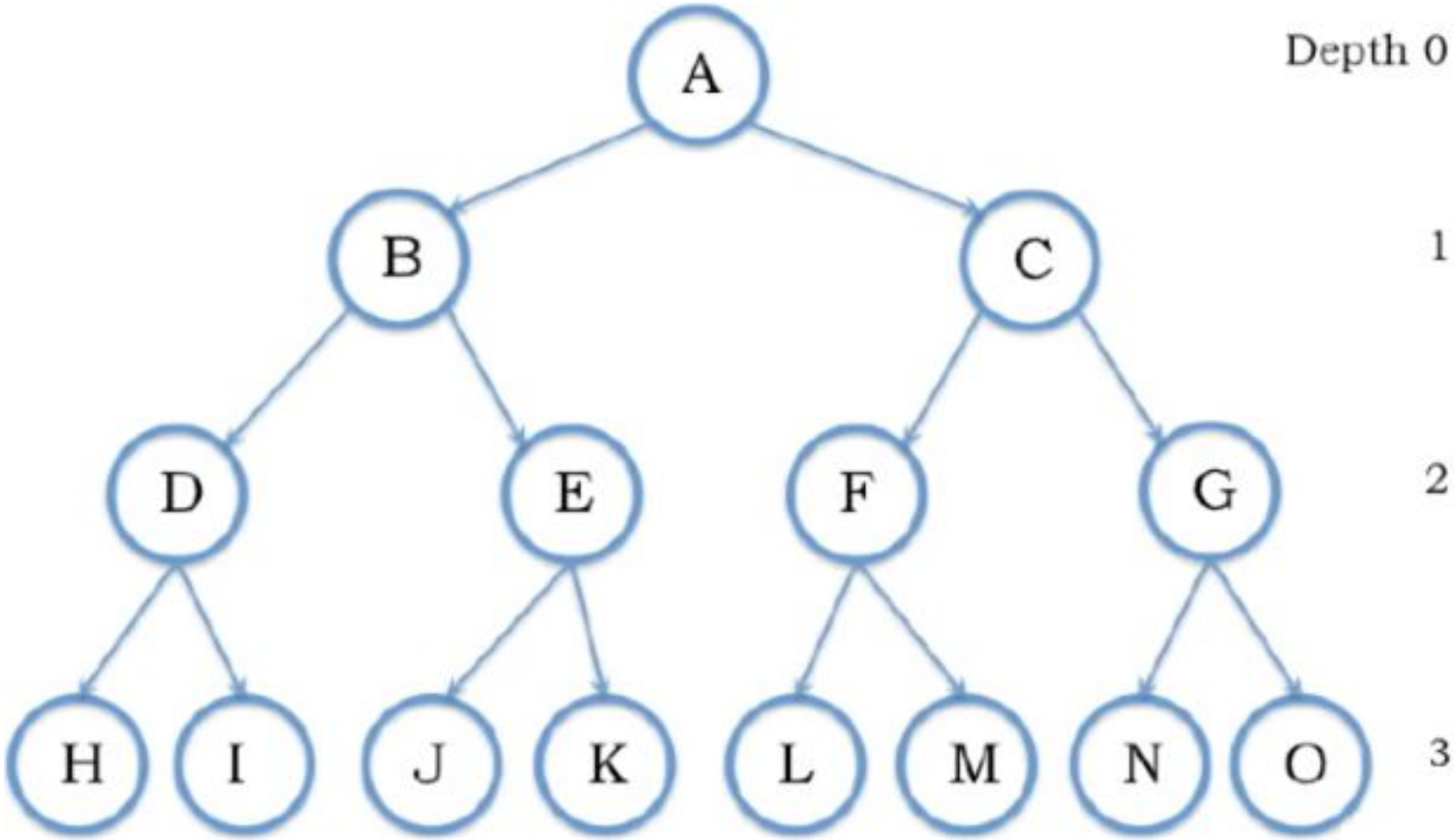
Elements pushed on stack

Stack
E
D
B
C
A

Depth Bound = 3

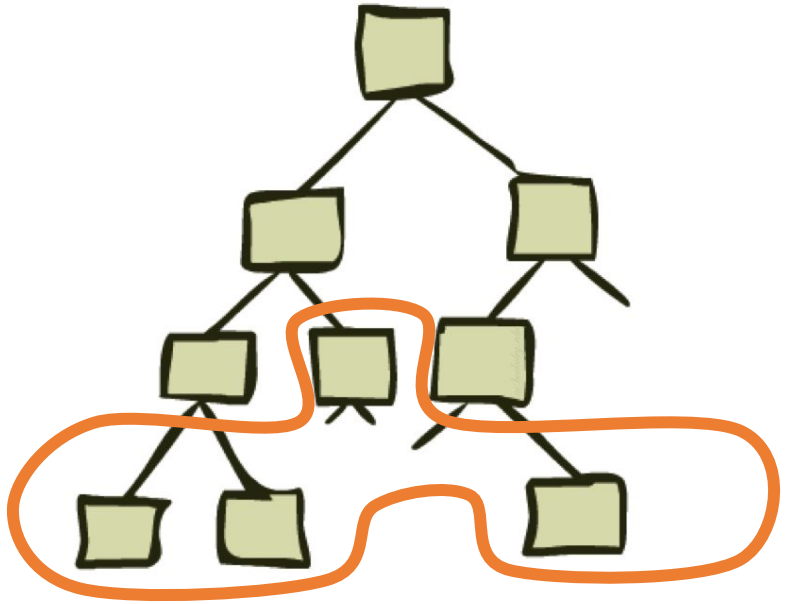
Over ALL the iterations, from depth bound 0 to 3, the order in which nodes removed from the frontier is:

A A B C A B D E C F G A B D H I E J K C F L M G N O



Uninformed Search-II

Depth First Search and its Variants



Depth-Limited Search (DLS)

- **DFS with a depth bound**
  - Searching is not permitted beyond the depth bound.

- Works well if we know what is the depth of the solution.

- If the solution is beneath the depth bound, the search cannot find the goal (hence this search algorithm is not complete).

Depth-Limited Search (DLS)

- **Main idea***:*
  - *Expand node at the deepest level, but limit depth to D.*
- **Implementation**:
  - *Enqueue nodes in LIFO (last-in, first-out) order. But limit depth to D*

- *Complete?*
  - No
  - Yes: if there is a goal state at a depth less than D
- Optimal?
  - No
- Time Complexity:
  - $O(b^D)$, where D is the cutoff.
- Space Complexity:
  - $O(b^D)$, where D is the cutoff.

Iterative Deepening Search

- To avoid the infinite depth problem of DFS:
  - Only search until depth L
  - i.e, don’t expand nodes beyond depth L
  - Depth-Limited Search
- What if solution is deeper than L?
  - Increase depth iteratively
  - Iterative Deepening Search
- IDS
  - Inherits the memory advantage of depth-first search
  - Has the completeness property of breadth-first search

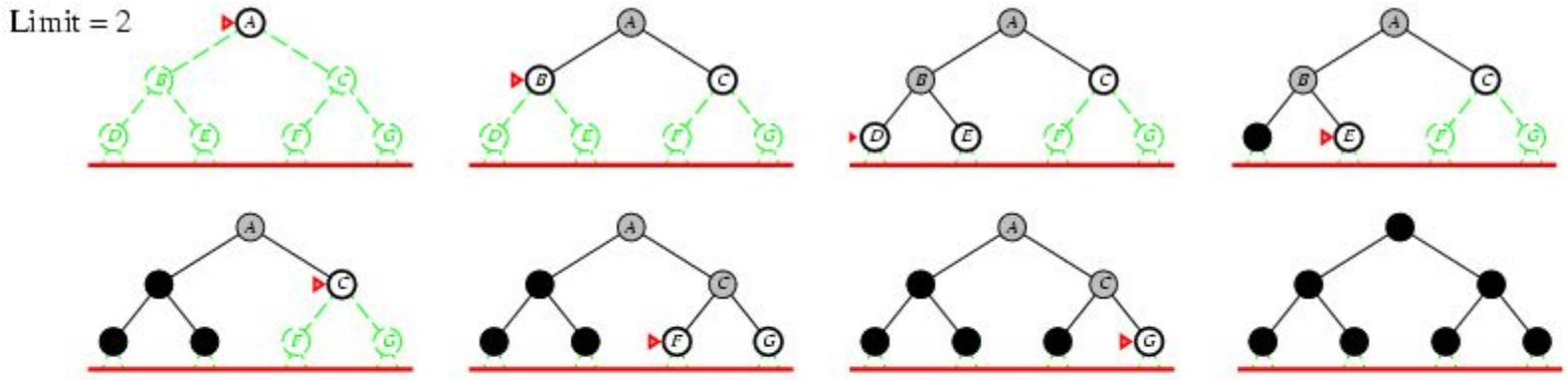
Iterative deepening search l =0



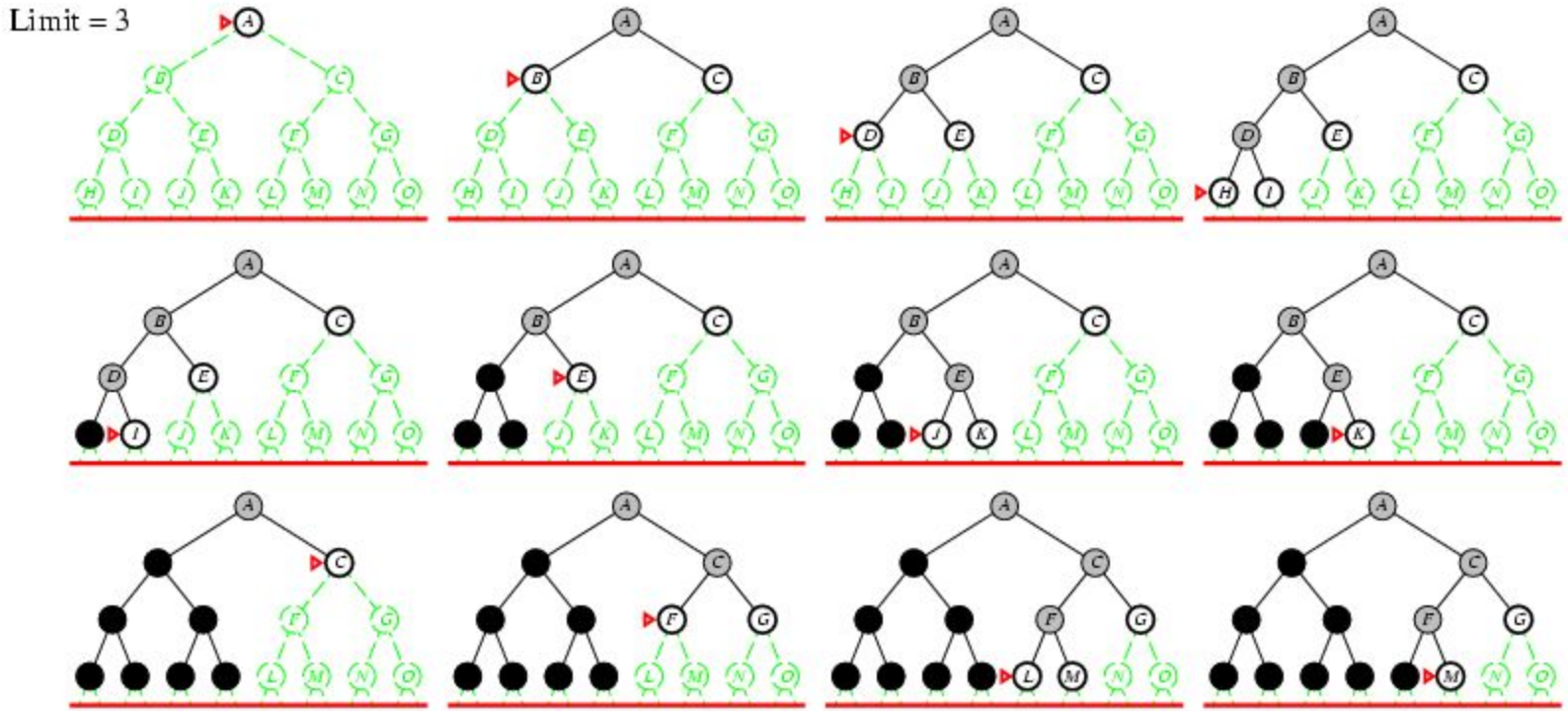
Iterative deepening search / =1



Iterative deepening search *l* =2

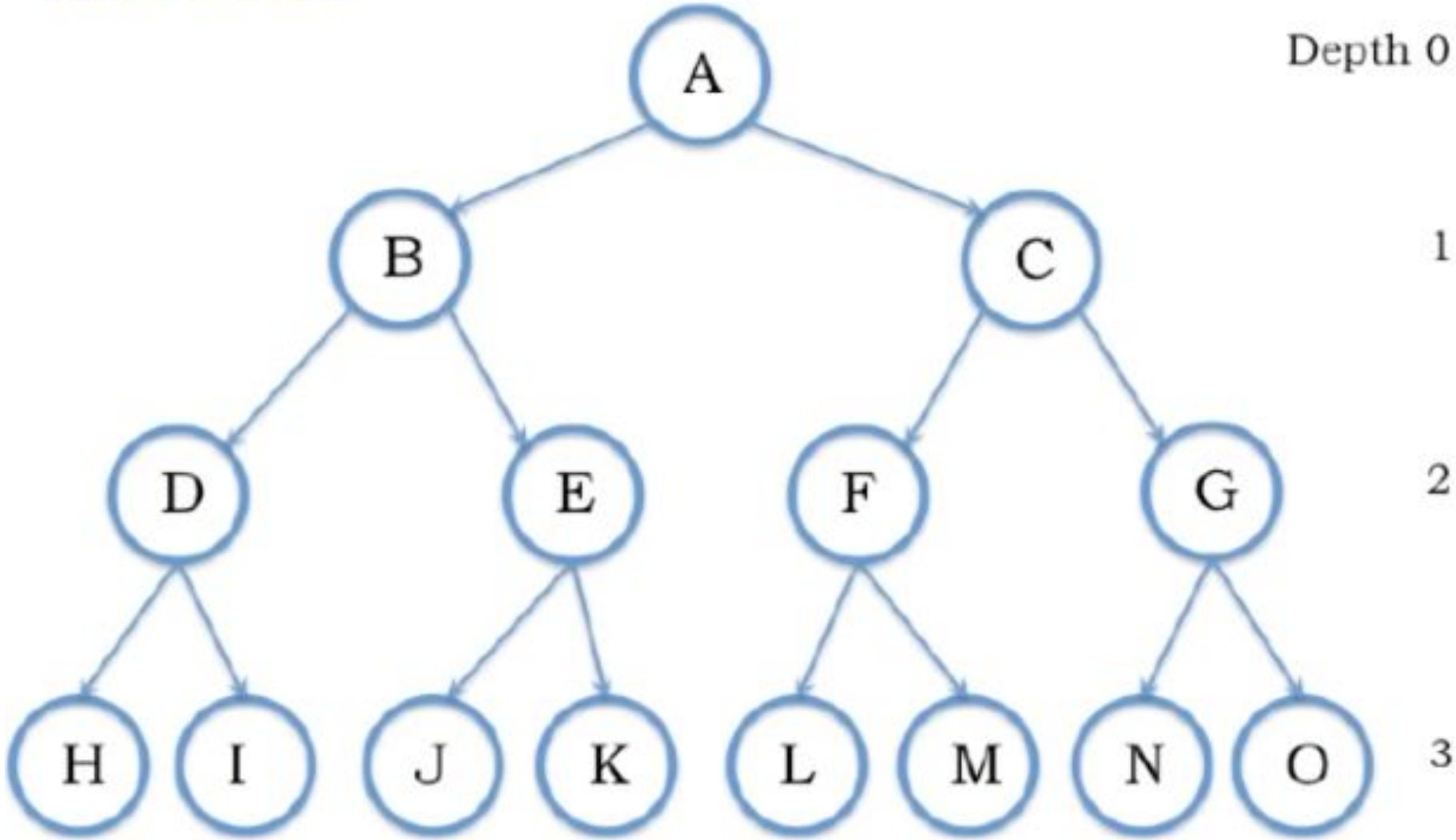


Iterative deepening search l =3

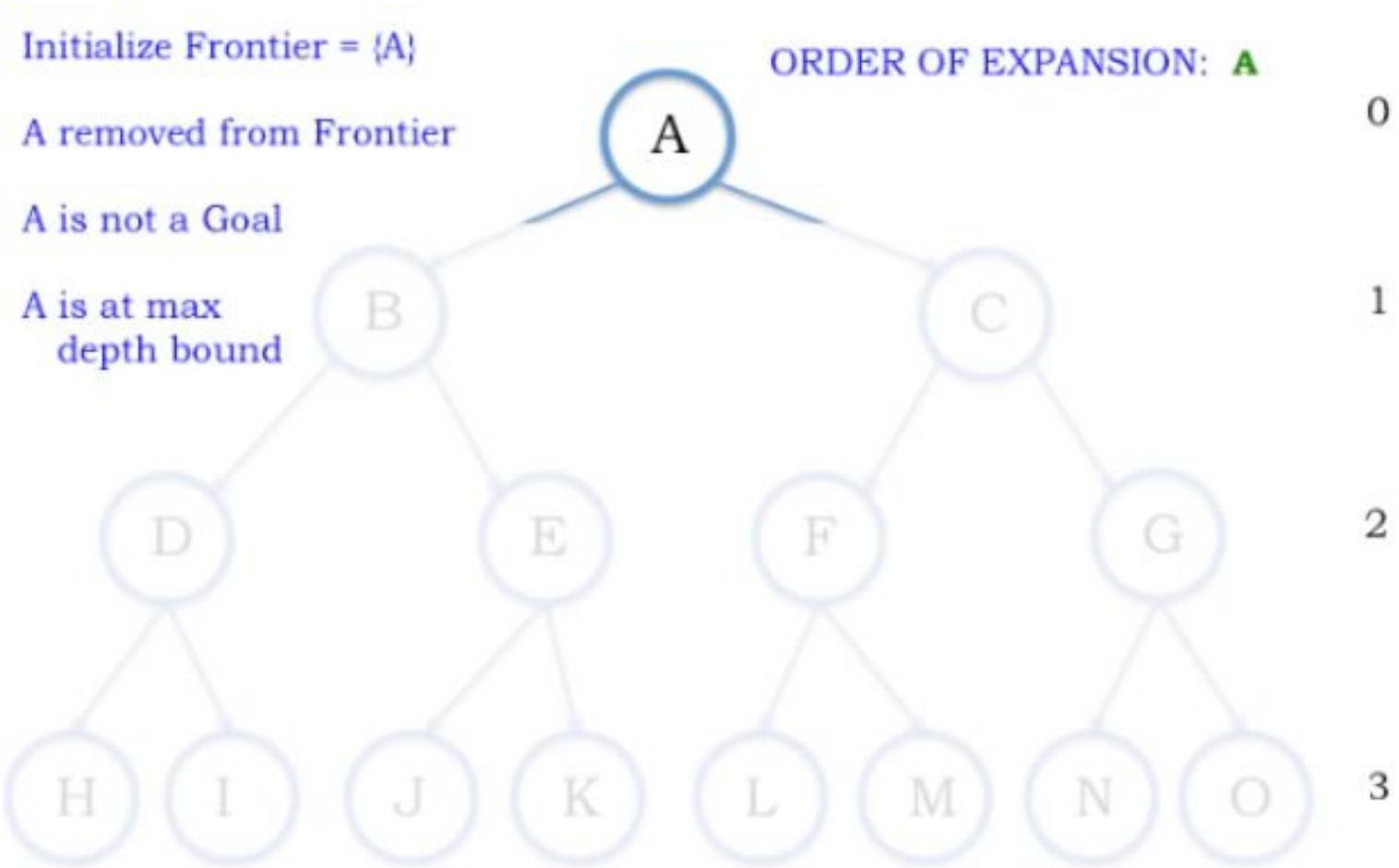


Operators (or actions) are not reversible
A is the start state

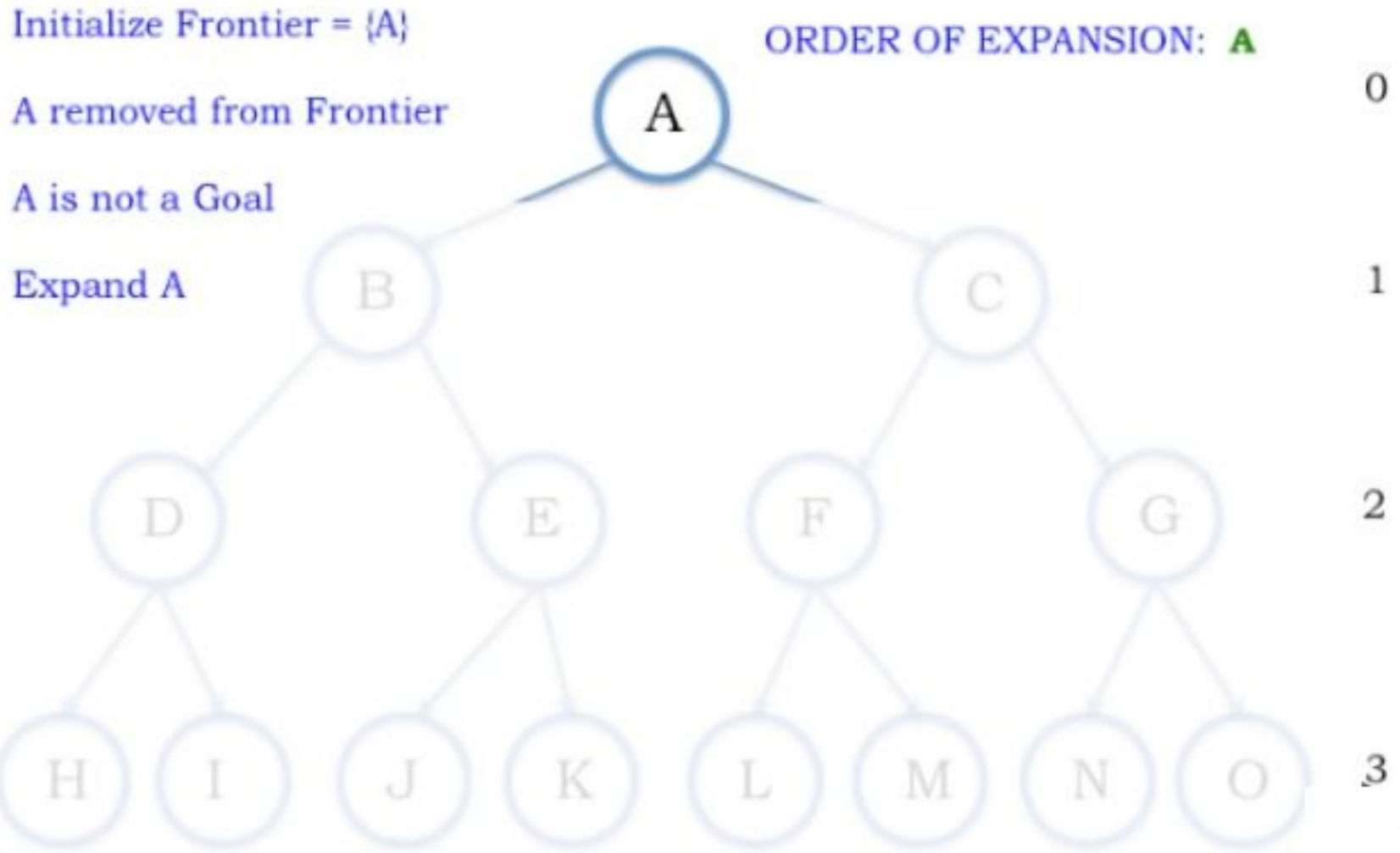
Repeated Depth Bounded searches



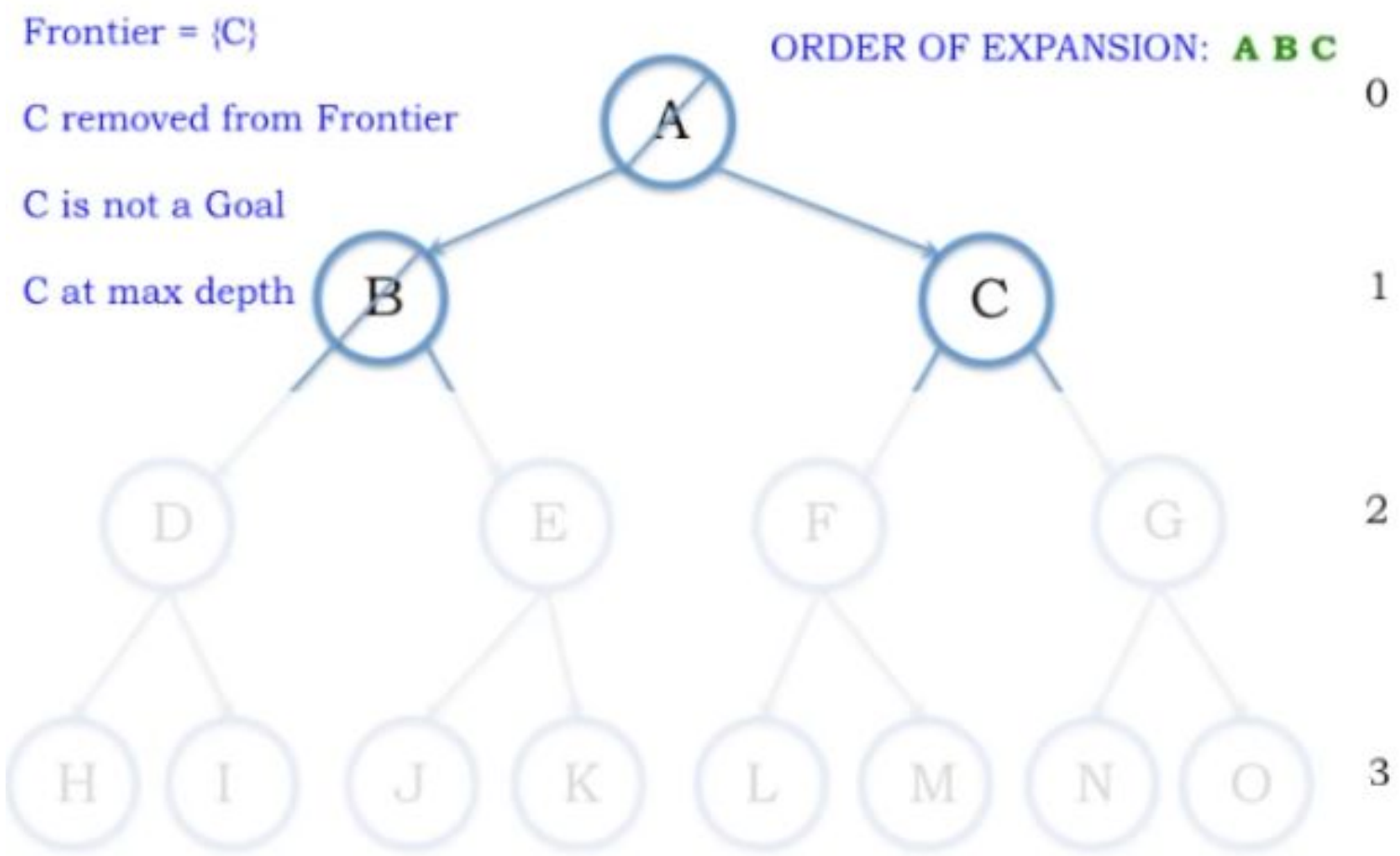
Depth Bound = 0



Depth Bound = 1



Depth Bound = 1



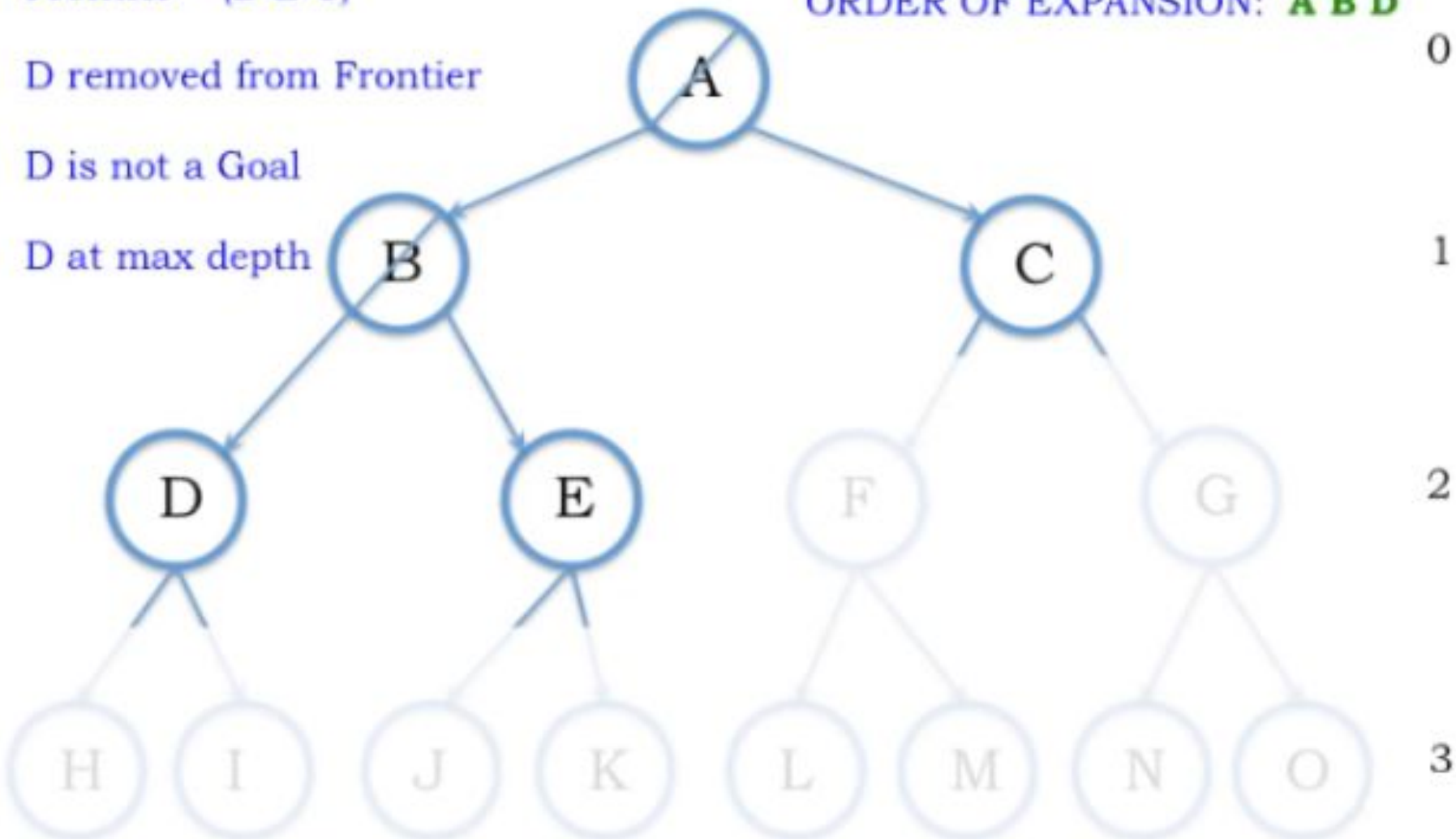
Frontier = {D E C}

D removed from Frontier

D is not a Goal

D at max depth

ORDER OF EXPANSION: **A B D**



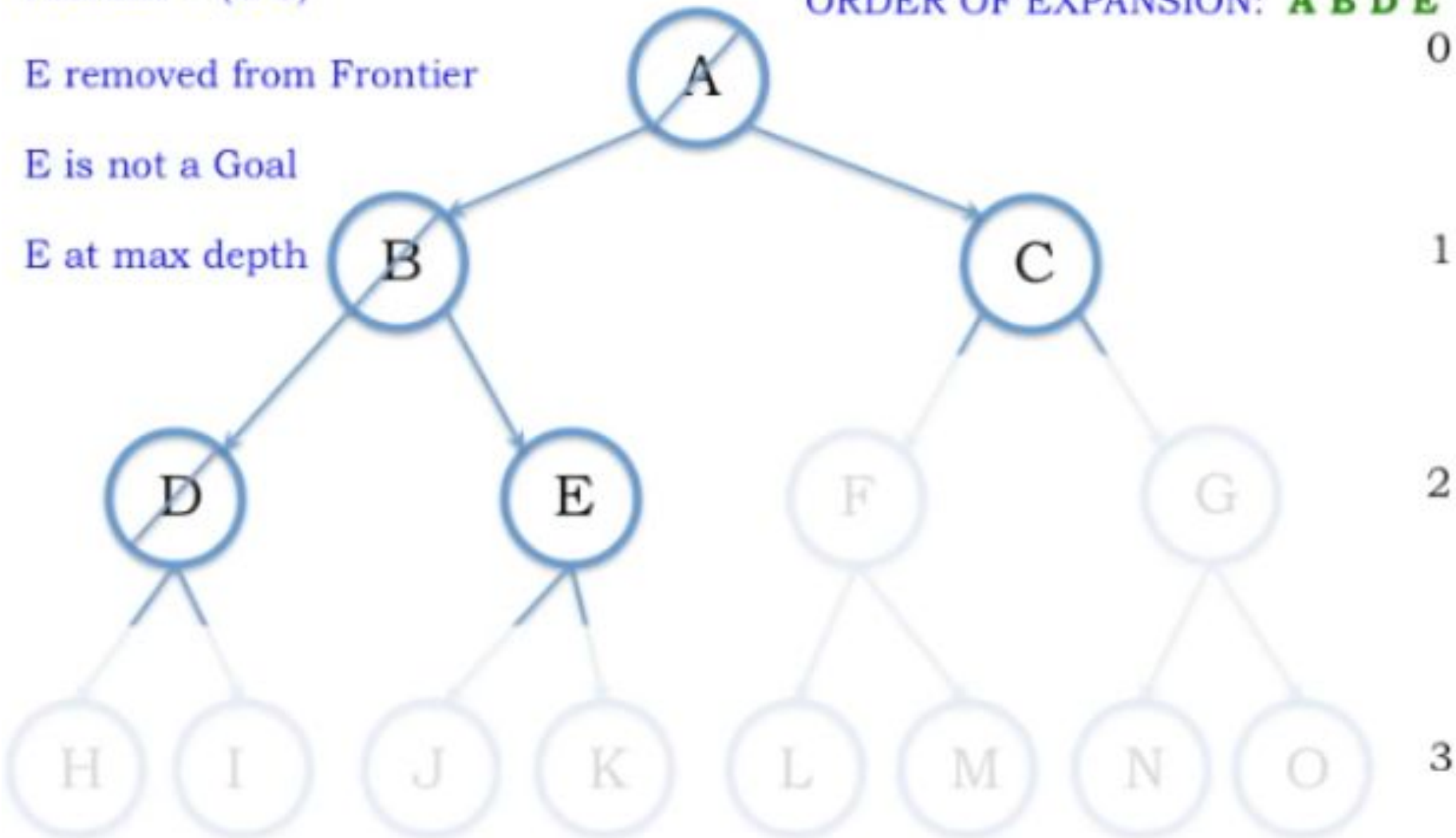
Frontier = {E C}

E removed from Frontier

E is not a Goal

E at max depth

ORDER OF EXPANSION: **A B D E**



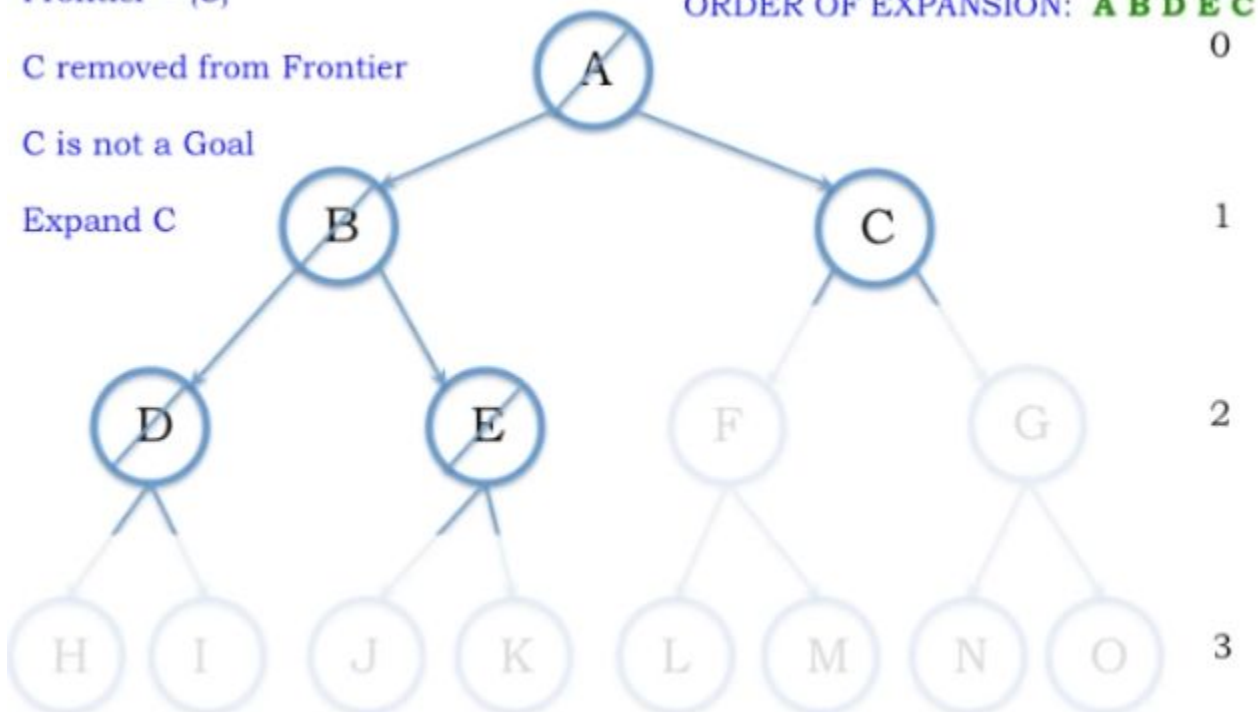
Frontier = {C}

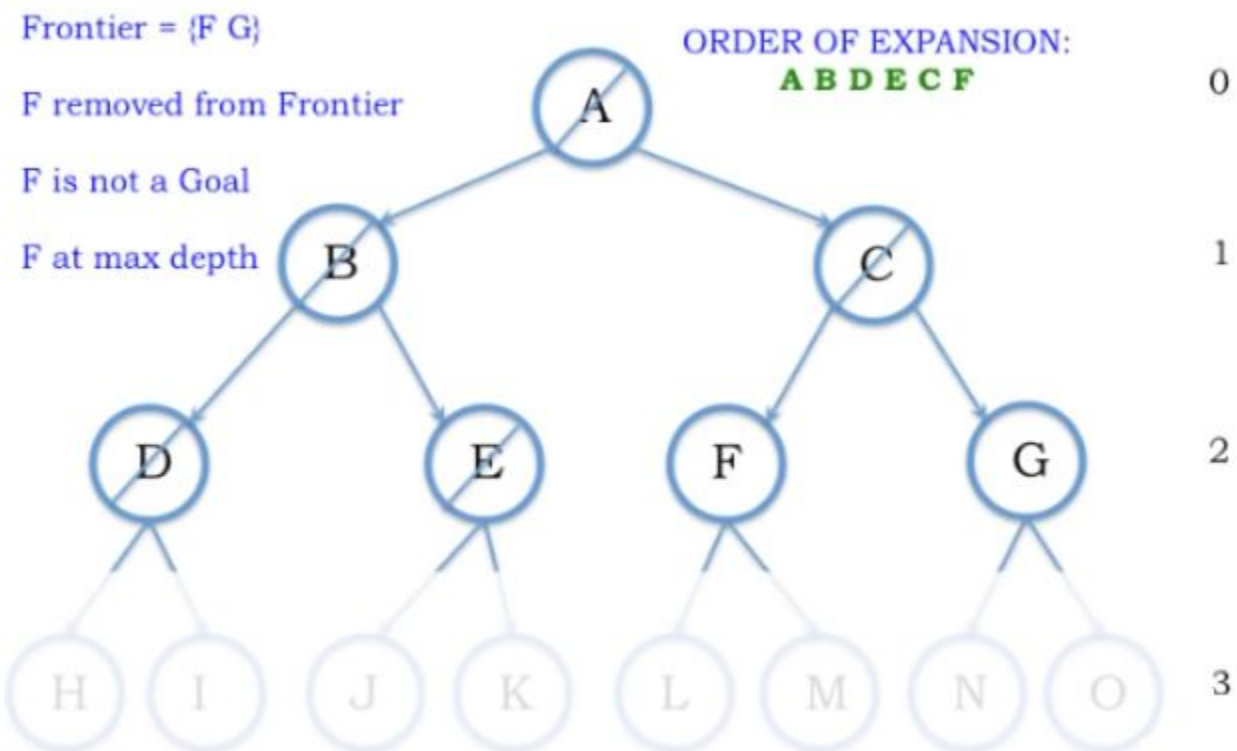
C removed from Frontier

C is not a Goal

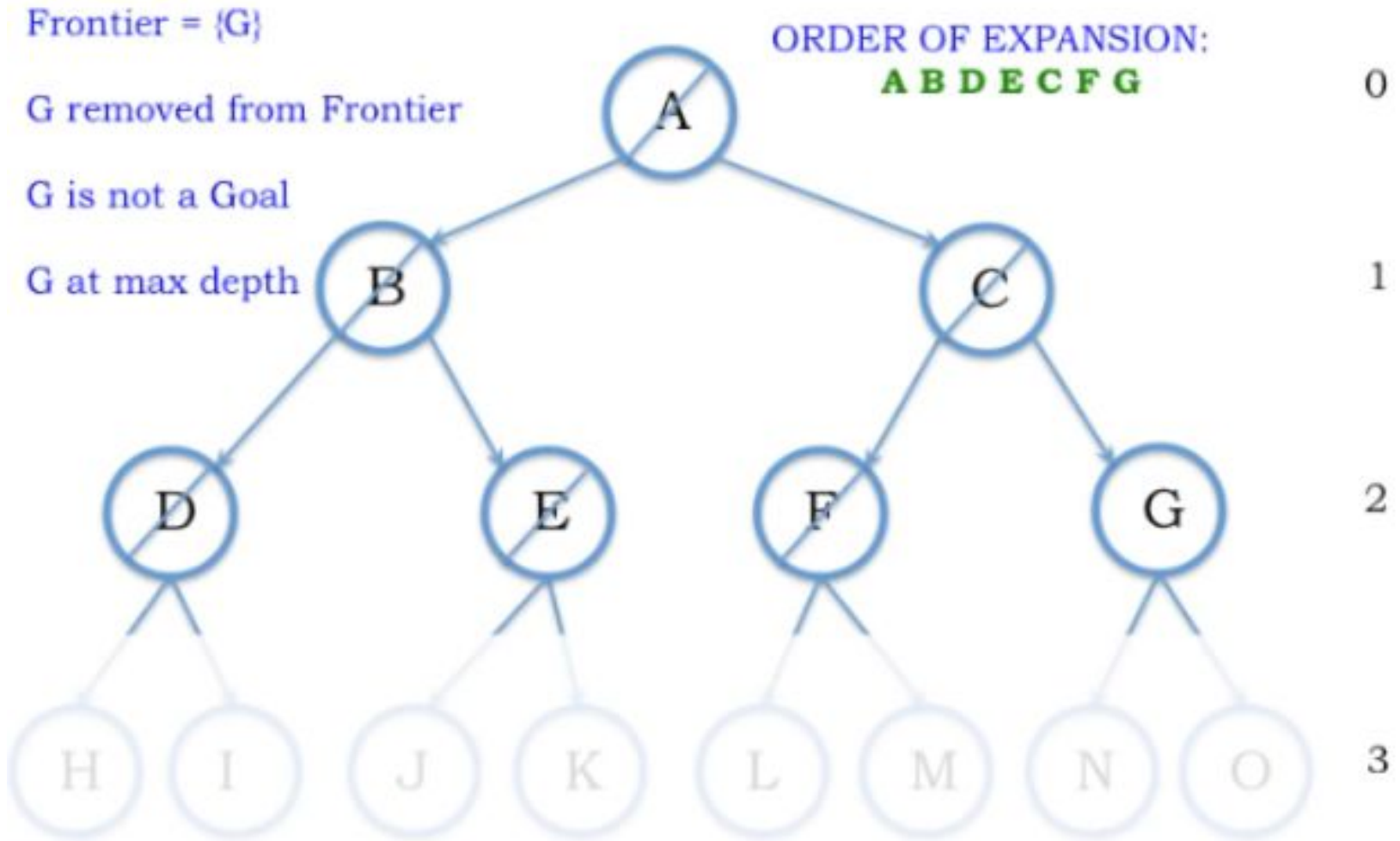
Expand C

ORDER OF EXPANSION: **A B D E C**



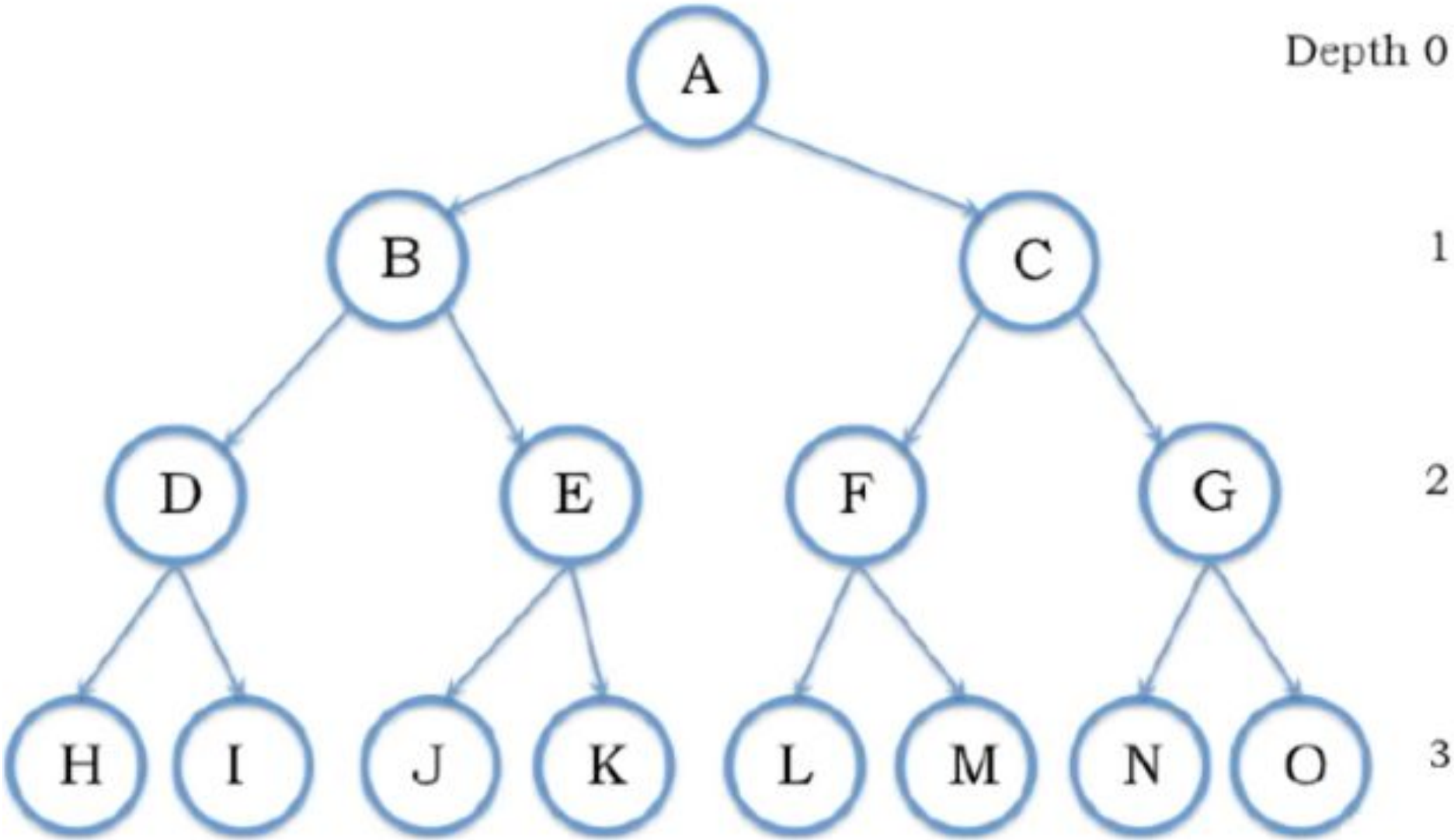

Frontier = {F G}
F removed from Frontier
F is not a Goal
F at max depth
ORDER OF EXPANSION:
A B D E C F
A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
0
1
2
3

Depth 2


Frontier = {G}
G removed from Frontier
G is not a Goal
G at max depth
ORDER OF EXPANSION:
A B D E C F G
A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
0
1
2
3

Over ALL the iterations, from depth bound 0 to 3, the order in which nodes removed from the frontier is:

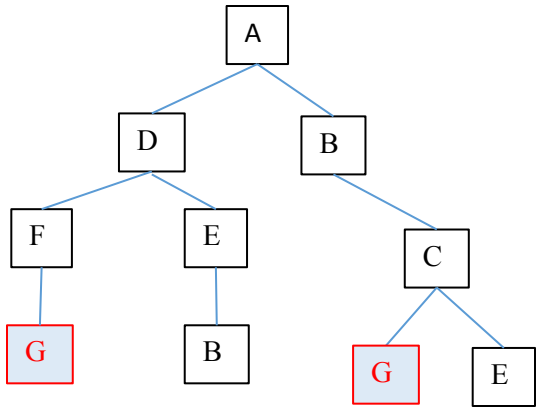
A A B C A B D E C F G A B D H I E J K C F L M G N O



Properties of Iterative Deepening Search

- Complete? Yes (in finite spaces)
- Time? $O(b^d)$
- Space? $O(bd)$
- Optimal? Yes, (if step cost = 1 i.e. identical step cost)

Example



Do it on notebook

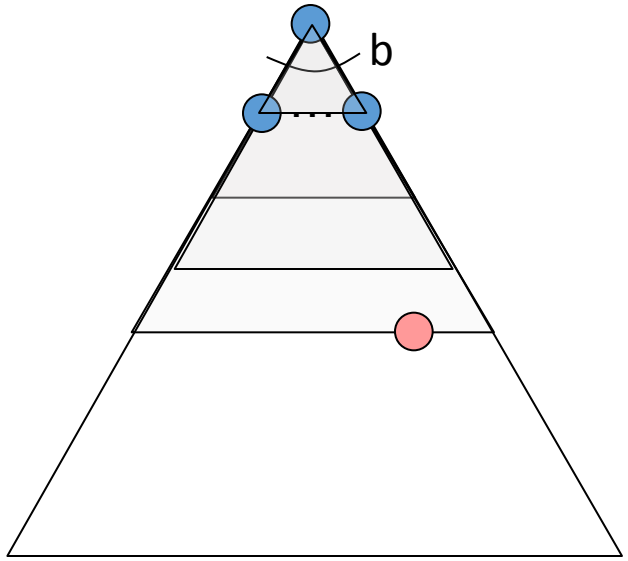
- Start Node: A
- Goal Node: G

Step **Frontier** **Expand[*] Explored: a set of nodes**

1	A	A	{-}
2	D B	B	{A}
3	D C	C	{A,B}
4	D G E	E	{A B C}

Iterative Deepening Search

- Idea: get DFS's space advantage with BFS's time / shallow-solution advantages
  - Run a DFS with depth limit 1. If no solution…
  - Run a DFS with depth limit 2. If no solution…
  - Run a DFS with depth limit 3. …..
- Isn't that wastefully redundant?
  - Generally most work happens in the lowest level searched, so not so bad!



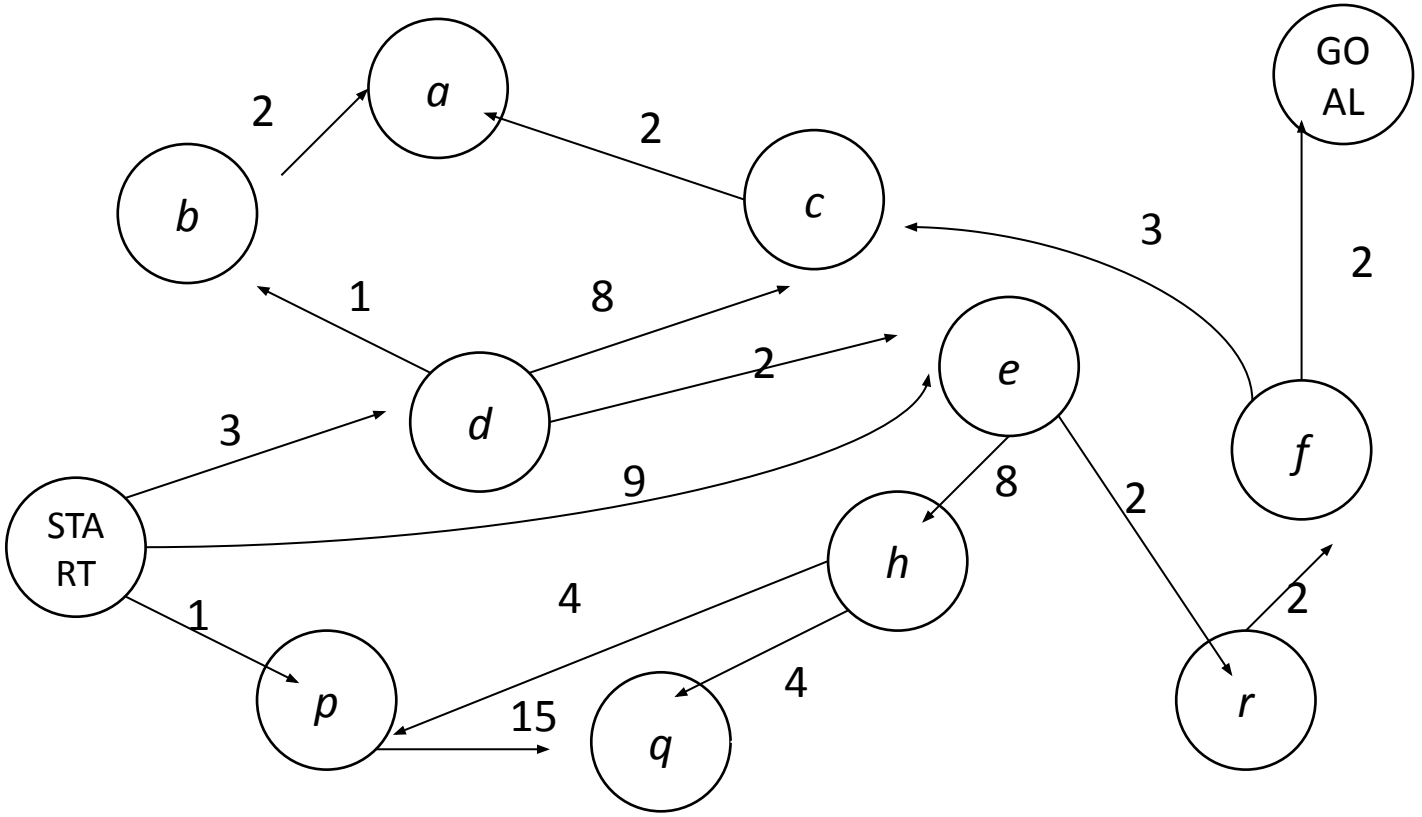
When to do goal test?

- Do Goal-Test when node is popped from queue
  - IF you care about finding the optimal path
  - AND your search space may have both short expensive and long cheap paths to a goal.
    - Guard against a short expensive goal.
    - E.g., Uniform Cost search with variable step costs.
- Otherwise, do Goal-Test when is node inserted.
  - E.g., Breadth-first Search, Depth-first Search, or Uniform Cost search when cost is a non-decreasing function of depth only (which is equivalent to Breadth-first Search).
- REASON ABOUT your search space & problem.
  - How could I possibly find a non-optimal goal?

When to do Goal-Test? Summary

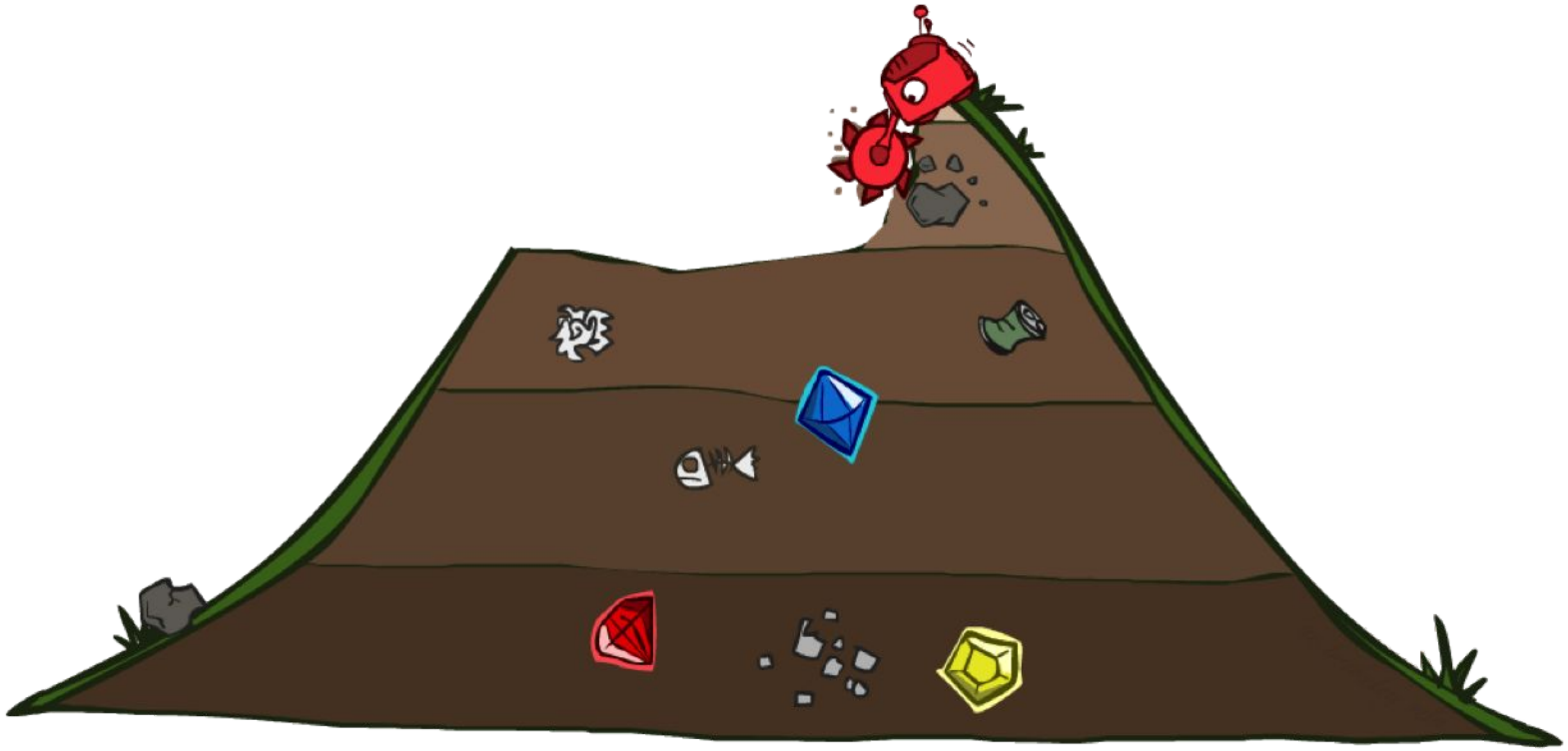
- For DFS, BFS, DLS, and IDS, the goal test is done when the child node is generated.
  - These are not optimal searches in the general case.
  - BFS and IDS are optimal if cost is a function of depth only; then, optimal goals are also shallowest goals and so will be found first
- For GBFS the behavior is the same whether the goal test is done when the node is generated or when it is removed
  - h(goal)=0 so any goal will be at the front of the queue anyway.
- For UCS and A* the goal test is done when the node is removed from the queue.
- This precaution avoids finding a short expensive path before a long cheap path.

Cost-Sensitive Search



BFS finds the shortest path in terms of number of actions.
It does not find the least-cost path. We will now cover
a similar algorithm which does find the least-cost path.

Uniform Cost Search



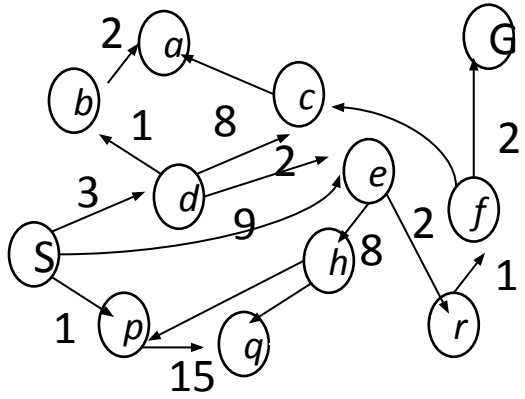
Uniform Cost Search

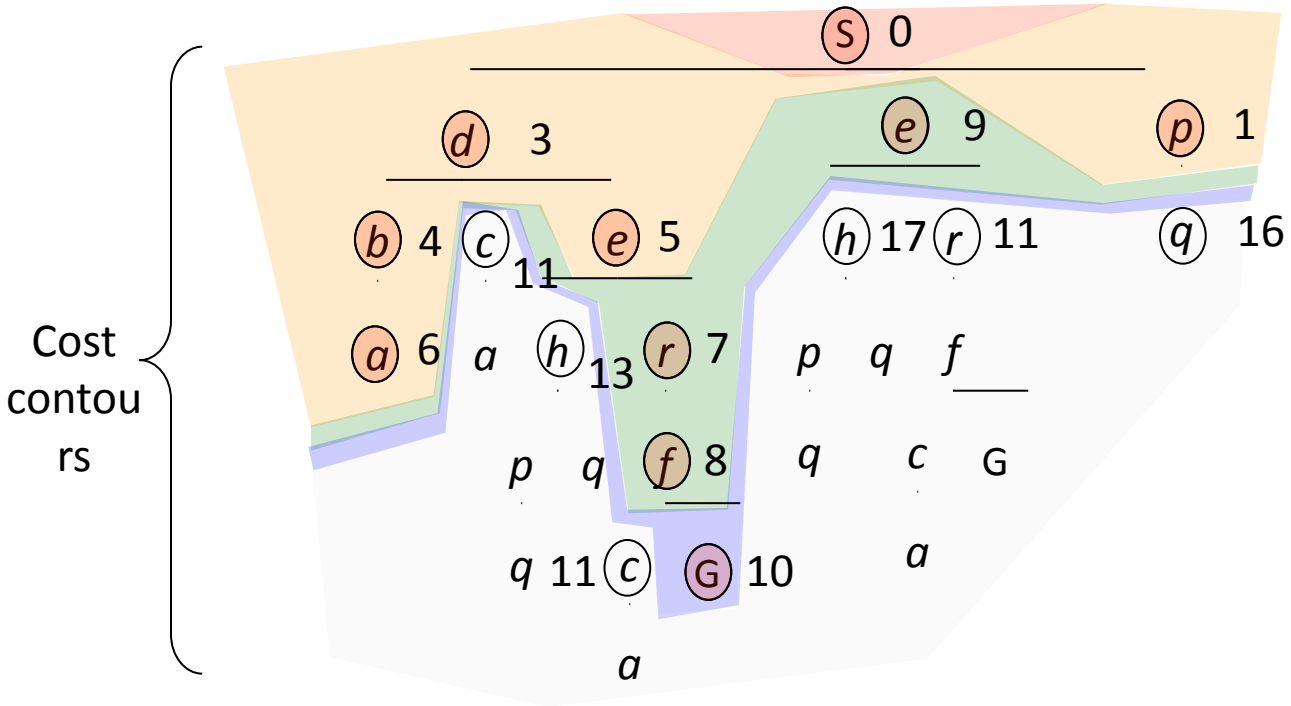
- Breadth-First Search find the shallowest goal state, but this may not always be the least-cost solution.
- Uniform-Cost Search modifies the Breadth-First Search strategy by always expanding the lowest path cost g(n) node on the fringe.
- *Frontier* is a priority queue, i.e., new successors are merged into the queue sorted by g(n).
  - Can remove successors already on queue w/higher g(n).
  - Saves memory, costs time; another space-time trade-off.

Uniform Cost Search

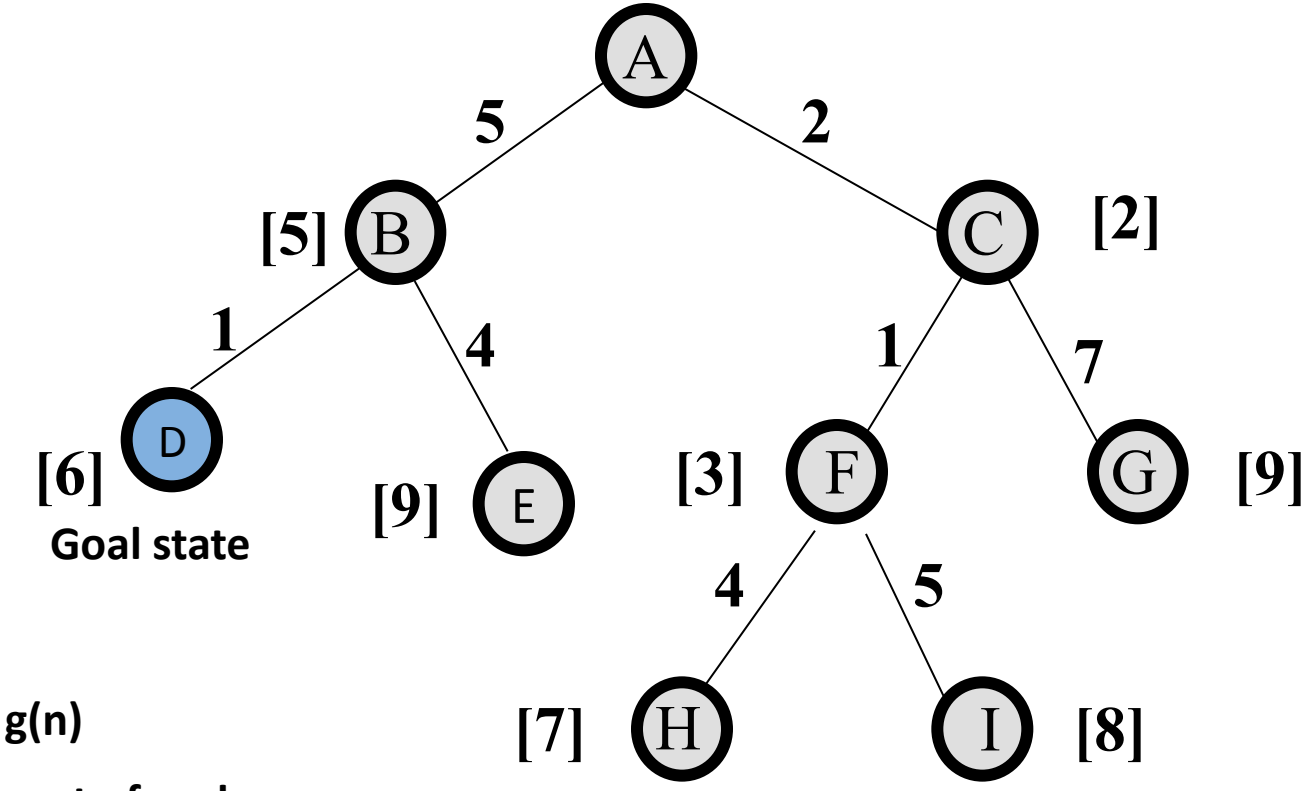
Strategy: expand a cheapest node first:

Fringe is a priority queue (priority: cumulative cost)





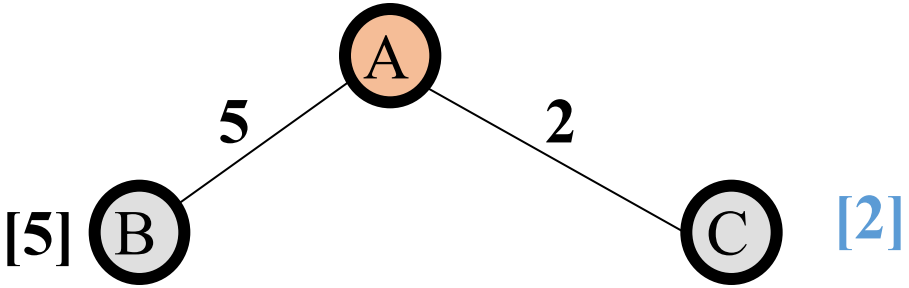
Uniform Cost Search (UCS)



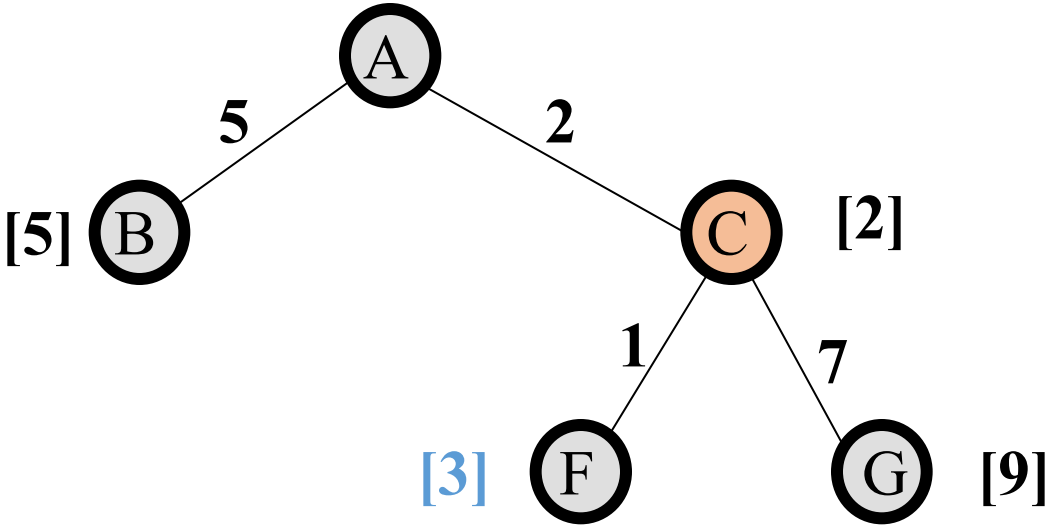
[x] = g(n)

path cost of node n

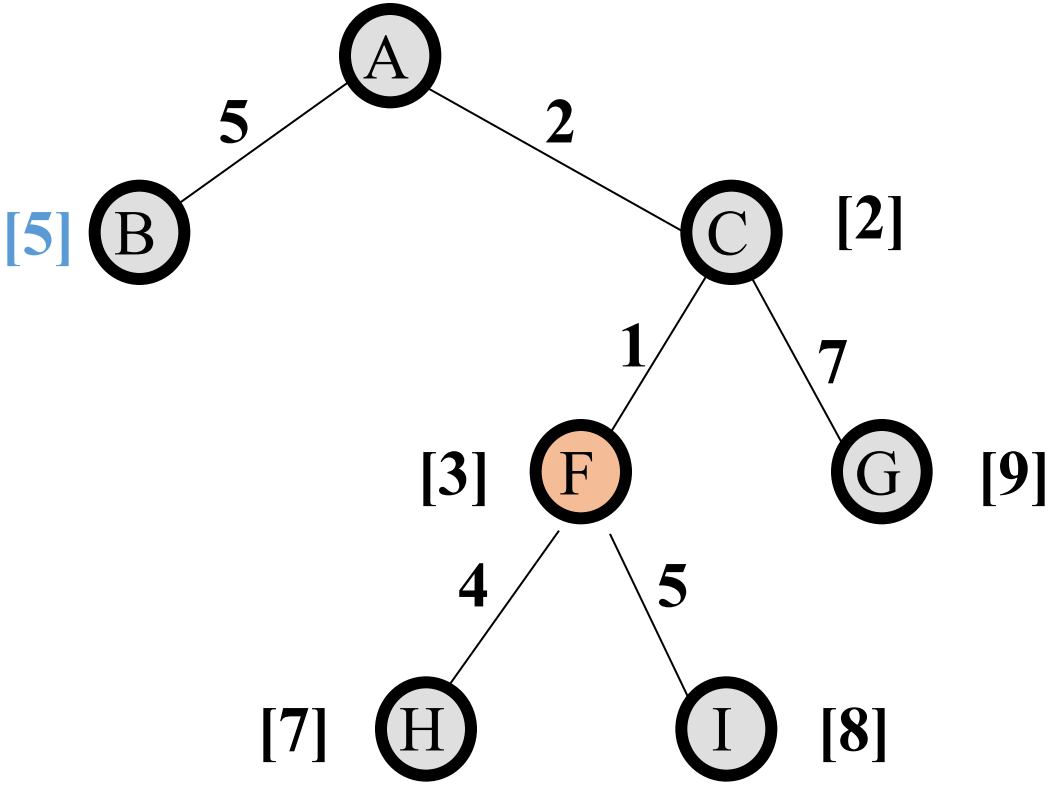
Uniform Cost Search (UCS)



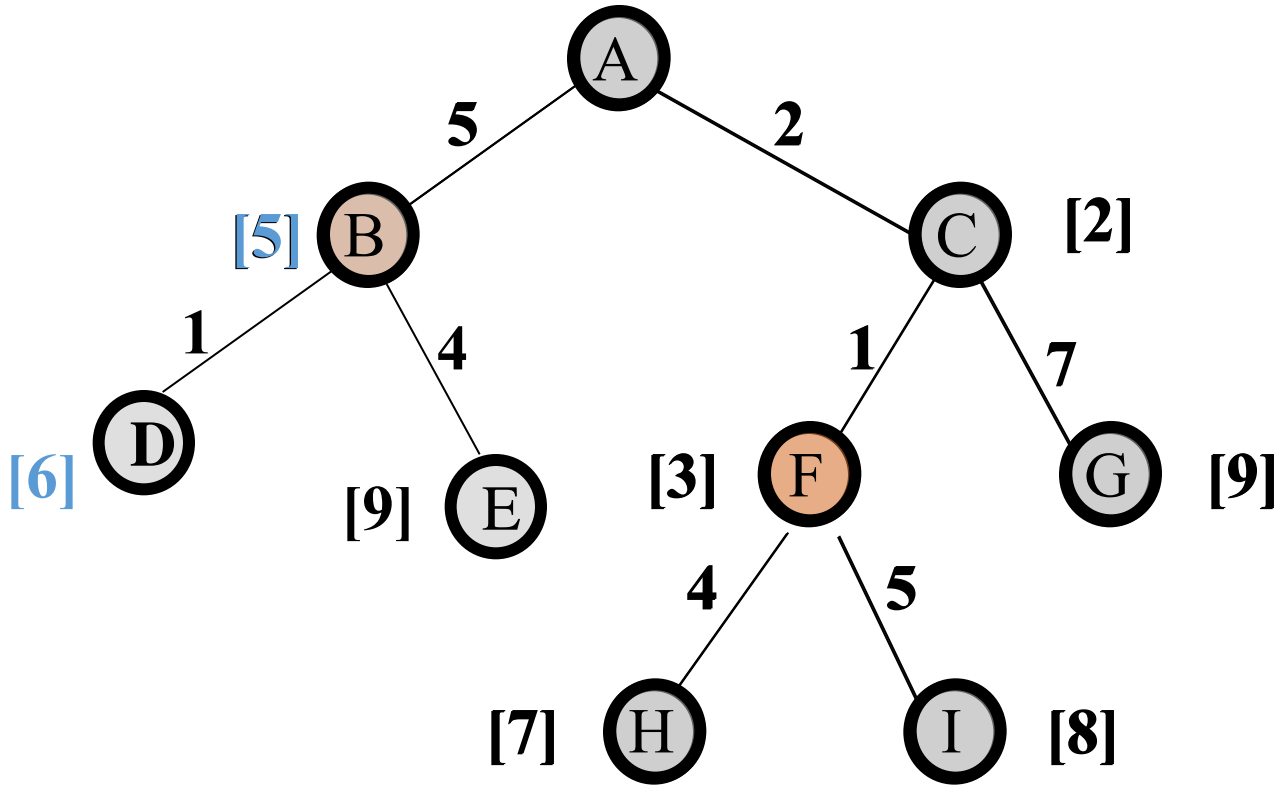
Uniform Cost Search (UCS)



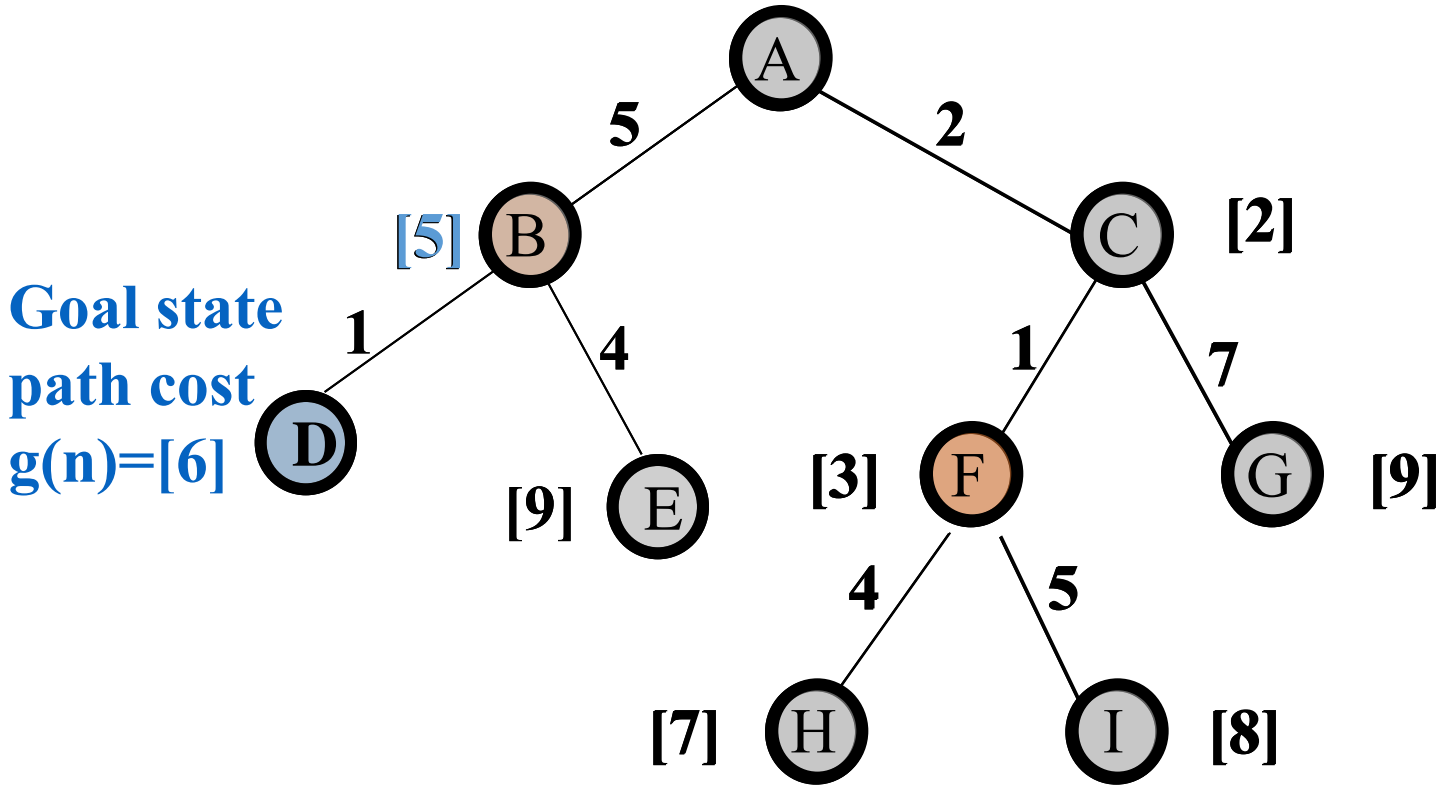
Uniform Cost Search (UCS)



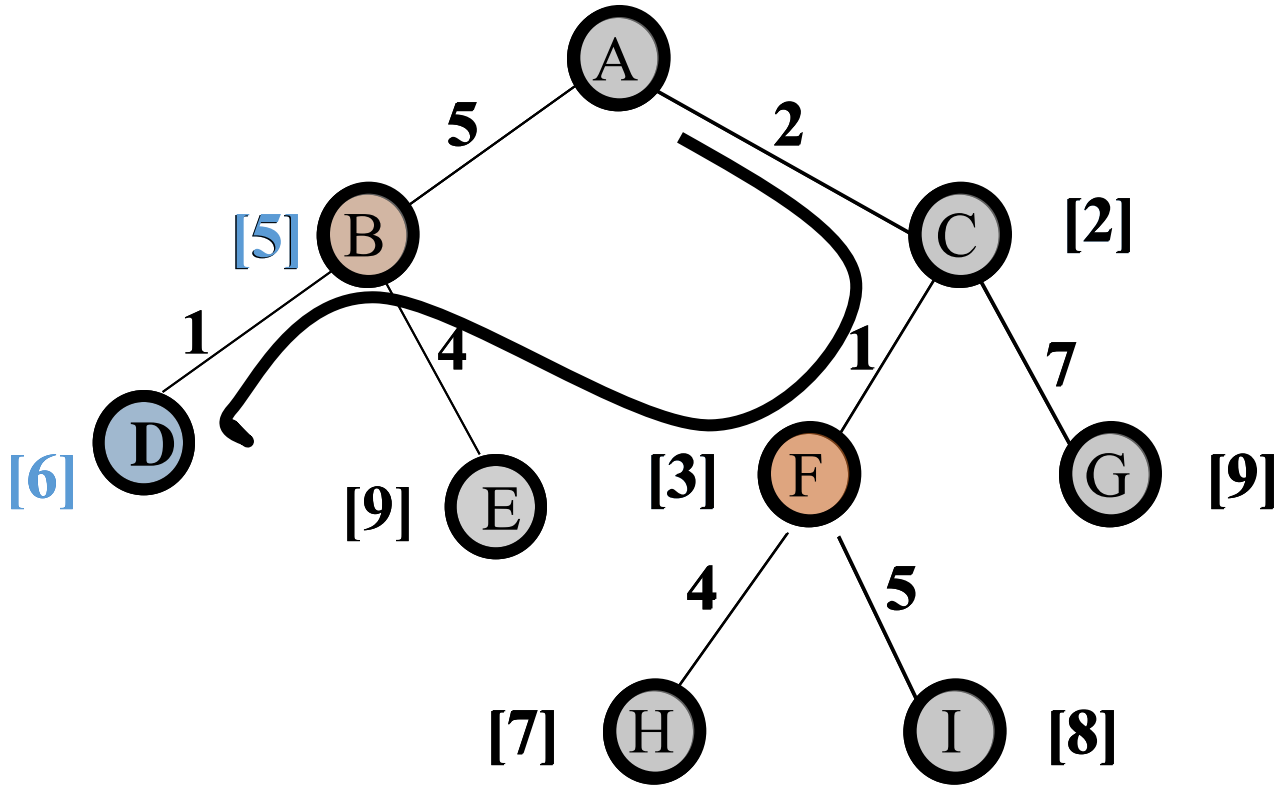
Uniform Cost Search (UCS)

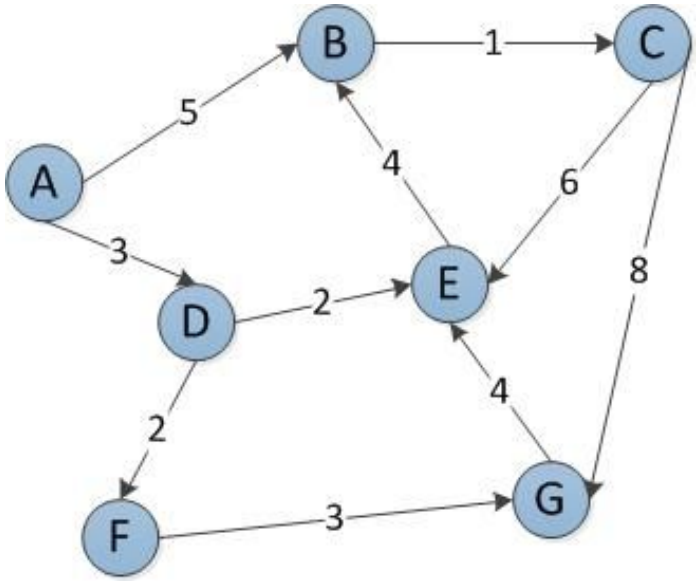


Uniform Cost Search (UCS)



Uniform Cost Search (UCS)





- Start Node: A
- Goal Node: G

Step	Frontier	Expand[*]	Explored: a set of nodes
1	{(A,0)}	A	∅
2	{(A-D,3),(A-B,5)}	D	{A}
3	{(A-B,5),(A-D-E,5),(A-D-F,5)}	B	{A,D}
4	{(A-D-E,5),(A-D-F,5),(A-B-C,6)}	E	{A,D,B}
5	{(A-D-F,5),(A-B-C,6)}[*]	F	{A,D,B,E}
6	{(A-B-C,6),(A-D-F-G,8)}	C	{A,D,B,E,F}
7	(A-D-F-G,8)}	G	{A,D,B,E,F,C}
8	∅		

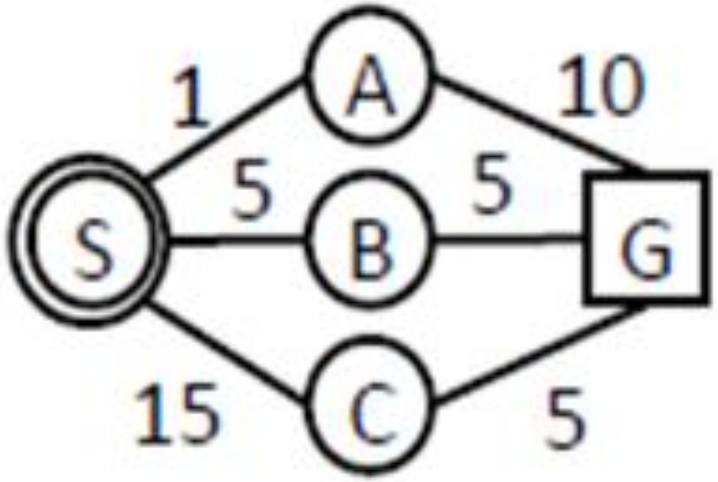
- Found the path: A -> D -> F -> G.
- *B is not added to the frontier because it is found in the explored set.

Uniform Cost Search

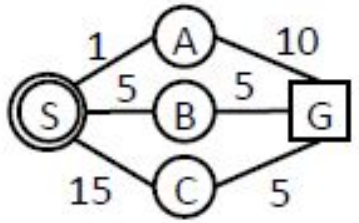
```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  frontier ← a priority queue ordered by PATH-COST, with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)  /* chooses the lowest-cost node in frontier */
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        frontier ← INSERT(child, frontier)
      else if child.STATE is in frontier with higher PATH-COST then
        replace that frontier node with child
```

Figure 3.14 Uniform-cost search on a graph. The algorithm is identical to the general graph search algorithm in Figure 3.7, except for the use of a priority queue and the addition of an extra check in case a shorter path to a frontier state is discovered. The data structure for *frontier* needs to support efficient membership testing, so it should combine the capabilities of a priority queue and a hash table.

Uniform Cost Search



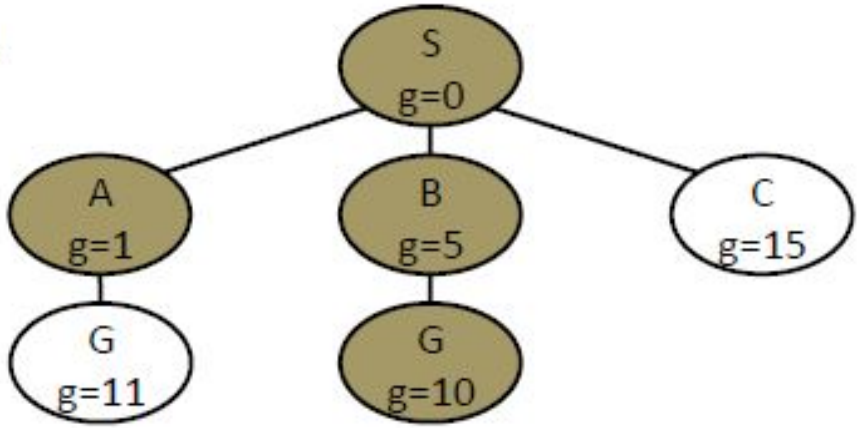
Uniform Cost Search



Route finding problem.
Steps labeled w/cost.

Order of node expansion: S A B G

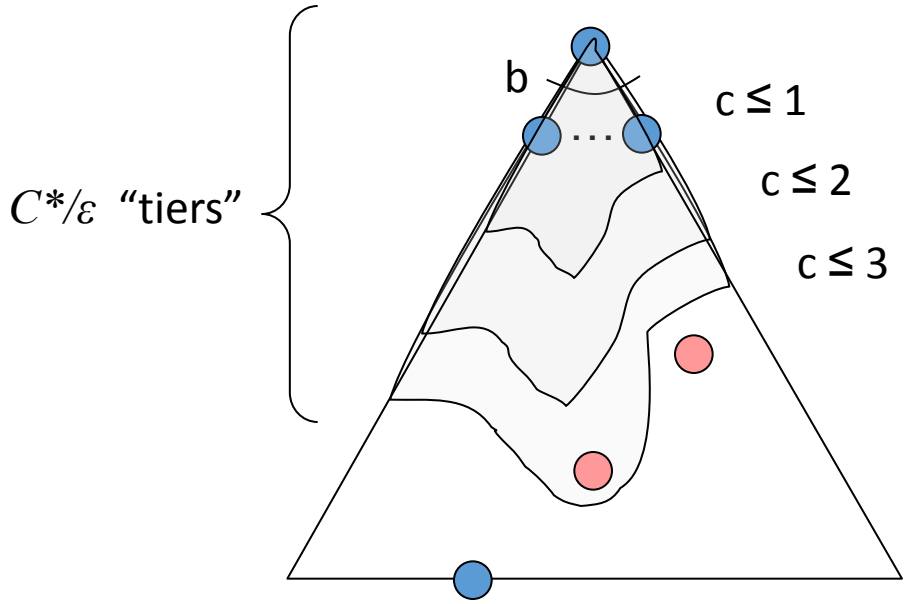
Path found: S B G Cost of path found: 10



Technically, the goal node is not really expanded, because we do not generate the children of a goal node. It is listed in "Order of node expansion" only for your convenience, to see explicitly where it was found.

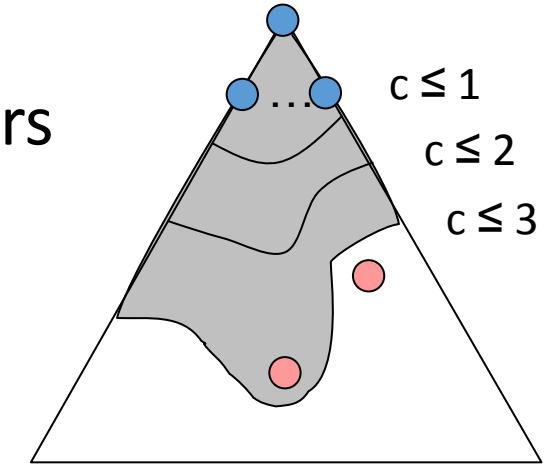
Uniform Cost Search (UCS) Properties

- What nodes does UCS expand?
  - Processes all nodes with cost less than cheapest solution!
  - If that solution costs C^* and arcs cost at least ε, then the "effective depth" is roughly C^*/ε
  - Takes time $O(b^{C^*/\varepsilon})$ (exponential in effective depth)
- How much space does the fringe take?
  - Has roughly the last tier, so $O(b^{C^*/\varepsilon})$
- Is it complete?
  - Assuming best solution has a finite cost and minimum arc cost is positive, yes!
- Is it optimal?
  - Yes! (Proof next lecture via A*)



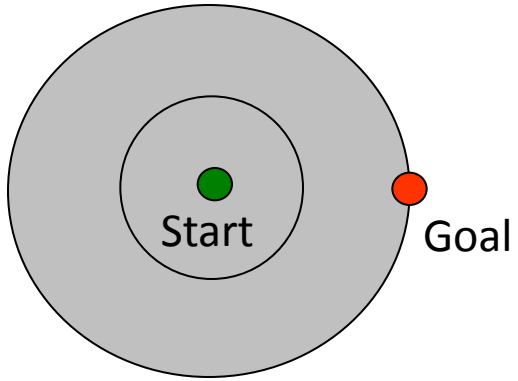
Uniform Cost Issues

- Remember: UCS explores increasing cost contours



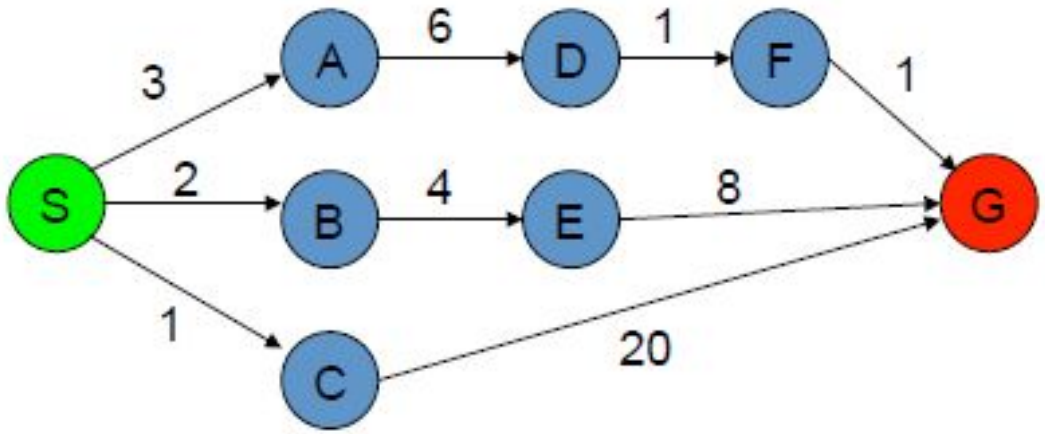
- The good: UCS is complete and optimal!

- The bad:
  - Explores options in every “direction”
  - No information about goal location

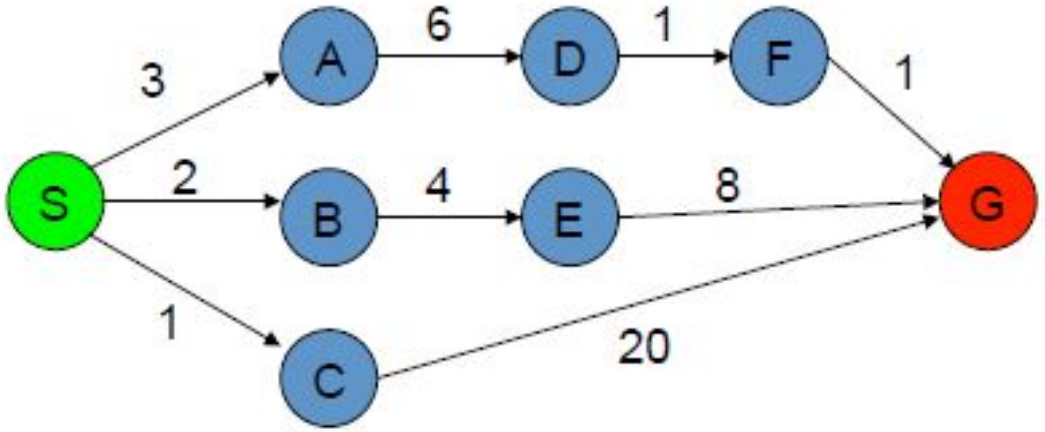


- We’ll fix that soon!

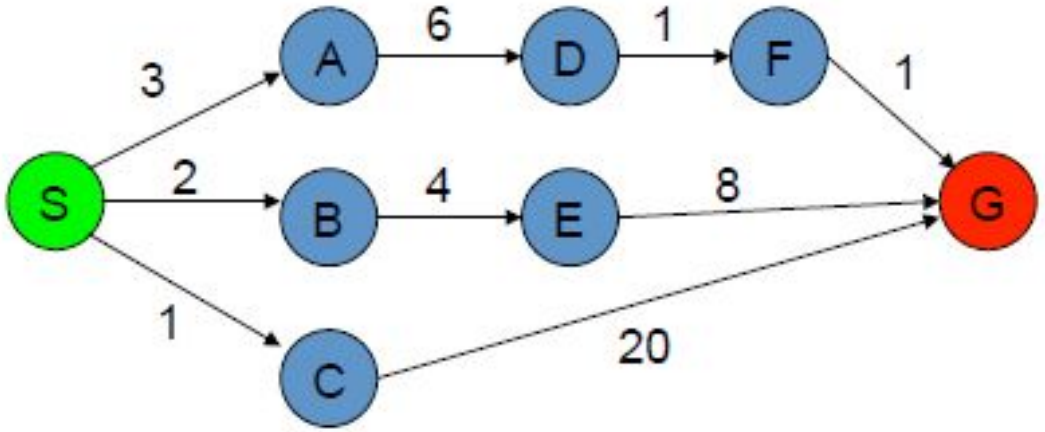
Uniform Cost Search



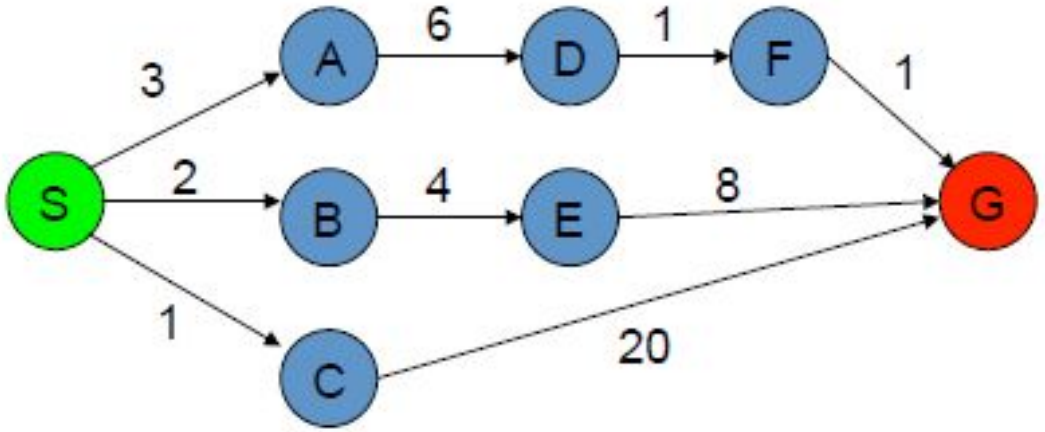
- The graph above shows the step-costs for different paths going from the start (S) to the goal (G).
- Use uniform cost search to find the optimal path to the goal.



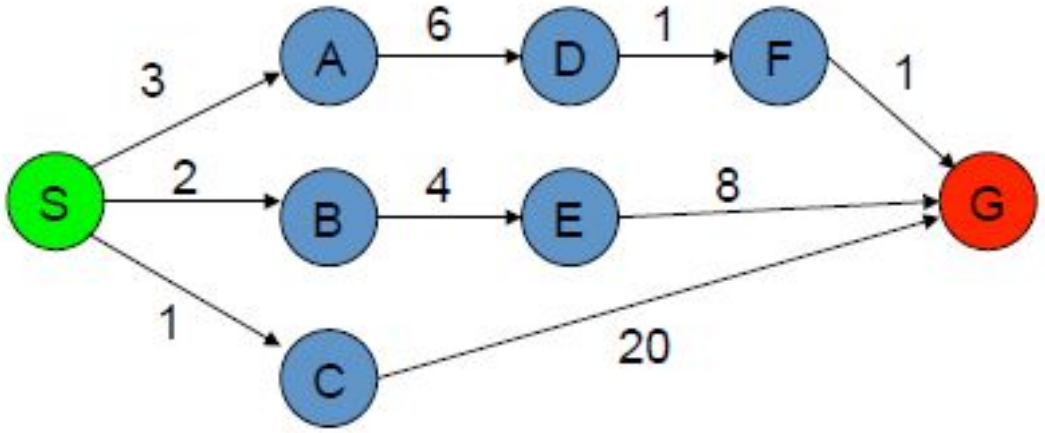
	Frontier	Expand	Explored
1	S	S	Empty



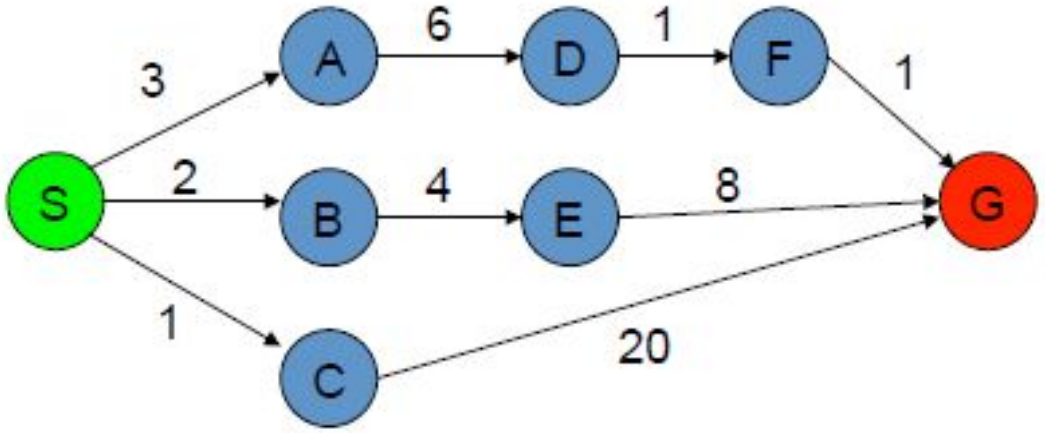
	Frontier	Expand	Explored
1	S	S	Empty
2	(S-C,1) (S-B,2)(S-A,3)	C	S



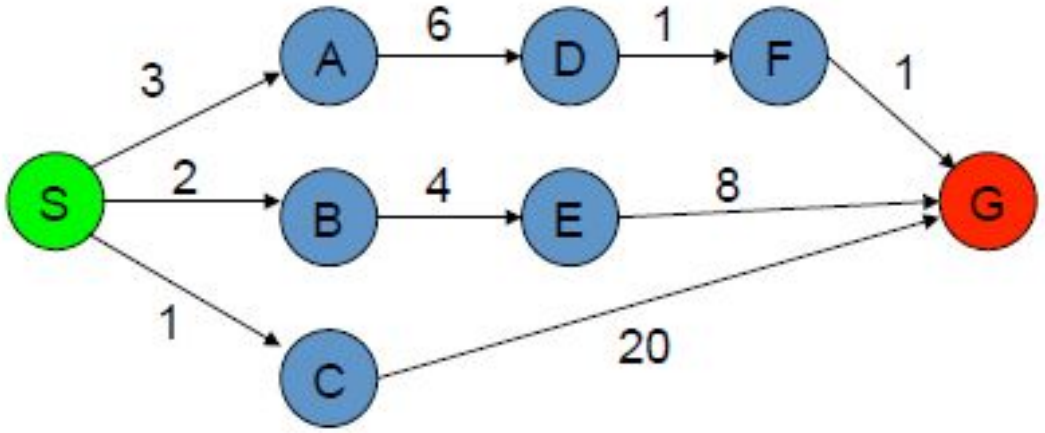
	Frontier	Expand	Explored
1	S	S	Empty
2	(S-C,1) (S-B,2)(S-A,3)	C	S
3	(S-B,2)(S-A,3)(S-C-G,21)	B	S,C



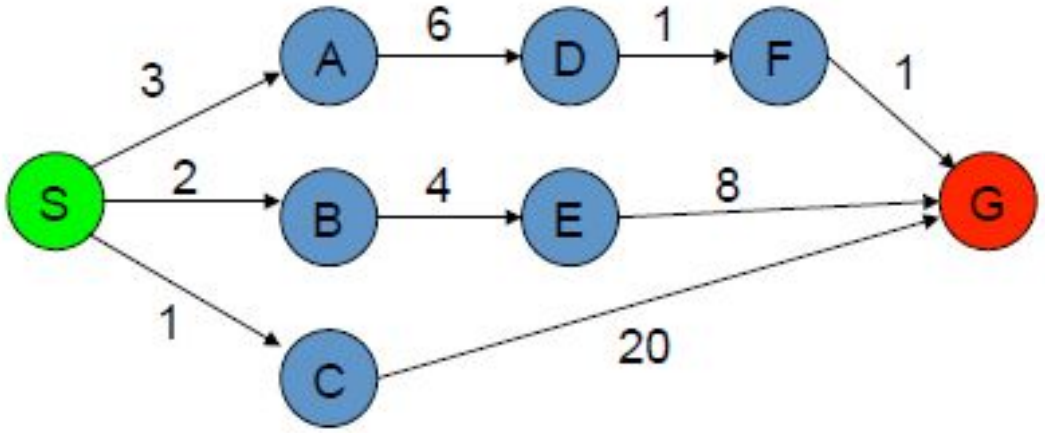
	Frontier	Expand	Explored
1	S	S	Empty
2	(S-C,1) (S-B,2)(S-A,3)	C	S
3	(S-B,2)(S-A,3)(S-C-G,21)	B	S,C
4	(S-A,3)(S-C-G,21)(S-B-E,6)	A	S,C,B



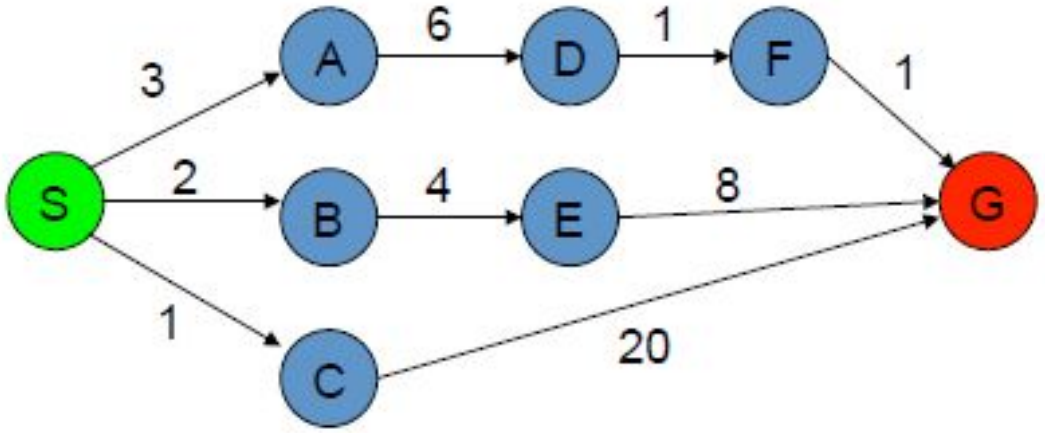
	Frontier	Expand	Explored
1	S	S	Empty
2	(S-C,1) (S-B,2)(S-A,3)	C	S
3	(S-B,2)(S-A,3)(S-C-G,21)	B	S,C
4	(S-A,3)(S-C-G,21)(S-B-E,6)	A	S,C,B
5	(S-C-G,21)(S-B-E,6)(S-A-D,9)	E	S,C,B,A



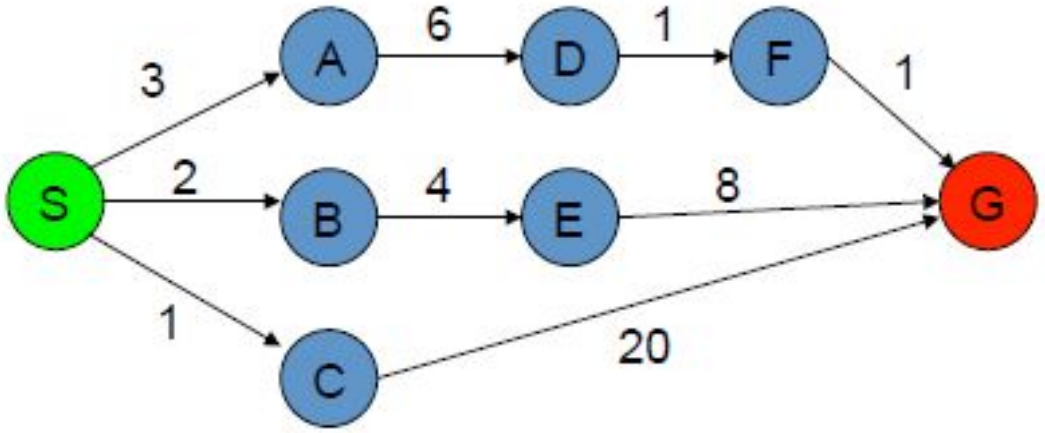
	Frontier	Expand	Explored
1	S	S	Empty
2	(S-C,1) (S-B,2)(S-A,3)	C	S
3	(S-B,2)(S-A,3)(S-C-G,21)	B	S,C
4	(S-A,3)(S-C-G,21)(S-B-E,6)	A	S,C,B
5	(S-C-G,21)(S-B-E,6)(S-A-D,9)	E	S,C,B,A
6	(S-C-G,21) (S-A-D,9) (S-B-E-G,14)	D	S,C,B,A,E



	Frontier	Expand	Explored
1	S	S	Empty
2	(S-C,1) (S-B,2)(S-A,3)	C	S
3	(S-B,2)(S-A,3)(S-C-G,21)	B	S,C
4	(S-A,3)(S-C-G,21)(S-B-E,6)	A	S,C,B
5	(S-C-G,21)(S-B-E,6)(S-A-D,9)	E	S,C,B,A
6	(S-C-G,21) (S-A-D,9) (S-B-E-G,14)	D	S,C,B,A,E
7	(S-C-G,21) (S-B-E-G,14) (S-A-D-F,10)	F	S,C,B,A,E,D

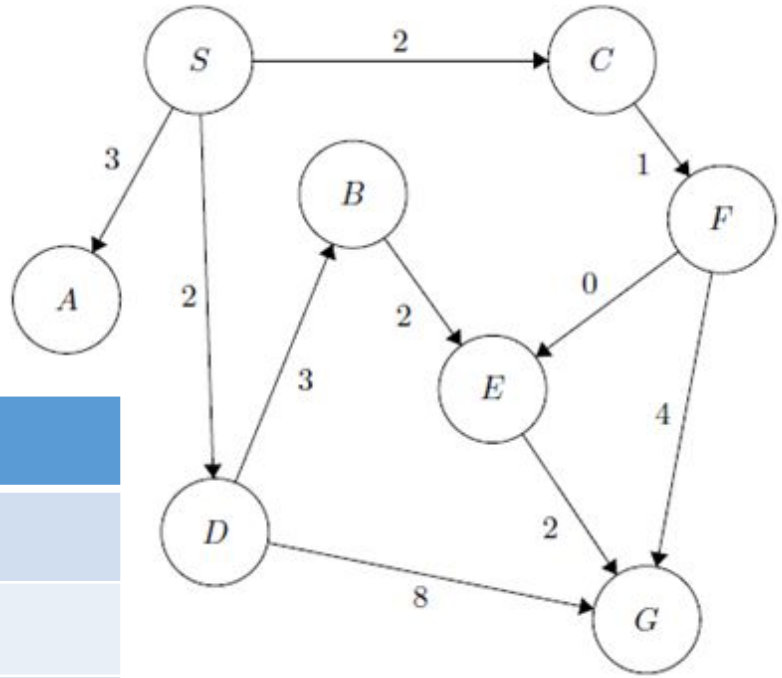


	Frontier	Expand	Explored
1	S	S	Empty
2	(S-C,1) (S-B,2)(S-A,3)	C	S
3	(S-B,2)(S-A,3)(S-C-G,21)	B	S,C
4	(S-A,3)(S-C-G,21)(S-B-E,6)	A	S,C,B
5	(S-C-G,21)(S-B-E,6)(S-A-D,9)	E	S,C,B,A
6	(S-C-G,21) (S-A-D,9) (S-B-E-G,14)	D	S,C,B,A,E
7	(S-C-G,21) (S-B-E-G,14) (S-A-D-F,10)	F	S,C,B,A,E,D
8	(S-C-G,21) (S-B-E-G,14) (S-A-D-F-G,11)		



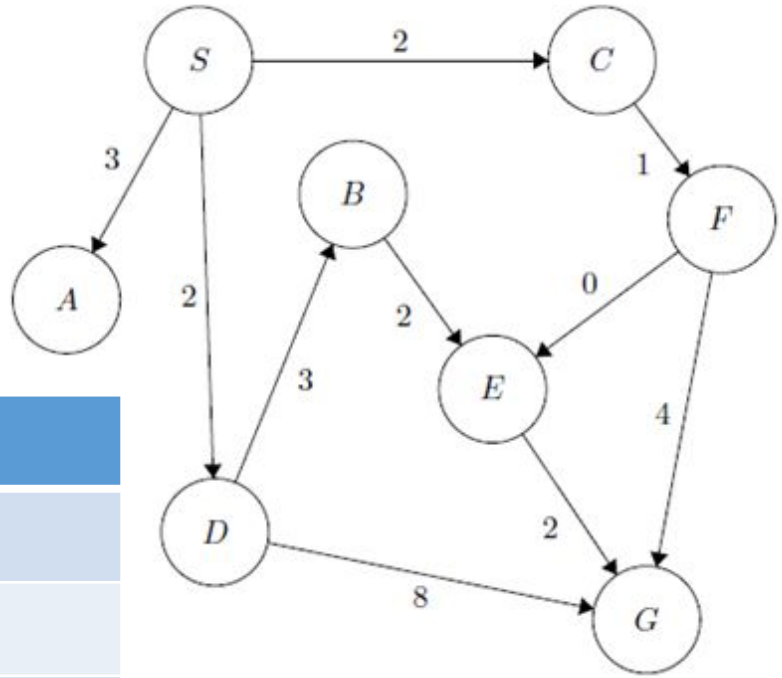
	Frontier	Expand	Explored
1	S	S	Empty
2	(S-C,1) (S-B,2)(S-A,3)	C	S
3	(S-B,2)(S-A,3)(S-C-G,21)	B	S,C
4	(S-A,3)(S-C-G,21)(S-B-E,6)	A	S,C,B
5	(S-C-G,21)(S-B-E,6)(S-A-D,9)	E	S,C,B,A
6	(S-C-G,21) (S-A-D,9) (S-B-E-G,14)	D	S,C,B,A,E
7	(S-C-G,21) (S-B-E-G,14) (S-A-D-F,10)	F	S,C,B,A,E,D
8	(S-C-G,21) (S-B-E-G,14) (S-A-D-F-G,11)		
9	S-A-D-F-G,11 Goal Found		S,C,B,A,E,D,F,G

Traverse from start state (S) to Goal state (G)



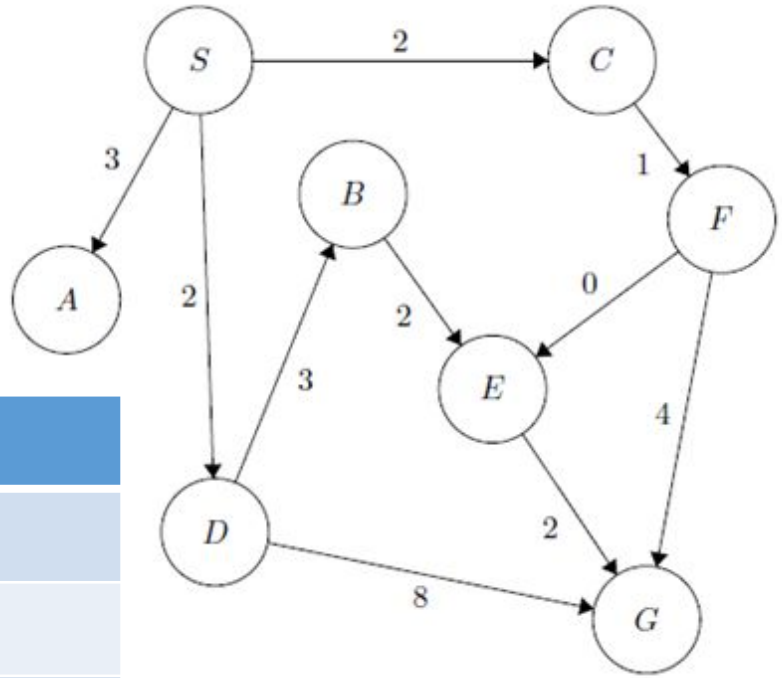
Step	Frontier	Expand	Explored: a set of Nodes
1	(S,0)	S	Ø

Traverse from start state (S) to Goal state (G)



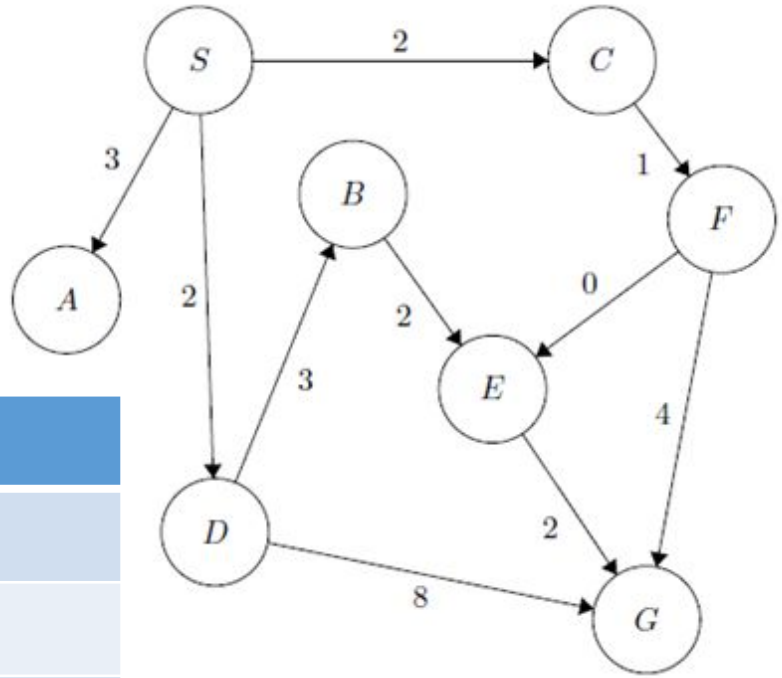
Step	Frontier	Expand	Explored: a set of Nodes
1	(S,0)	S	Ø
2	(S-C,2) (S-D,2) (S-A,3)	C	S

Traverse from start state (S) to Goal state (G)



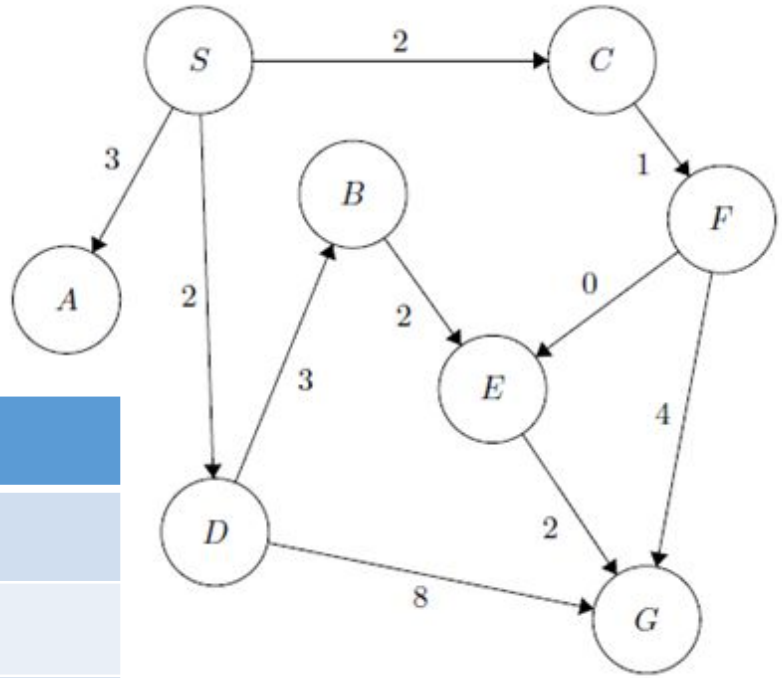
Step	Frontier	Expand	Explored: a set of Nodes
1	(S,0)	S	Ø
2	(S-C,2) (S-D,2) (S-A,3)	C	S
3	(S-D,2) (S-A,3)(S-C-F,3)	D	S,C

Traverse from start state (S) to Goal state (G)



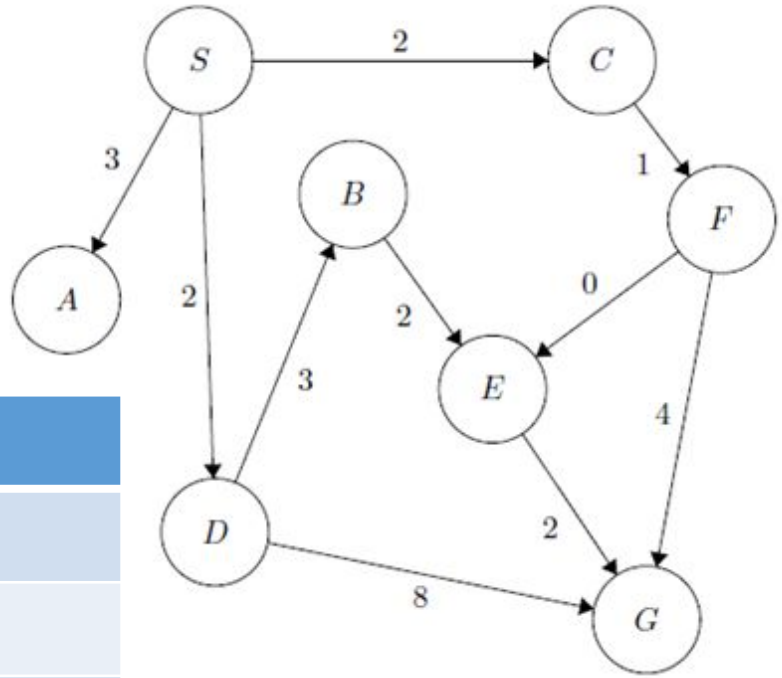
Step	Frontier	Expand	Explored: a set of Nodes
1	(S,0)	S	Ø
2	(S-C,2) (S-D,2) (S-A,3)	C	S
3	(S-D,2) (S-A,3)(S-C-F,3)	D	S,C
4	(S-A,3)(S-C-F,3)(S-D-B,5)(S-D-G,10)	A	S,C,D

Traverse from start state (S) to Goal state (G)



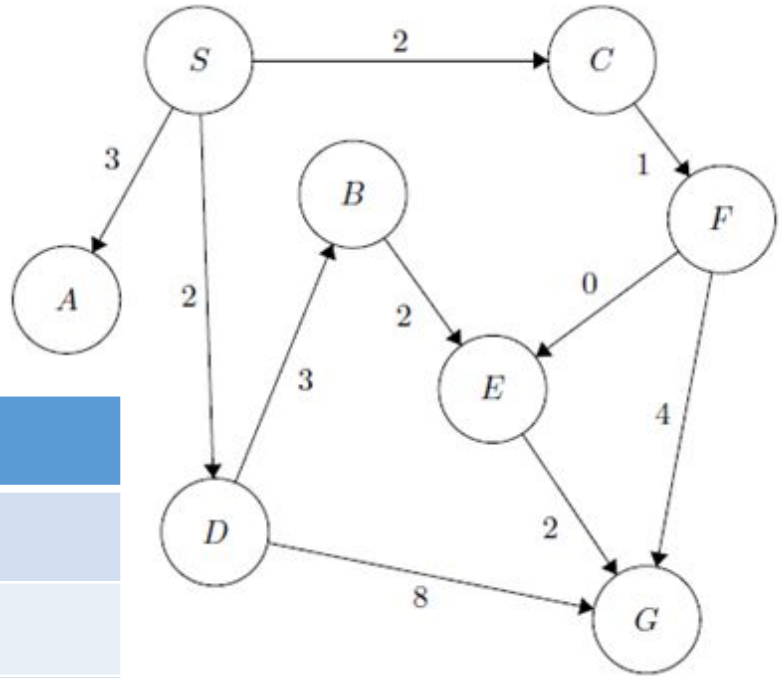
Step	Frontier	Expand	Explored: a set of Nodes
1	(S,0)	S	Ø
2	(S-C,2) (S-D,2) (S-A,3)	C	S
3	(S-D,2) (S-A,3)(S-C-F,3)	D	S,C
4	(S-A,3)(S-C-F,3)(S-D-B,5)(S-D-G,10)	A	S,C,D
5	(S-C-F,3)(S-D-B,5)(S-D-G,10)	F	S,C,D,A

Traverse from start state (S) to Goal state (G)



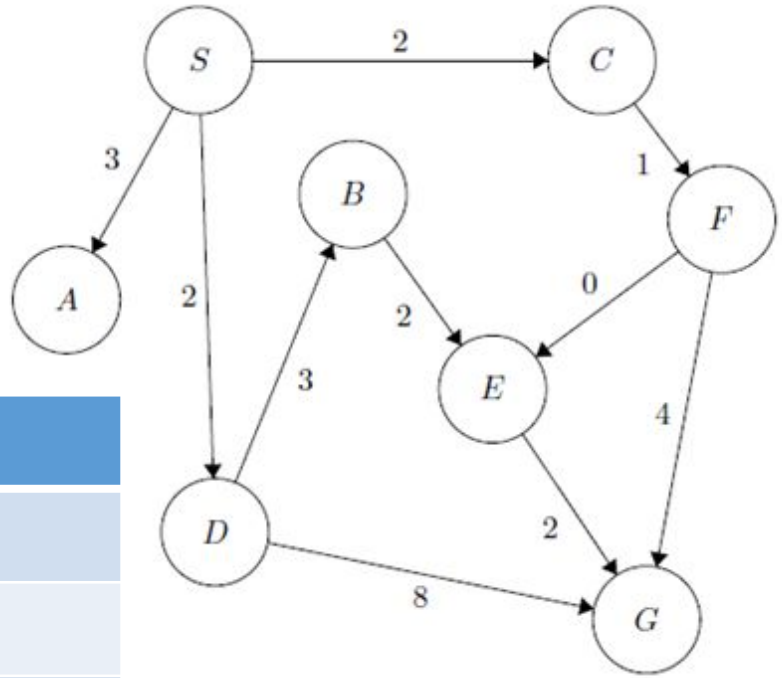
Step	Frontier	Expand	Explored: a set of Nodes
1	(S,0)	S	Ø
2	(S-C,2) (S-D,2) (S-A,3)	C	S
3	(S-D,2) (S-A,3)(S-C-F,3)	D	S,C
4	(S-A,3)(S-C-F,3)(S-D-B,5)(S-D-G,10)	A	S,C,D
5	(S-C-F,3)(S-D-B,5)(S-D-G,10)	F	S,C,D,A
6	(S-D-B,5)(S-D-G,10)(S-C-F-E,3)(S-C-F-G,7)	E	S,C,D,A,F

Traverse from start state (S) to Goal state (G)



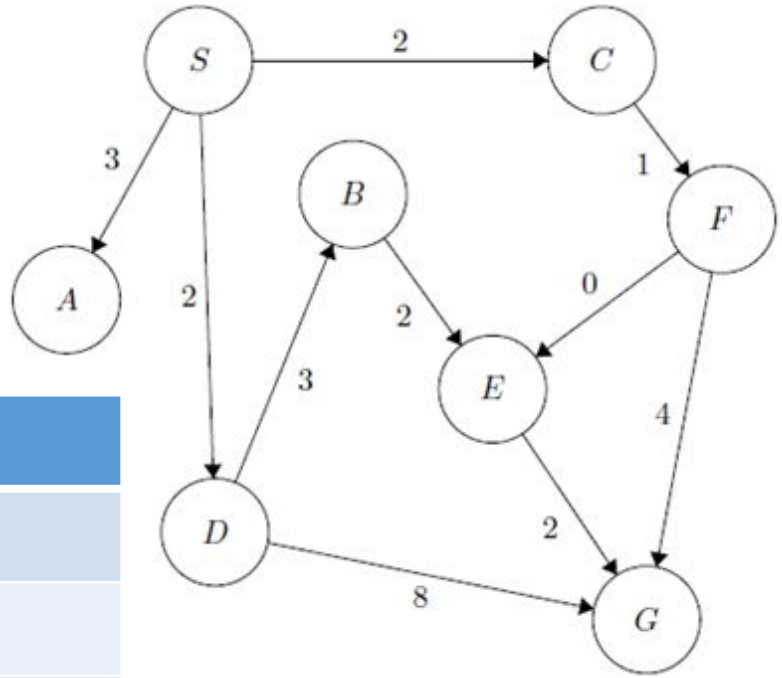
Step	Frontier	Expand	Explored: a set of Nodes
1	(S,0)	S	Ø
2	(S-C,2) (S-D,2) (S-A,3)	C	S
3	(S-D,2) (S-A,3)(S-C-F,3)	D	S,C
4	(S-A,3)(S-C-F,3)(S-D-B,5)(S-D-G,10)	A	S,C,D
5	(S-C-F,3)(S-D-B,5)(S-D-G,10)	F	S,C,D,A
6	(S-D-B,5)(S-D-G,10)(S-C-F-E,3)(S-C-F-G,7)	E	S,C,D,A,F
7	(S-D-B,5)(S-D-G,10) (S-C-F-G,7) (S-C-F-E-G,5)	B	S,C,D,A,F,E

Traverse from start state (S) to Goal state (G)



Step	Frontier	Expand	Explored: a set of Nodes
1	(S,0)	S	Ø
2	(S-C,2) (S-D,2) (S-A,3)	C	S
3	(S-D,2) (S-A,3)(S-C-F,3)	D	S,C
4	(S-A,3)(S-C-F,3)(S-D-B,5)(S-D-G,10)	A	S,C,D
5	(S-C-F,3)(S-D-B,5)(S-D-G,10)	F	S,C,D,A
6	(S-D-B,5)(S-D-G,10)(S-C-F-E,3)(S-C-F-G,7)	E	S,C,D,A,F
7	(S-D-B,5)(S-D-G,10) (S-C-F-G,7) (S-C-F-E-G,5)	B	S,C,D,A,F,E
8	(S-D-G,10) (S-C-F-G,7) (S-C-F-E-G,5)~~(S-D-B-E,7)~~	G	S,C,D,A,F,E,B

Traverse from start state (S) to Goal state (G)



Step	Frontier	Expand	Explored: a set of Nodes
1	(S,0)	S	Ø
2	(S-C,2) (S-D,2) (S-A,3)	C	S
3	(S-D,2) (S-A,3)(S-C-F,3)	D	S,C
4	(S-A,3)(S-C-F,3)(S-D-B,5)(S-D-G,10)	A	S,C,D
5	(S-C-F,3)(S-D-B,5)(S-D-G,10)	F	S,C,D,A
6	(S-D-B,5)(S-D-G,10)(S-C-F-E,3)(S-C-F-G,7)	E	S,C,D,A,F
7	(S-D-B,5)(S-D-G,10) (S-C-F-G,7) (S-C-F-E-G,5)	B	S,C,D,A,F,E
8	(S-D-G,10) (S-C-F-G,7) (S-C-F-E-G,5)~~(S-D-B-E,7)~~	G	S,C,D,A,F,E,B
9	(S-C-F-E-G,5) Goal Found		

Bidirectional Search

- Idea
  - simultaneously search forward from S and backwards from G
  - stop when both “meet in the middle”
  - need to keep track of the intersection of 2 open sets of nodes
- What does searching backwards from G mean
  - need a way to specify the predecessors of G
  - this can be difficult,
  - e.g., predecessors of checkmate in chess?
- what if there are multiple goal states?
  - what if there is only a goal test, no explicit list?
- Complexity
  - Time complexity is best: $O\left(2b^{(d/2)}\right) = O\left(b^{(d/2)}\right)$
  - memory complexity is the same

Bidirectional Search

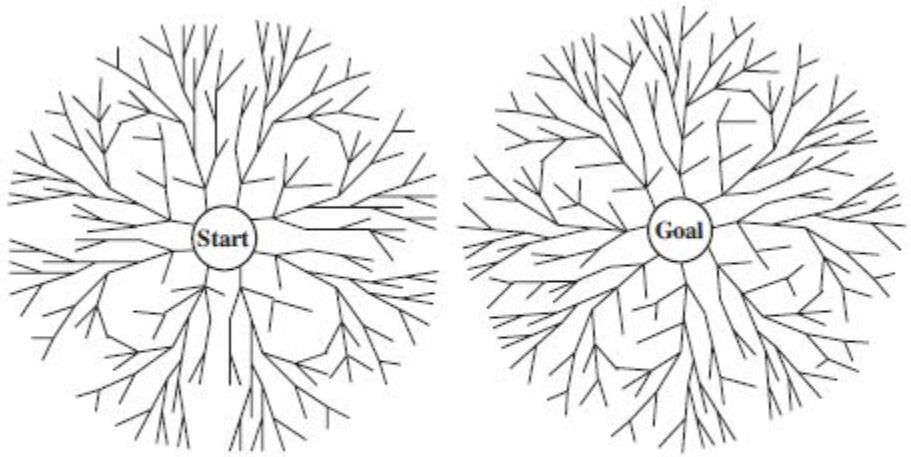


Figure 3.20 A schematic view of a bidirectional search that is about to succeed when a branch from the start node meets a branch from the goal node.

Summary of Algorithms

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes[a]	Yes[a,b]	No	No	Yes[a]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$
Optimal?	Yes[c]	Yes	No	No	Yes[c]

b branching factor

d depth of the shallowest solution

m maximum depth of the search tree

l depth limit

Superscripts:

a complete if b is finite

b complete if step costs ≥ epsilon for +ve epsilon

c optimal if step costs are all identical

Summary of Algorithms

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes[a]	Yes[a,b]	No	No	Yes[a]	Yes[a,d]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes[c]	Yes	No	No	Yes[c]	Yes[c,d]

Figure 3.21 Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: [a] complete if b is finite; [b] complete if step costs $\geq \epsilon$ for positive ϵ; [c] optimal if step costs are all identical; [d] if both directions use breadth-first search.